

CSE 4303

Data Structures

Topic: Segment Tree Range update
with Lazy Propagation Technique

Sabbir Ahmed

Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

Scenario

- Given a large integer array ($\sim 10^5$ *items*).
- Query for sum/max/min etc. of items within range (i, j) .
- Simple approach:
 - Run a loop starting from i and add all integers until you reach j .
 - Okay for a single query.

Scenario

- Given a large integer array ($\sim 10^5$ items).
- Query for sum/max/min etc. of items within range (i, j) .
- Simple approach:
 - Run a loop starting from i and add all integers until you reach j .
 - Okay for a single query.
- How to handle this for large number of Queries? ($\sim 10^5$)
- Time Complexity = $O(n^2)$ (Traverse 10^5 items for 10^5 queries in the worst case.)

Scenario

- Another solution: Remember the cumulative sum for each index.
- This can lead to $O(n)$ solution.

Scenario

- Another solution: Remember the cumulative sum for each index.
- This can lead to $O(n)$ solution.

```
a[] = {4, -9, 3, 7, 1, 0, 2}
pref[0] = 0
// pref[k] = sum of first k items

for(i=1; i<=n; i++)
    pref[i] = pref[i-1] + a[i-1]

//result
pref={0, 4, -5, -2, 5, 6, 6, 8}
```

```
Query(i, j)
    return pref[j] - pref[i-1]
```

Scenario

- Another solution: Remember the cumulative sum for each index.
- This can lead to $O(n)$ solution.

This won't work if we have to update the items frequently!

Segment Tree comes to the picture!

```
a[] = {4, -9, 3, 7, 1, 0, 2}
pref[0] = 0
// pref[k] = sum of first k items

for(i=1; i<=n; i++)
    pref[i] = pref[i-1] + a[i-1]

//result
pref={0, 4, -5, -2, 5, 6, 6, 8}
```

```
Query(i, j)
    return pref[j] - pref[i-1]
```

Segment Tree

- Answers Range Queries
- Provides sum/max/min of items in range (i, j) in $O(\log N)$

Segment Tree

- Answers Range Queries
- Provides sum/max/min of items in range (i, j) in $O(\log N)$
- Supports update operation:
 - Update a **particular value** between (i, j) in $O(\log N)$.
 - Update a **range of values** between (i, j) in $O(\log N)$ (uses a technique called Lazy-propagation).

Visualizing the array as tree

- Can be visualized as a **Conceptual tree**.
- Tree will be stored in an array (e.g., Heap).
- **Leaf nodes**: Individual values of the array.
- **Intermediate nodes**: result of queries on the left and Right subtree.

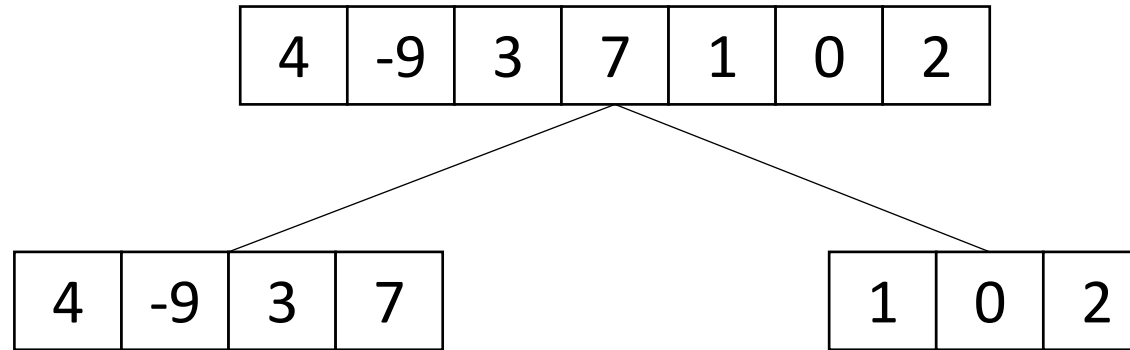
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Different parts of the Segment tree save results for different segments (ranges)!

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

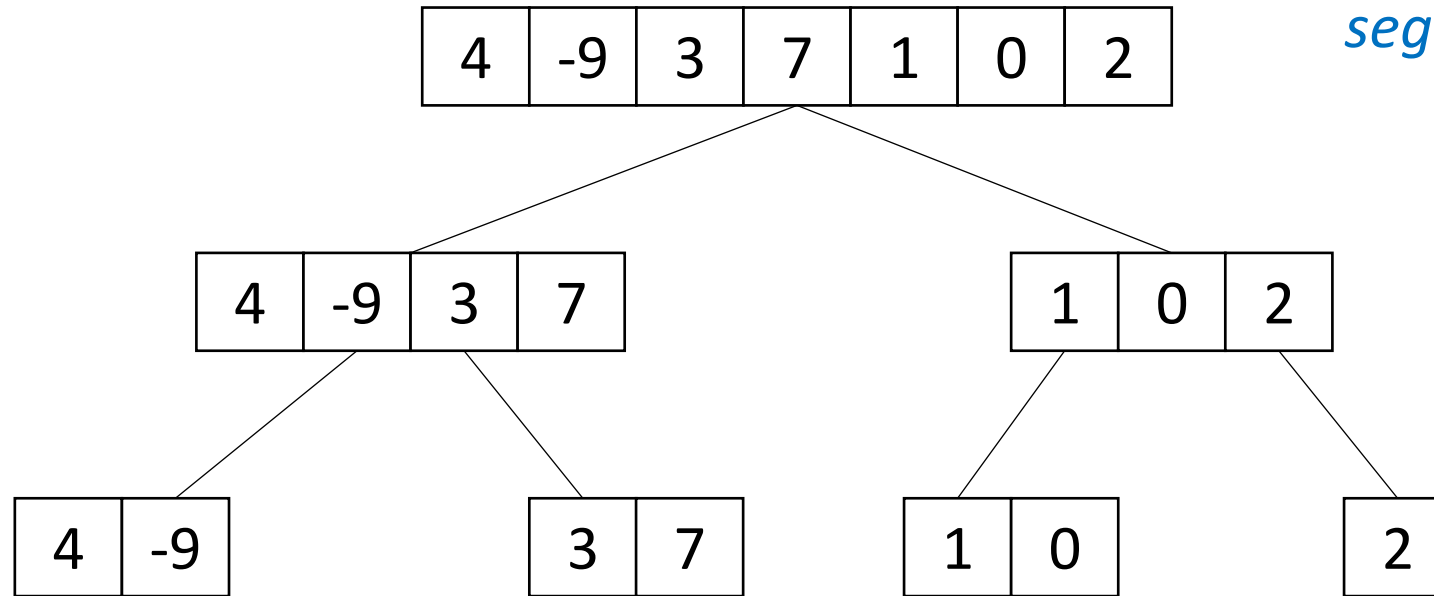


Different parts of the Segment tree save results for different segments (ranges)!

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

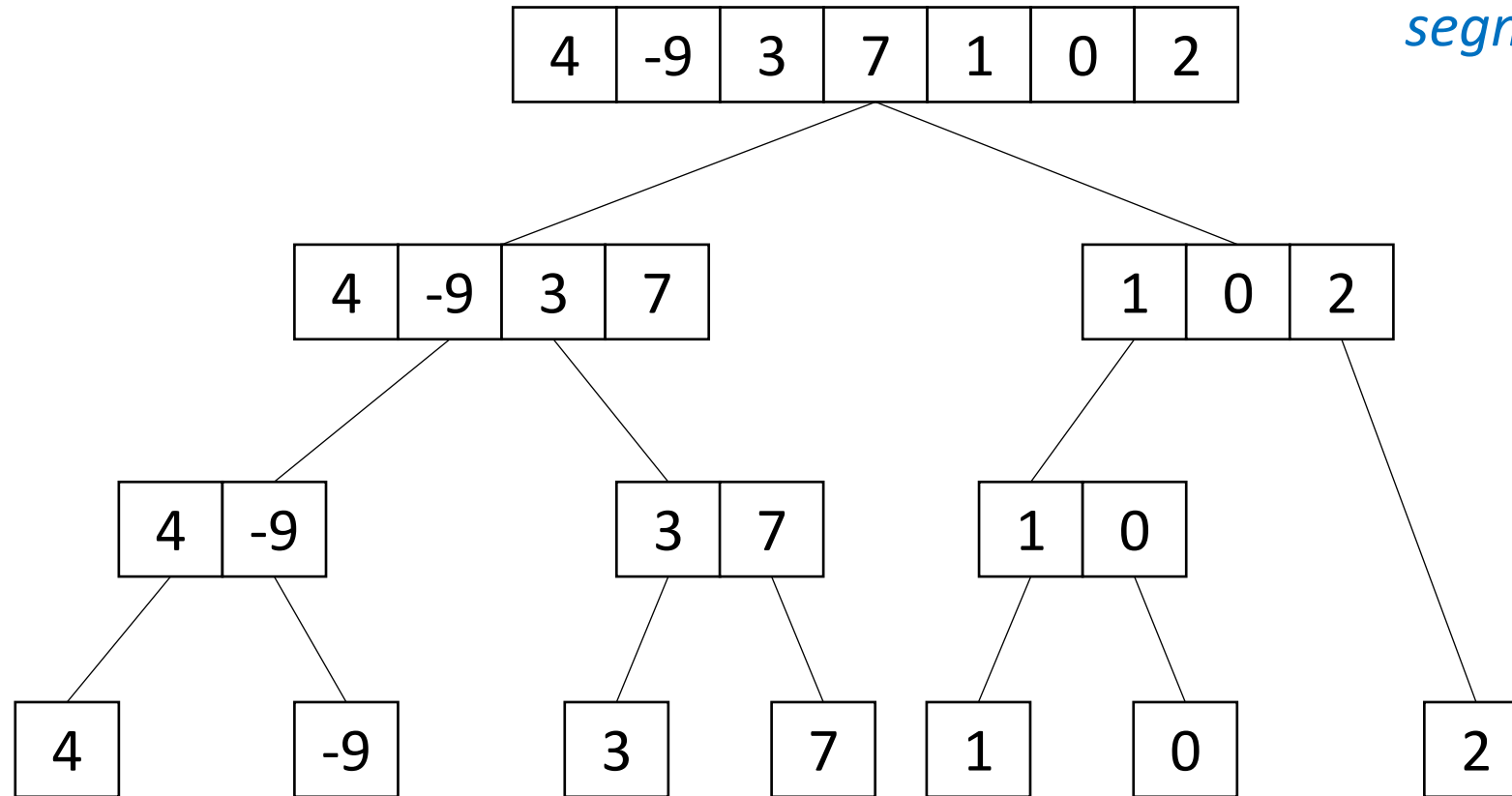
Different parts of the Segment tree save results for different segments (ranges)!



Input array (1-based index)

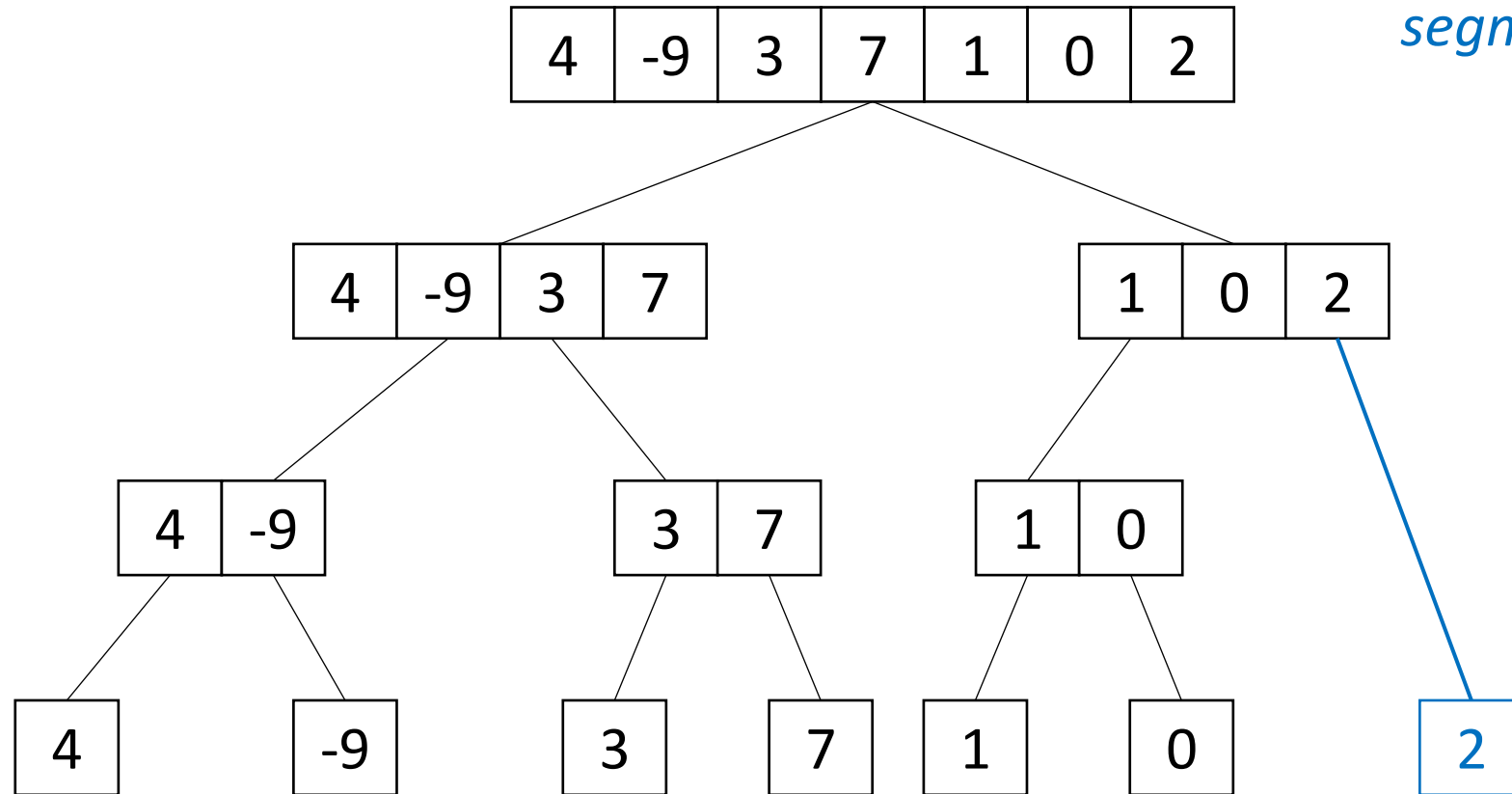
4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Different parts of the Segment tree save results for different segments (ranges)!



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7



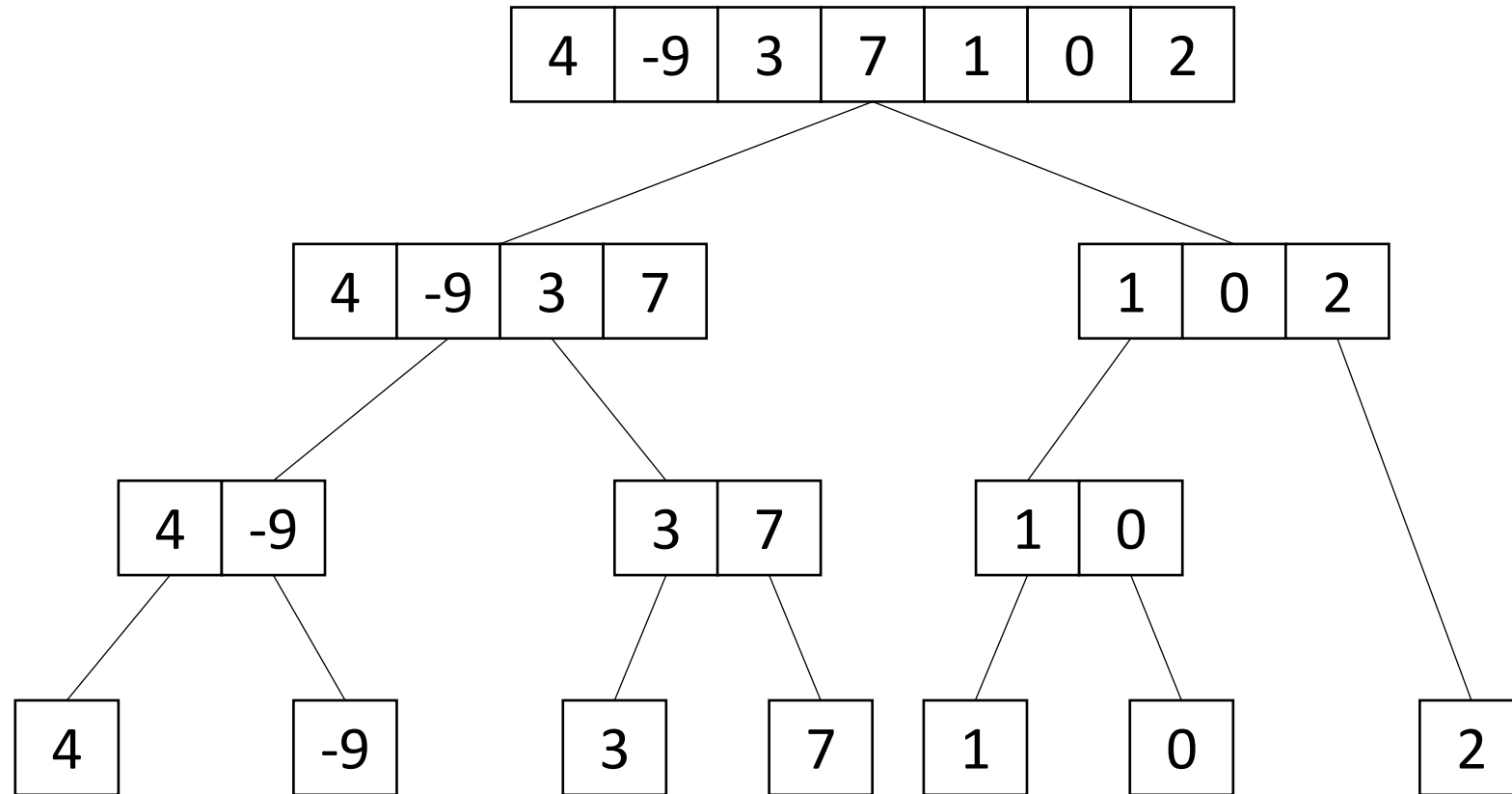
Different parts of the Segment tree save results for different segments (ranges)!

Due to off number of values in input array. (We'll see how the build function takes care of it)

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

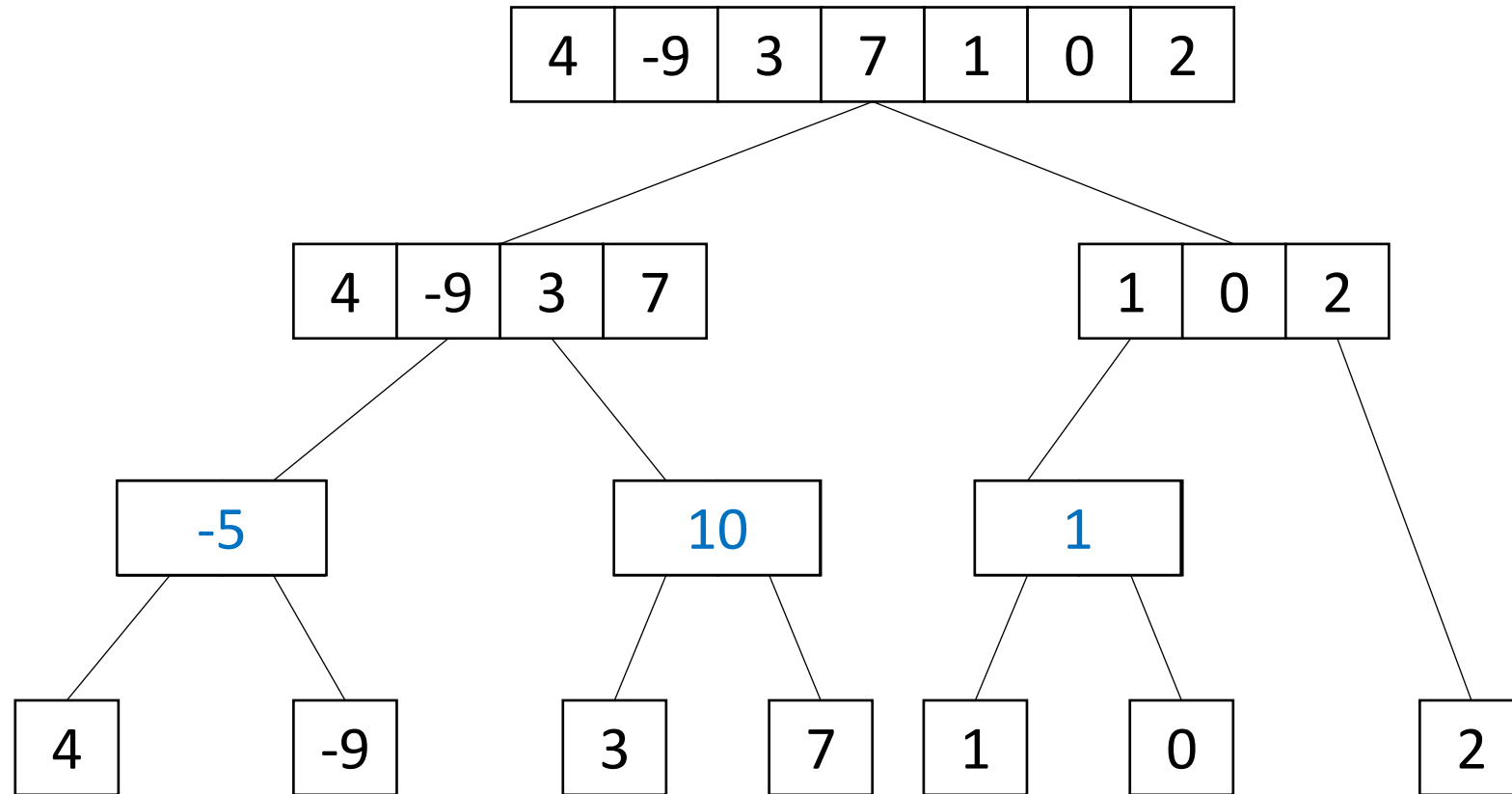
*Sum of a segment =
sum (left subtree + right subtree)*



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

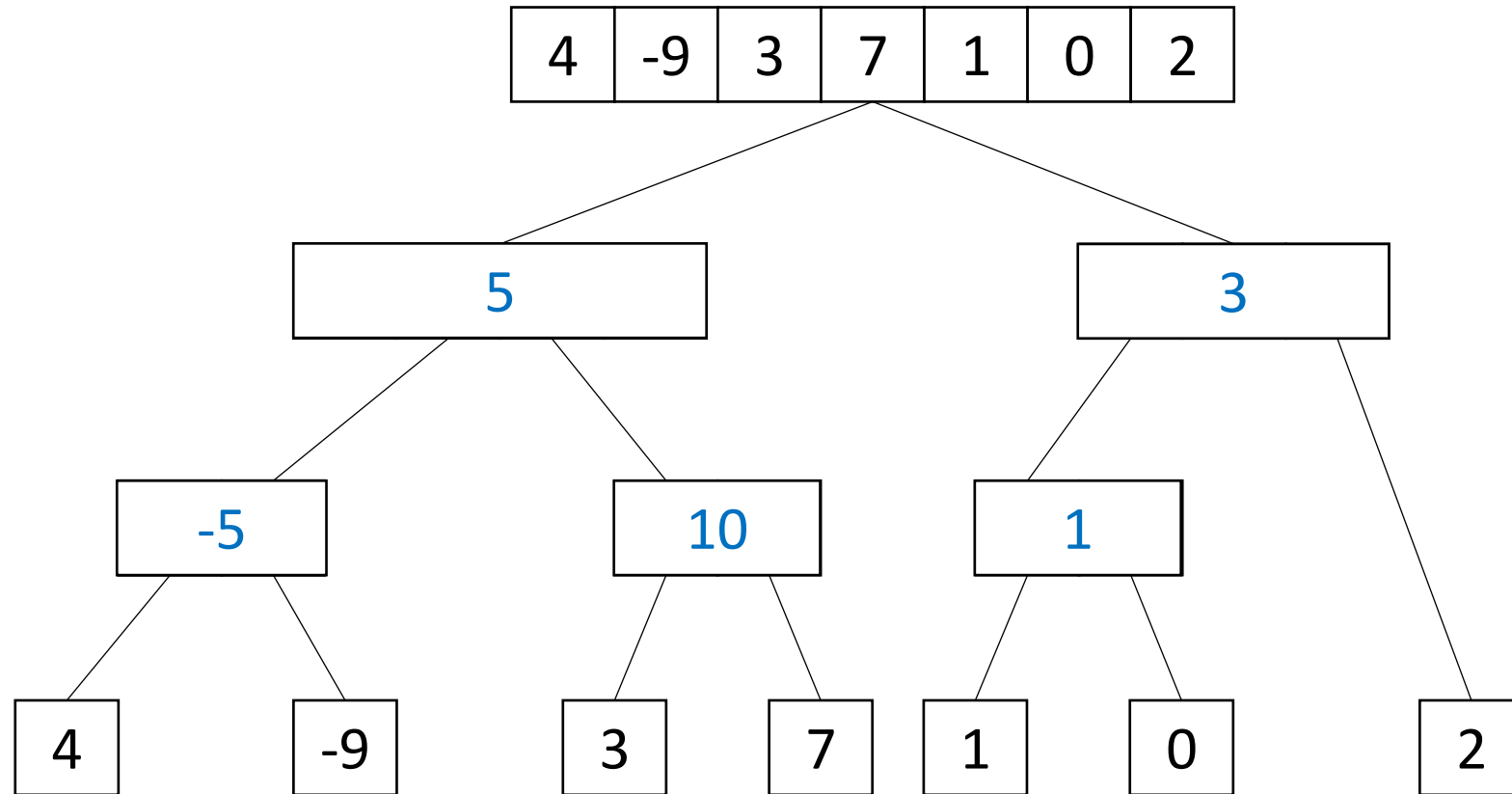
*Sum of a segment =
sum (left subtree + right subtree)*



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

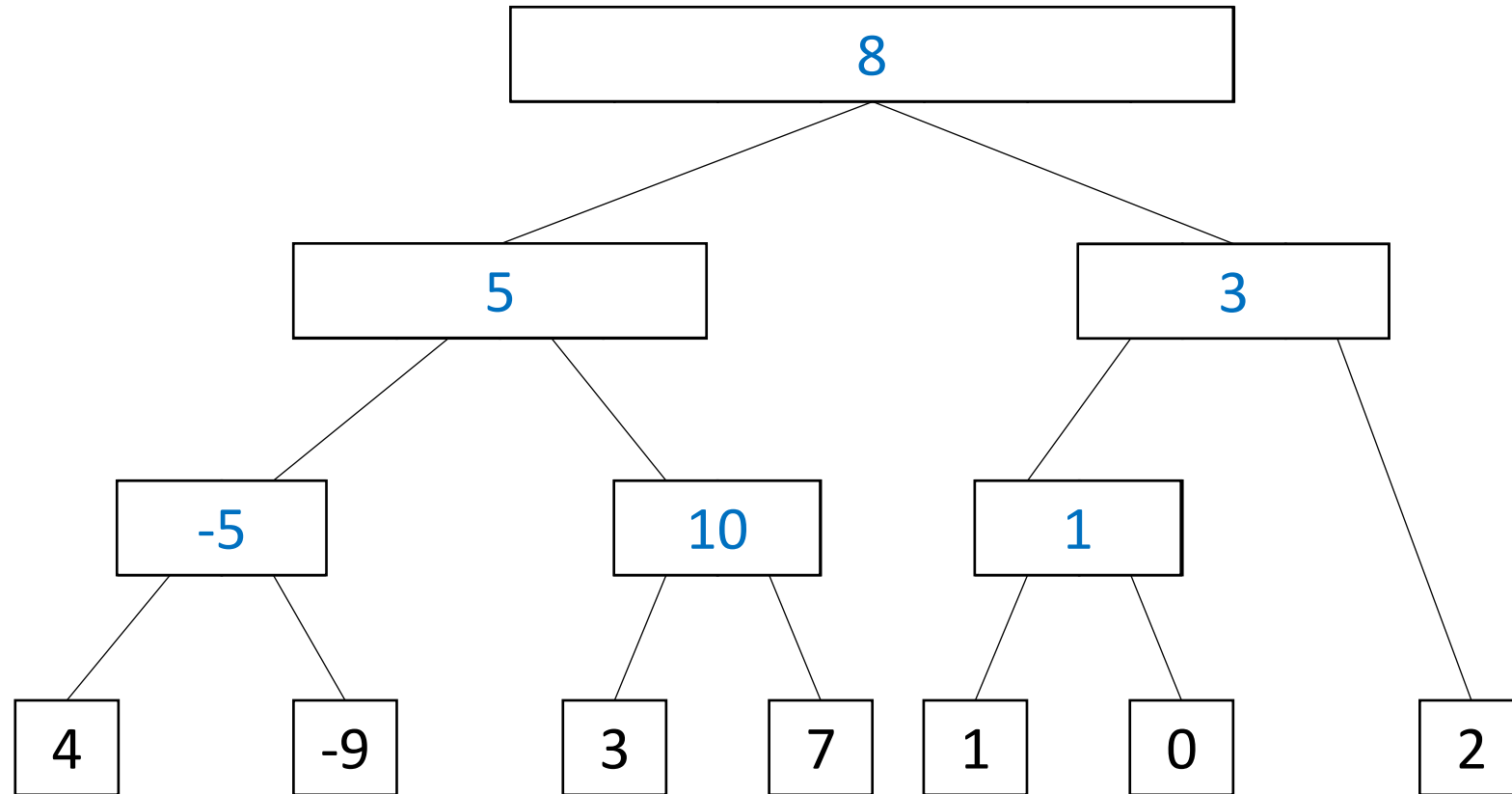
*Sum of a segment =
sum (left subtree + right subtree)*



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

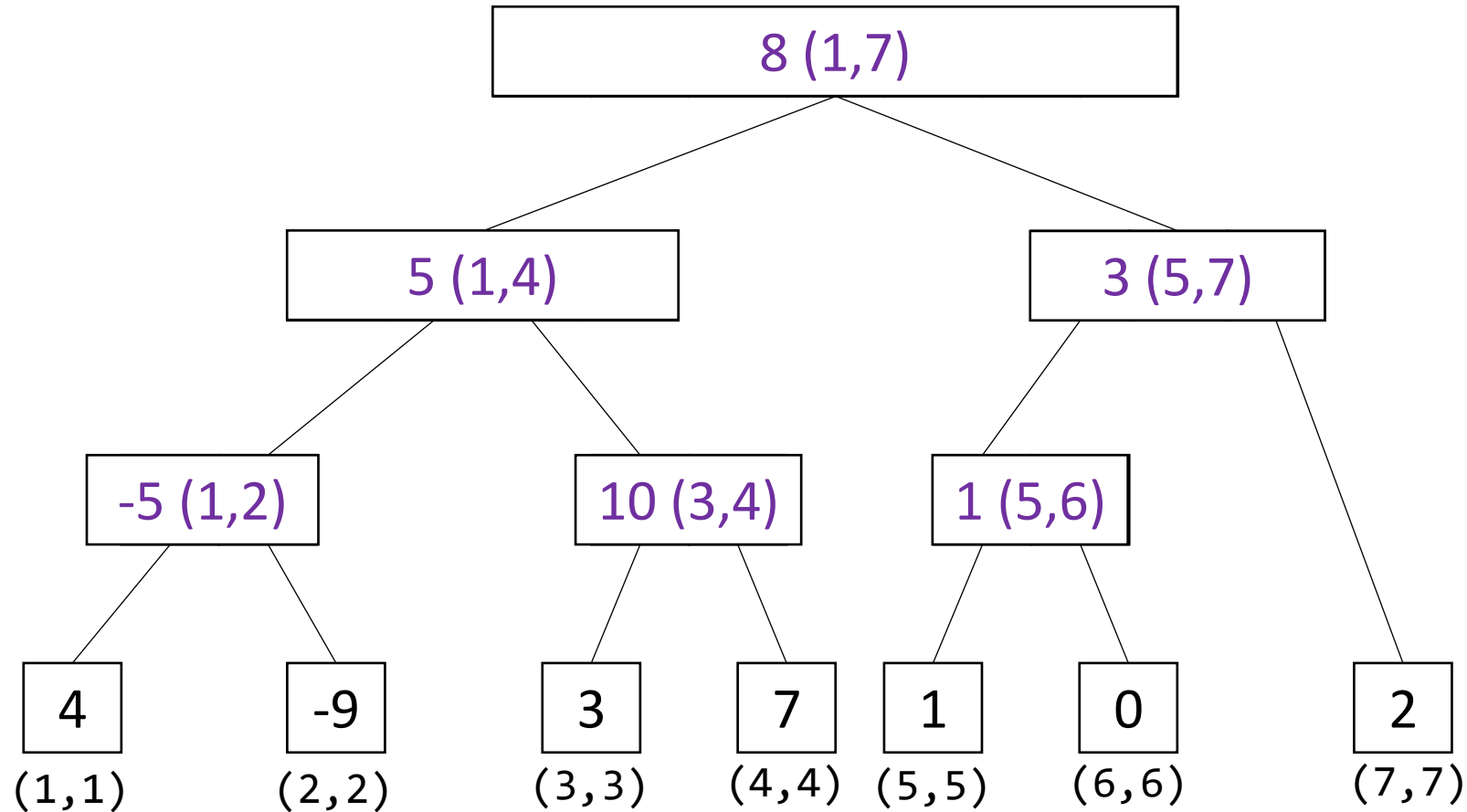
*Sum of a segment =
sum (left subtree + right subtree)*



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

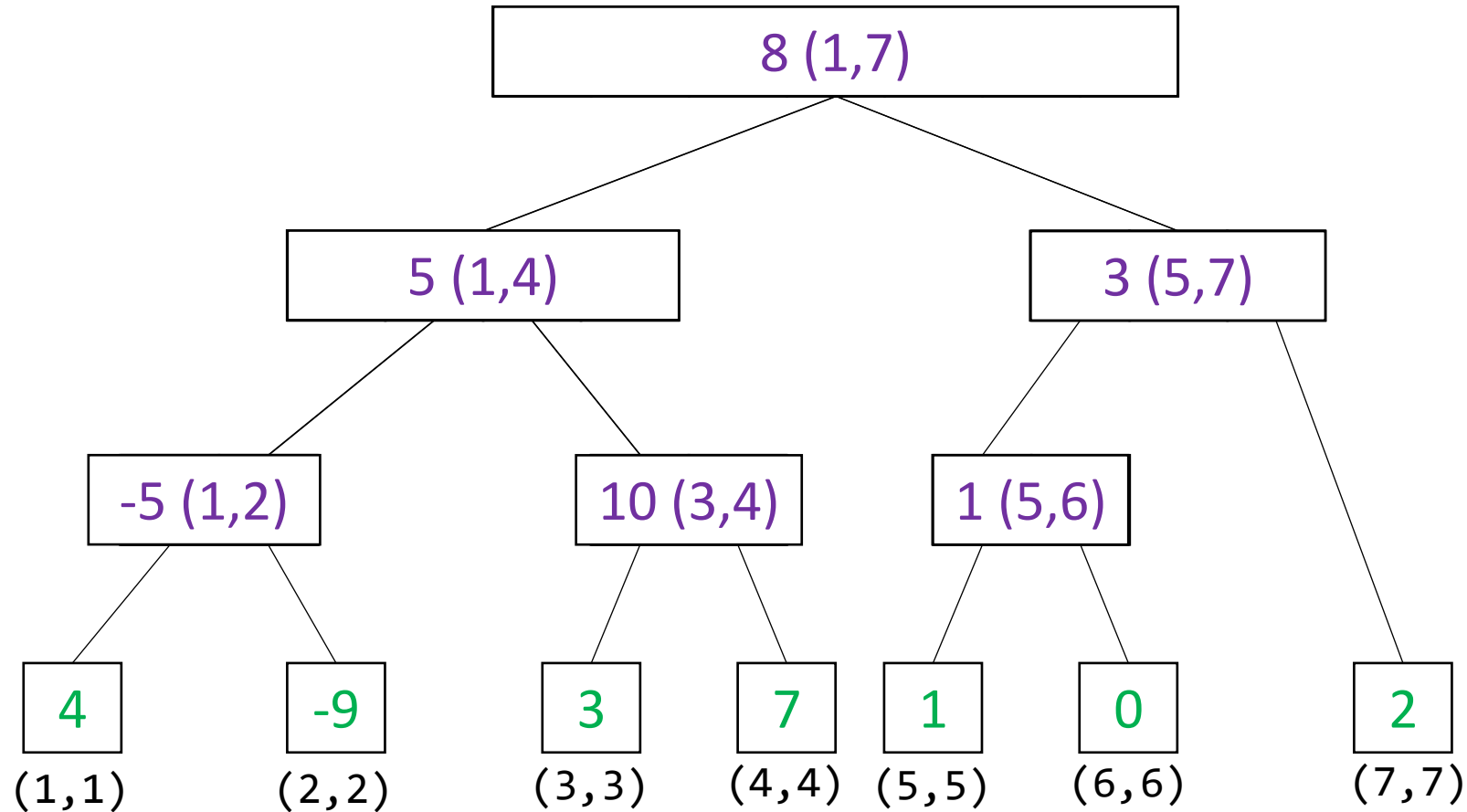
Each Internal node represents a range



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

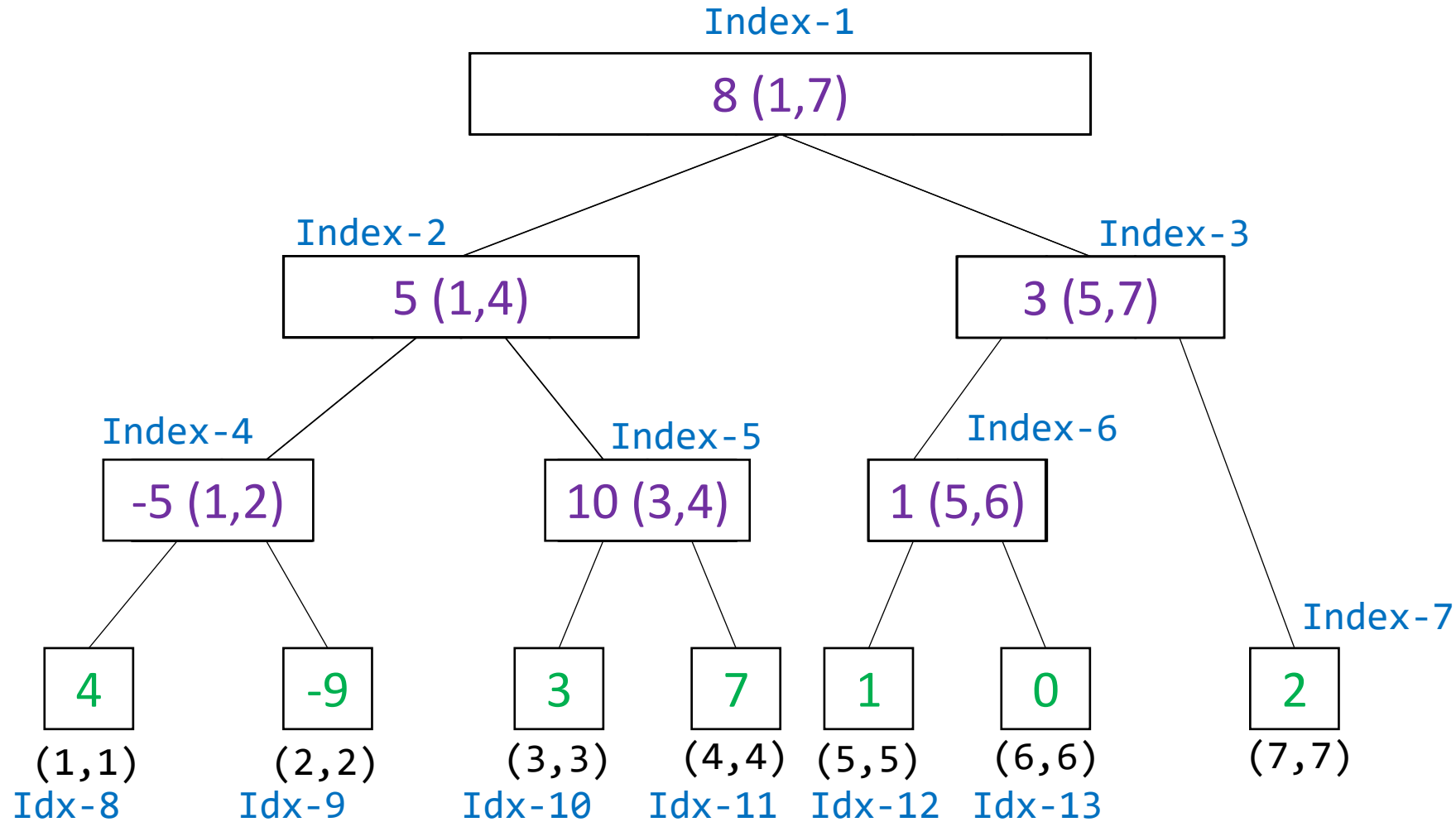
Each Internal node represents a range
Leaf nodes are values from input array



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

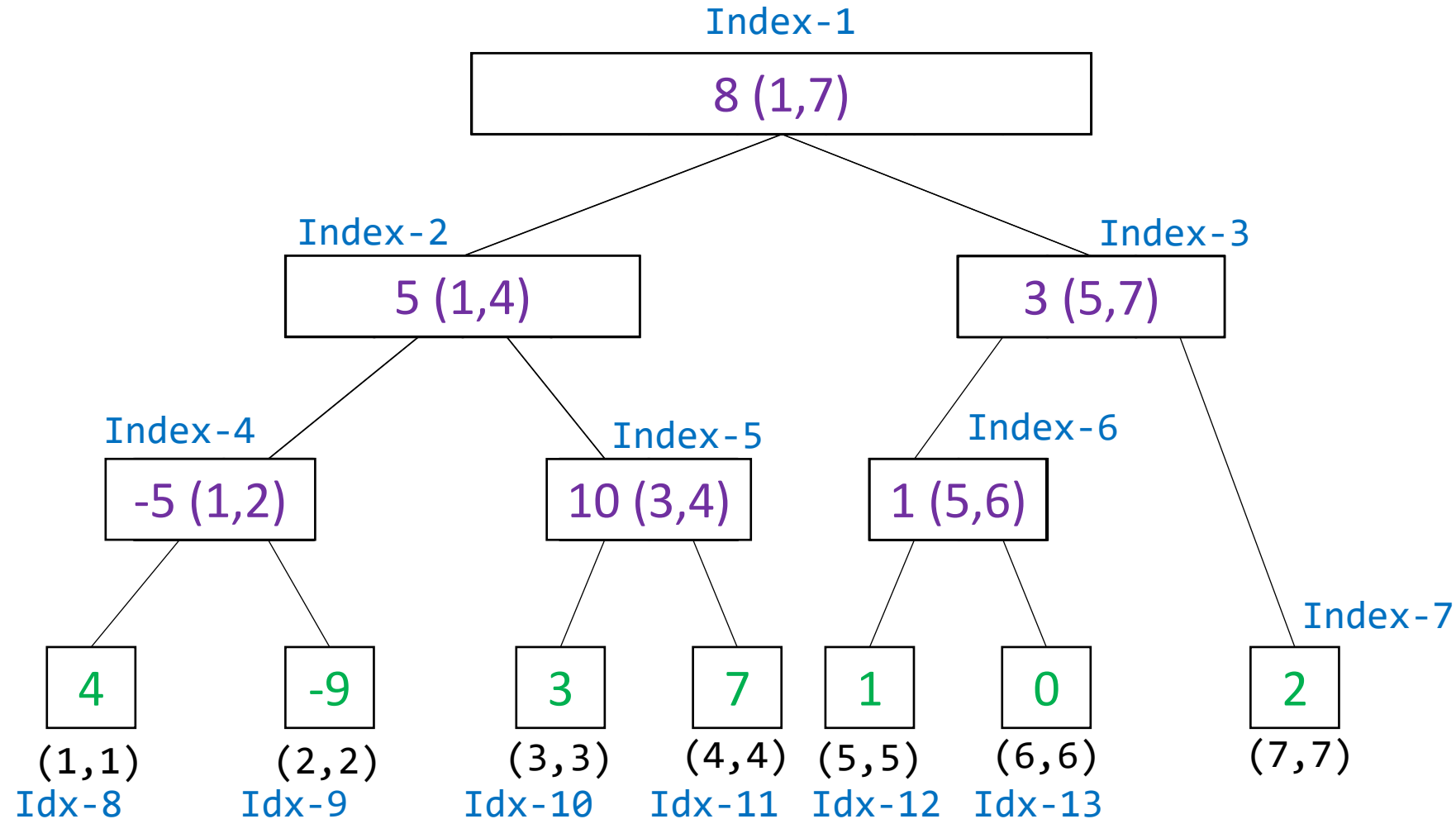
Each Internal node represents a range
Leaf nodes are values from input array



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Each Internal node represents a range
Leaf nodes are values from input array



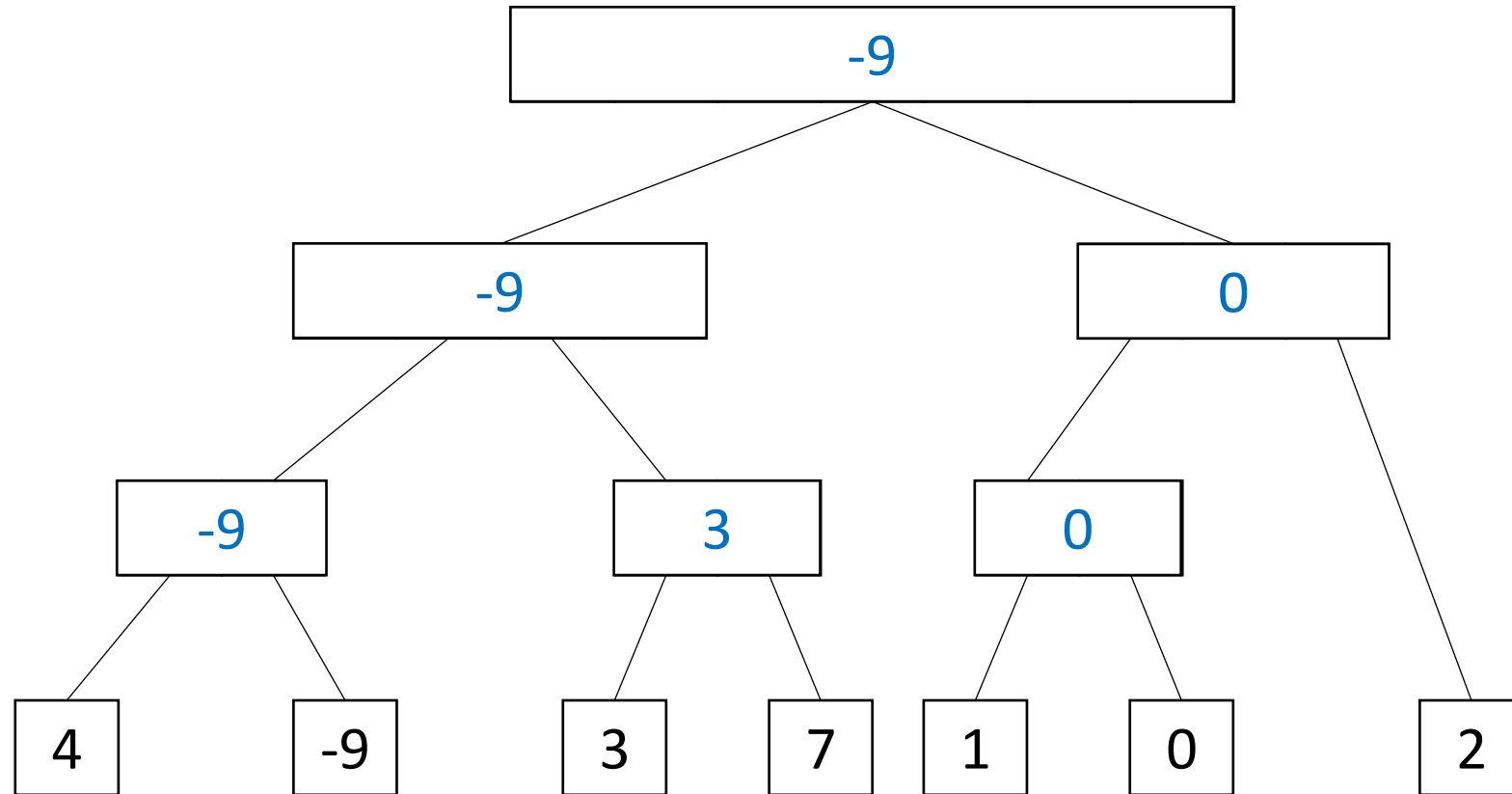
Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

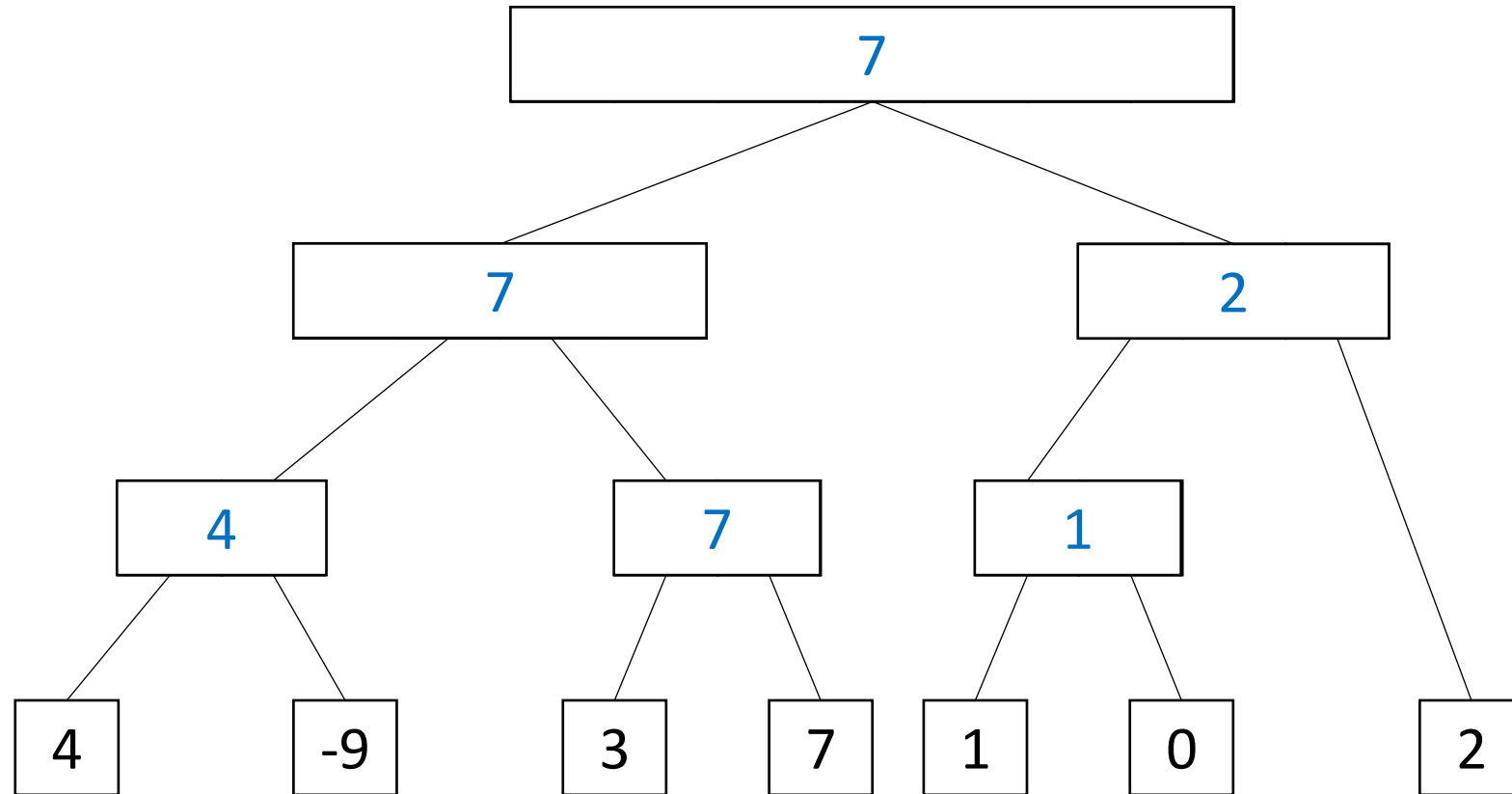
*min of a segment =
min (left subtree + right subtree)*



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

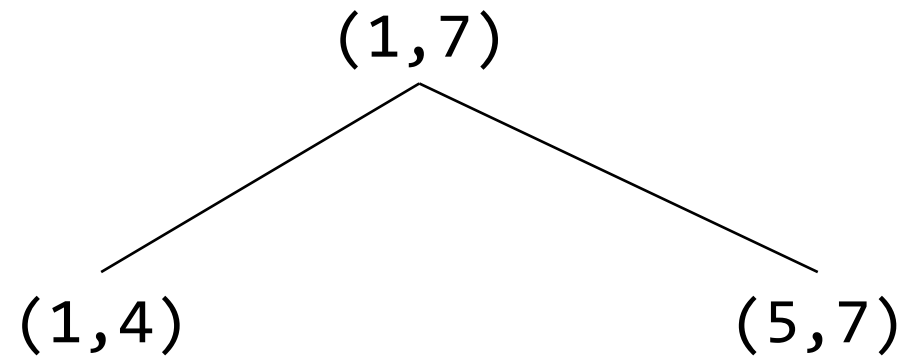
*max of a segment =
max (left subtree + right subtree)*



Resulting Segment Tree

8	5	3	5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

- $leftchild = parent \times 2$
- $rightchild = (parent \times 2) + 1$
- $parent = floor(child \div 2)$
- Conceptual Binary Tree. Can be stored in an Array.



How to form the segments?

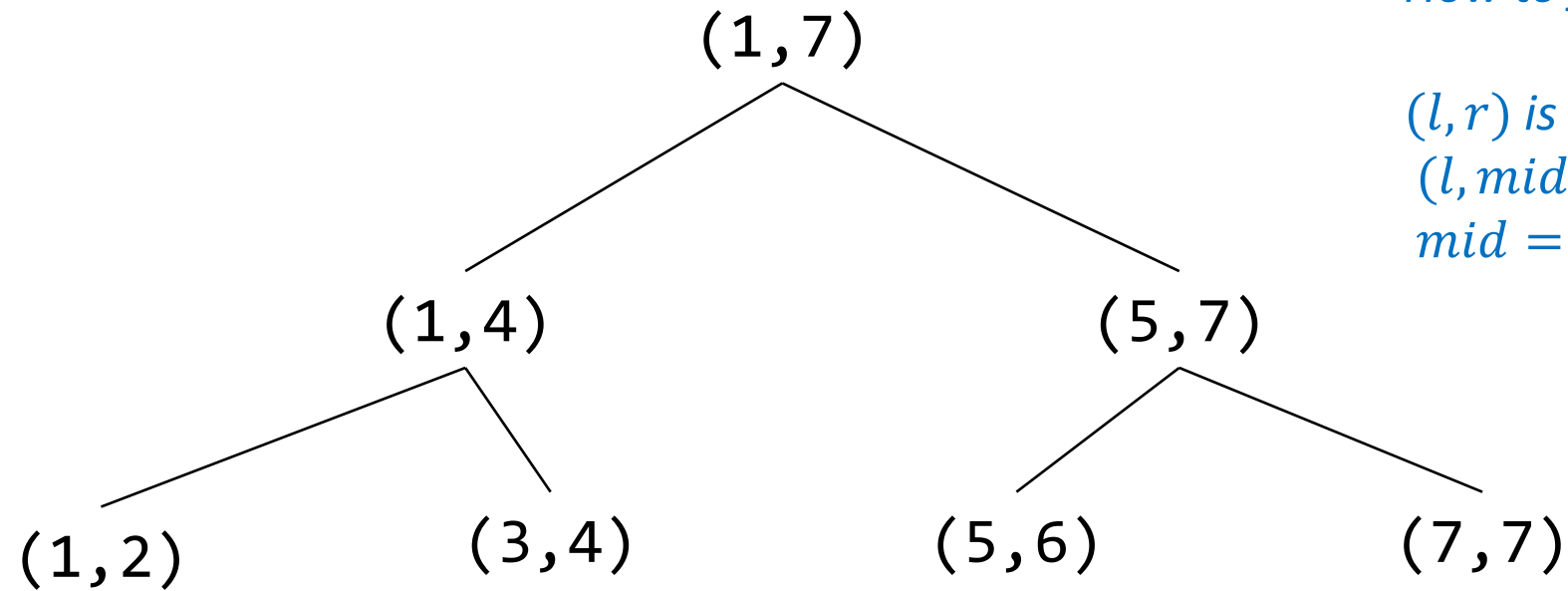
(l, r) is segmented to:

(l, mid) & $(mid + 1, r)$

$mid = (l + r)/2$

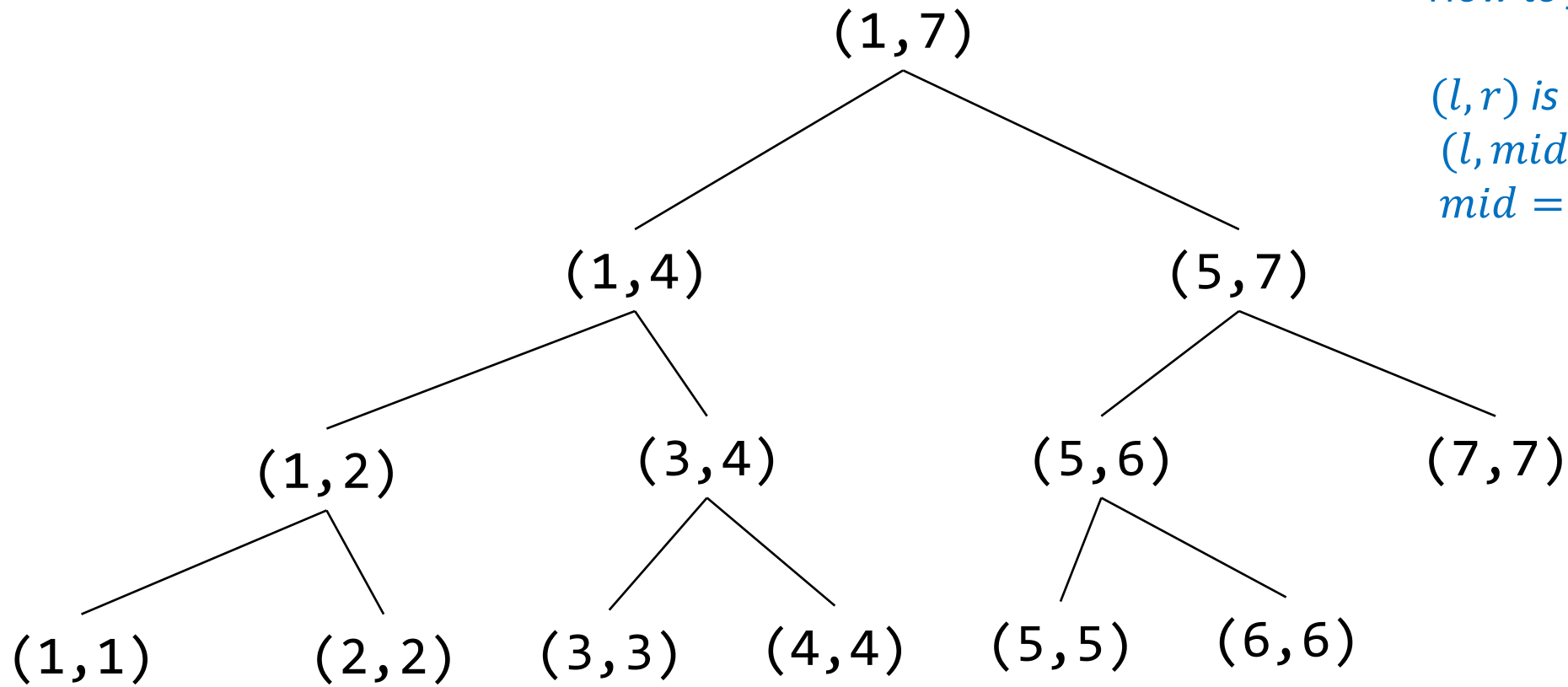
How to form the segments?

*(l, r) is segmented to:
 (l, mid) & $(mid + 1, r)$
 $mid = (l + r) / 2$*



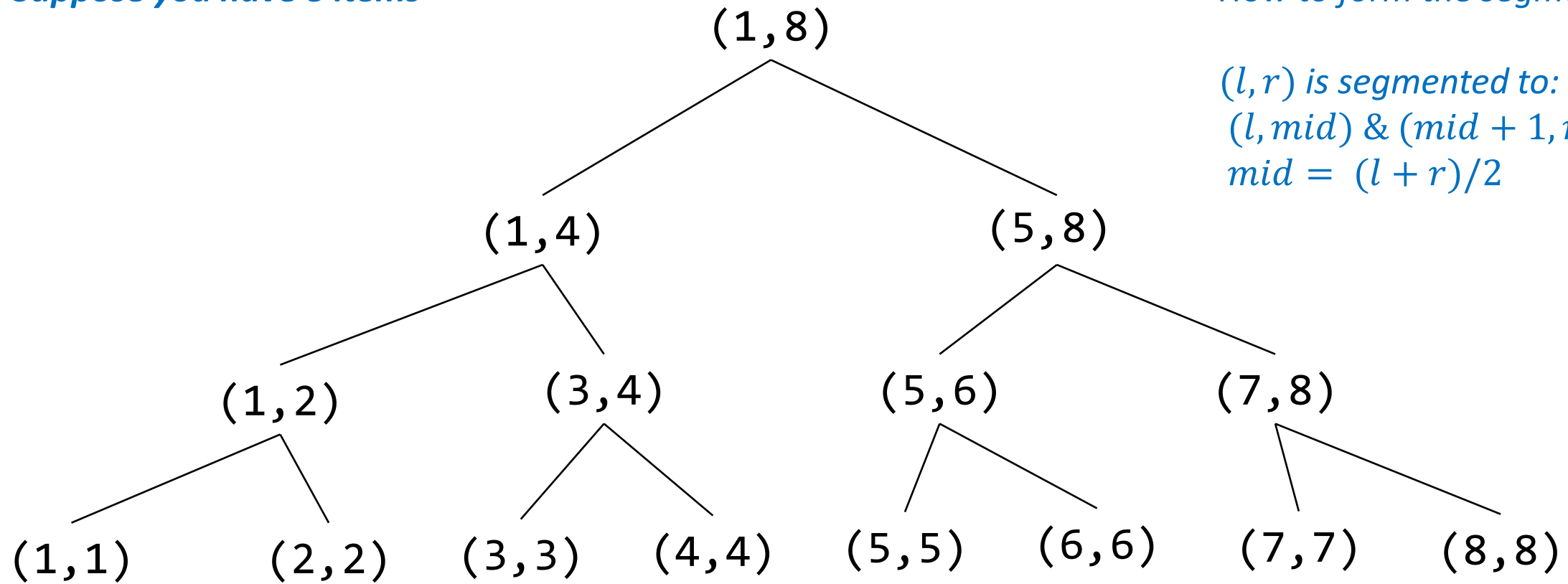
How to form the segments?

*(l, r) is segmented to:
 (l, mid) & $(mid + 1, r)$
 $mid = (l + r) / 2$*



Suppose you have 8 items

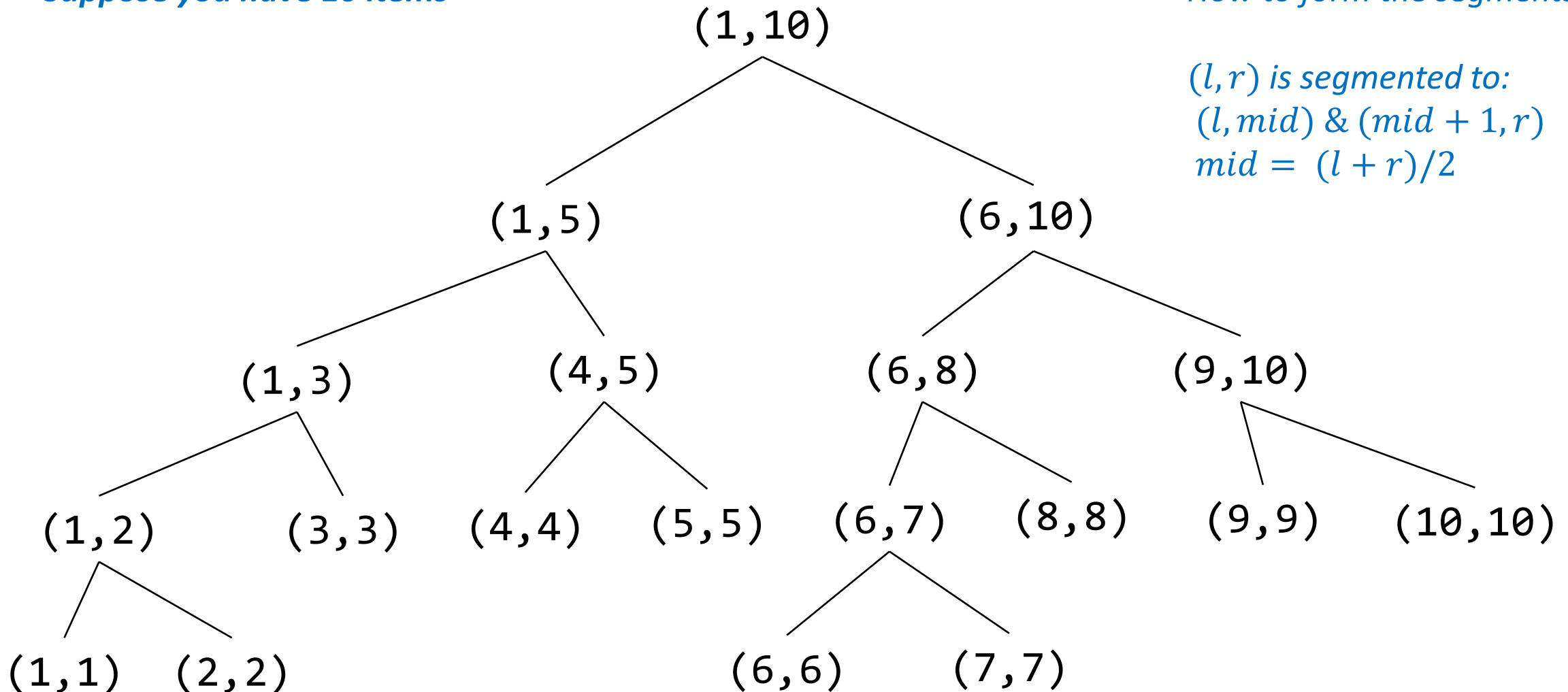
How to form the segments?



Suppose you have 10 items

How to form the segments?

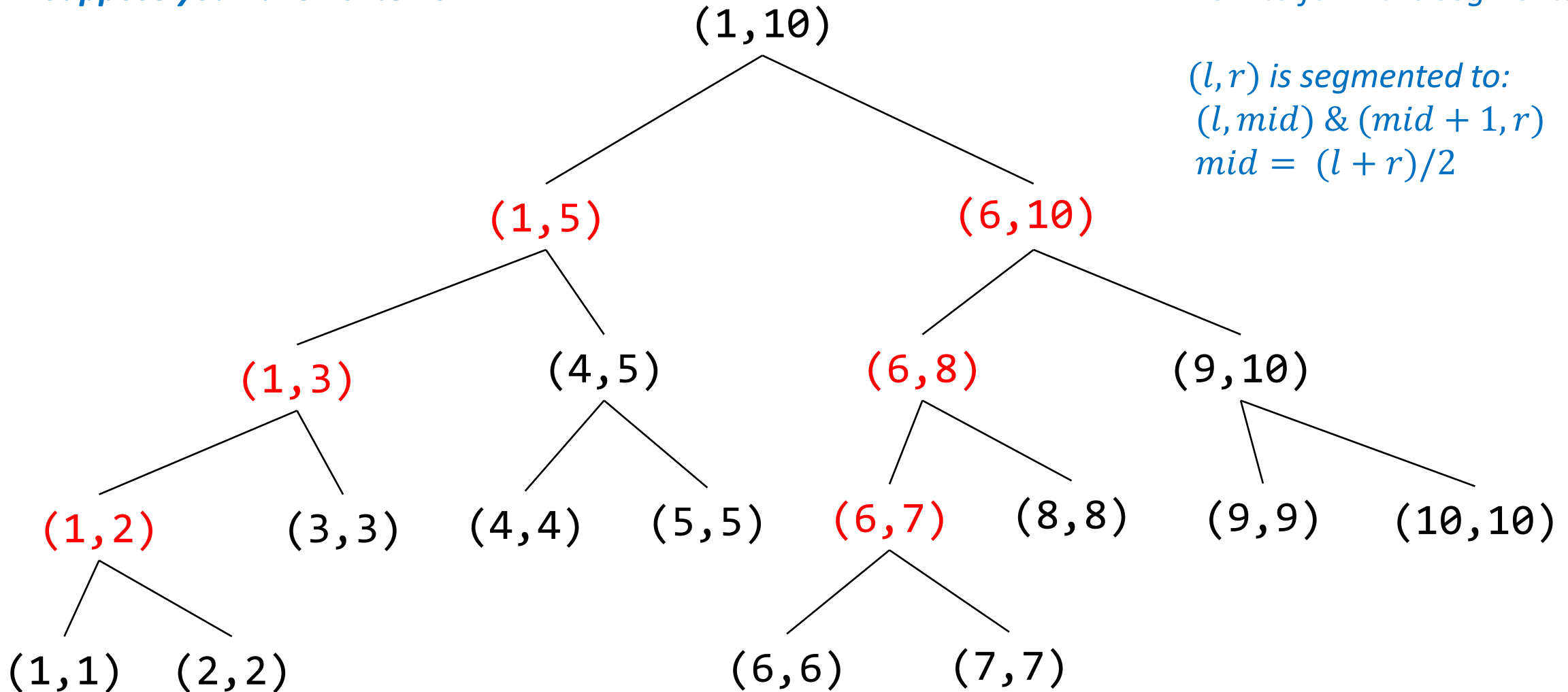
(l, r) is segmented to:
 (l, mid) & $(mid + 1, r)$
 $mid = (l + r) / 2$



Suppose you have 10 items

How to form the segments?

(l, r) is segmented to:
 (l, mid) & $(mid + 1, r)$
 $mid = (l + r) / 2$



Caution: the segments are not random!
Need to follow (l, mid) , $(mid + 1, r)$

How to build the tree?

Observation:

- Leaf Nodes = values from the original array.
- Internal nodes = result of the queries on left & right subtree.


```
1 void init ( int node, int begin, int end) {  
2     if (begin == end)  
3         // leaf node found. Insert directly.  
4         tree[node] = arr[begin]  
5         return;  
  
6  
7  
8  
9     init(  )  
10    init(  )  
  
11  
12 }
```

```
1 void init ( int node, int begin, int end) {  
2     If (begin == end)  
3         // leaf node found. Insert directly.  
4         tree[node] = arr[begin]  
5         return;  
  
6     int left = node × 2  
7     int right = (node × 2) + 1  
8     int mid = (begin + end) / 2  
  
9     init()  
10    init()  
  
11  
12 }
```

```
1 void init ( int node, int begin, int end) {  
2     If (begin == end)  
3         // leaf node found. Insert directly.  
4         tree[node] = arr[begin]  
5         return;  
  
6     int left = node × 2  
7     int right = (node × 2) + 1  
8     int mid = (begin + end) / 2  
  
9     init(left, begin, mid)  
10    init()  
  
11  
12 }
```

```
1 void init ( int node, int begin, int end) {  
2     If (begin == end)  
3         // leaf node found. Insert directly.  
4         tree[node] = arr[begin]  
5         return;  
  
6     int left = node × 2  
7     int right = (node × 2) + 1  
8     int mid = (begin + end) / 2  
  
9     init(left, begin, mid)  
10    init(right, mid+1, end)  
  
11  
12 }
```

```
1 void init ( int node, int begin, int end) {  
2     If (begin == end)  
3         // leaf node found. Insert directly.  
4         tree[node] = arr[begin]  
5         return;  
  
6     int left = node × 2  
7     int right = (node × 2) + 1  
8     int mid = (begin + end) / 2  
  
9     init(left, begin, mid)  
10    init(right, mid+1, end)  
  
11    tree[node] = tree[left] + tree[right]  
  
12 }
```

*This line will change as per the type
of query*

Init(1, 1, 7) (call from main function)

f(1,1,7)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }
```

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

1	2	3	4	5	6	7	8	9	10	11	12	13

Init(2,1,4)
Init(1, 1, 7) (call from main function)

f(1,1,7)
 /
 f(2,1,4)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }
```

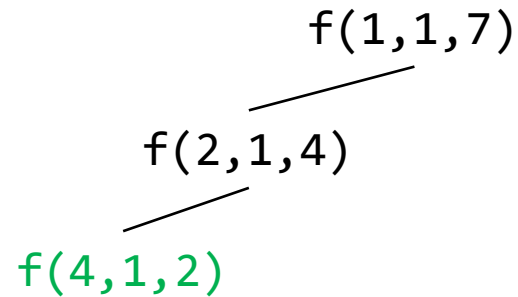
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

1	2	3	4	5	6	7	8	9	10	11	12	13

Init(4,1,2)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }
  
```

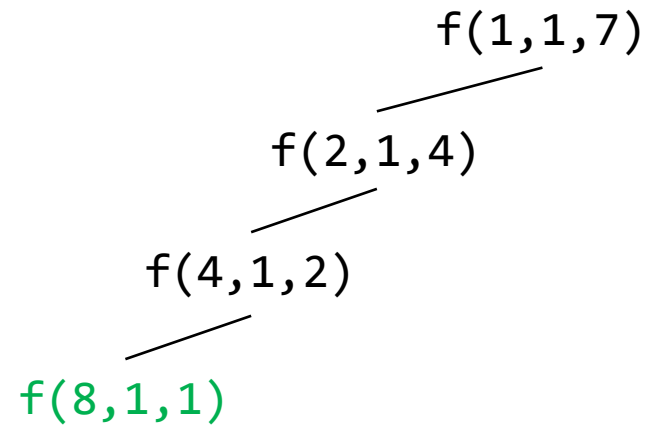
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

1	2	3	4	5	6	7	8	9	10	11	12	13

Init(8,1,1)
Init(4,1,2)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }

```

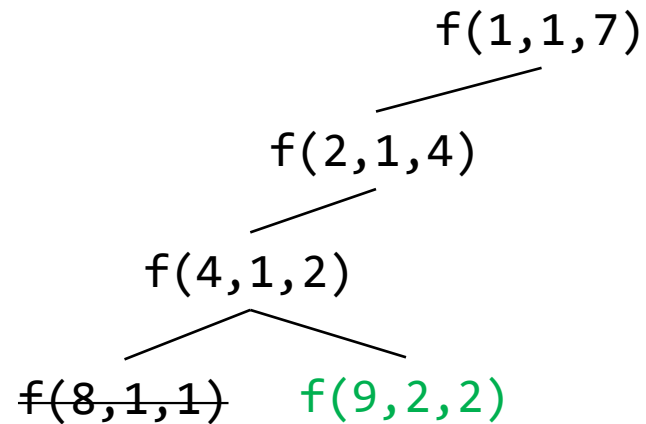
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

							4						
1	2	3	4	5	6	7	8	9	10	11	12	13	

Init(9,2,2)
Init(8,1,1)
Init(4,1,2)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }
  
```

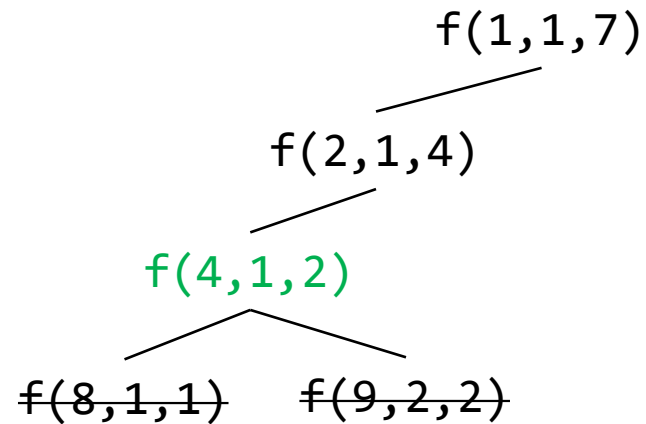
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

							4	-9				
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(9,2,2)
Init(8,1,1)
Init(4,1,2)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



Resulting Segment Tree

			-5				4	-9					
1	2	3	4	5	6	7	8	9	10	11	12	13	

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

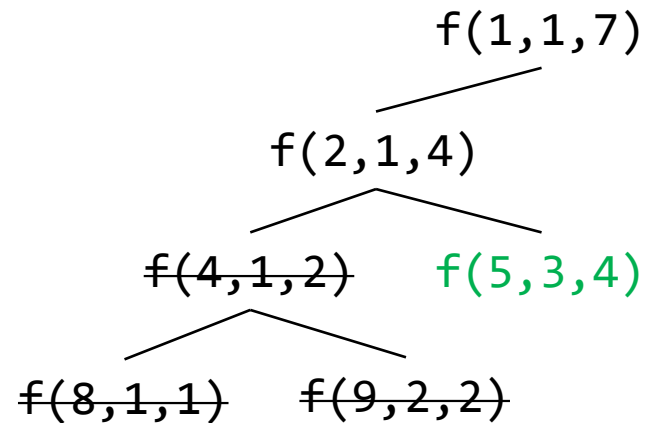
9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }
  
```

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Init(5,3,4)
Init(9,2,2)
Init(8,1,1)
Init(4,1,2)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



Resulting Segment Tree

			-5				4	-9					
1	2	3	4	5	6	7	8	9	10	11	12	13	

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]

12 }

```

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)

```

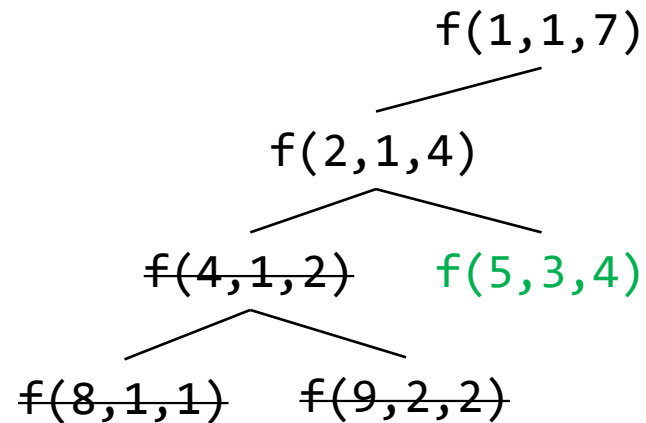
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

			-5				4	-9				
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(10,3,3)
Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)

```

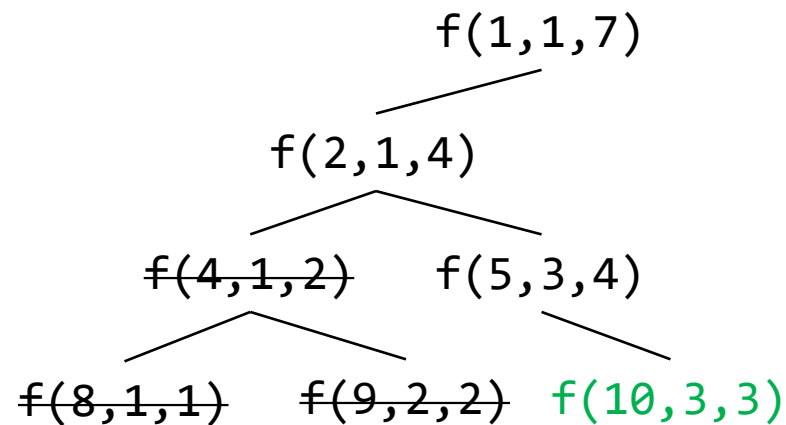
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

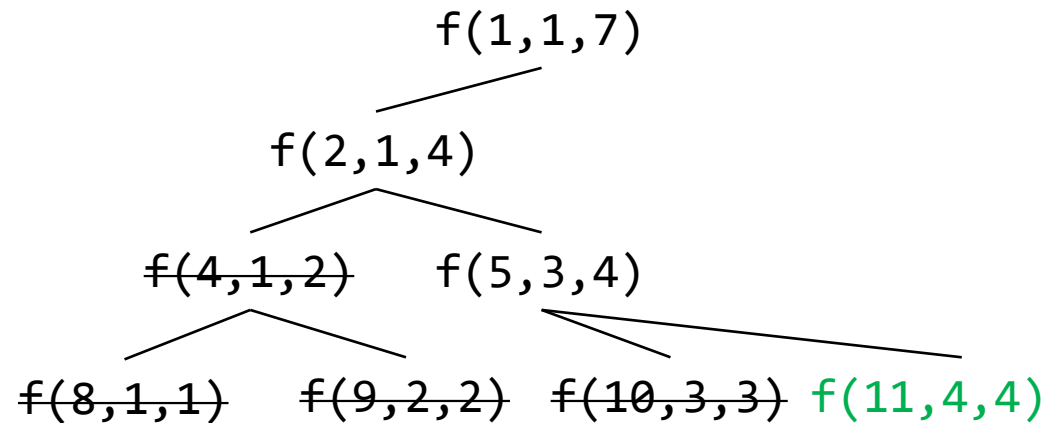
			-5				4	-9	3			
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(11,4,4)
Init(10,3,3)
Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;
6
7     int left = node × 2
8     int right = (node × 2) + 1
9     int mid = (begin + end) / 2
10
11     init(left, begin, mid)
12     init(right, mid+1, end)
13
14     tree[node] = tree[left] + tree[right]
15 }

```



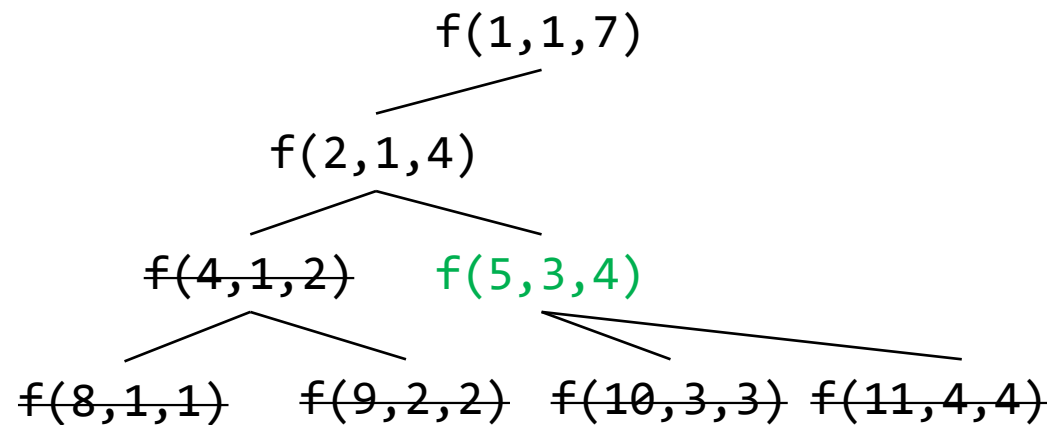
Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

			-5				4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(11,4,4)
Init(10,3,3)
Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)



```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```

Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

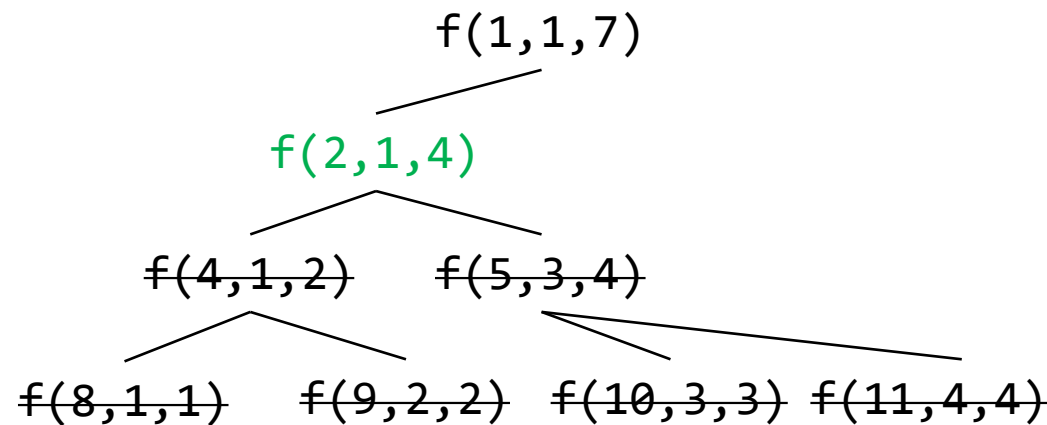
			-5	10			4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;
6
7     int left = node × 2
8     int right = (node × 2) + 1
9     int mid = (begin + end) / 2
10
11     init(left, begin, mid)
12     init(right, mid+1, end)
13
14     tree[node] = tree[left] + tree[right]
15 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10			4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(3,5,7)
Init(5,3,4)
Init(2,1,4)
Init(1, 1, 7) (call from main function)

```

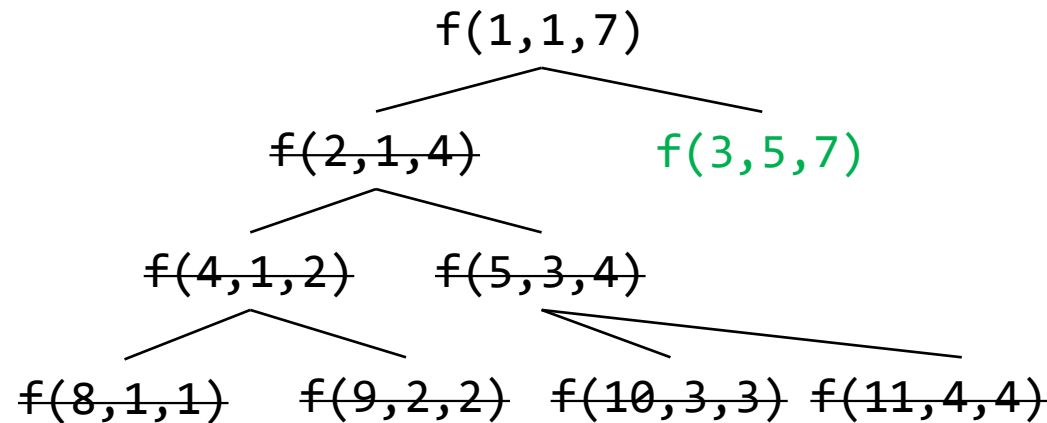
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

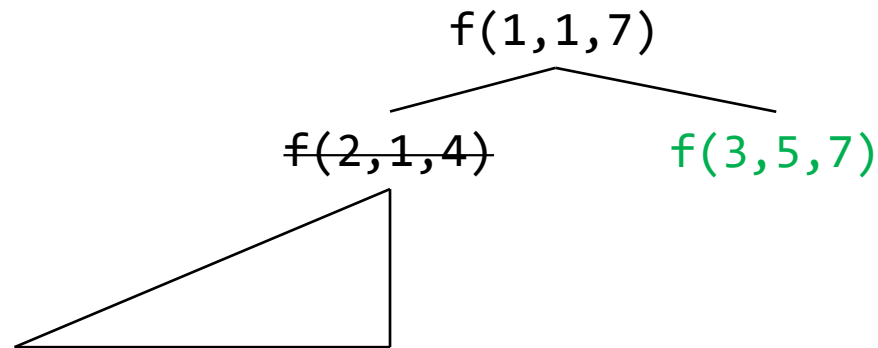
	5		-5	10			4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;
6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2
9     init(left, begin, mid)
10    init(right, mid+1, end)
11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10			4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

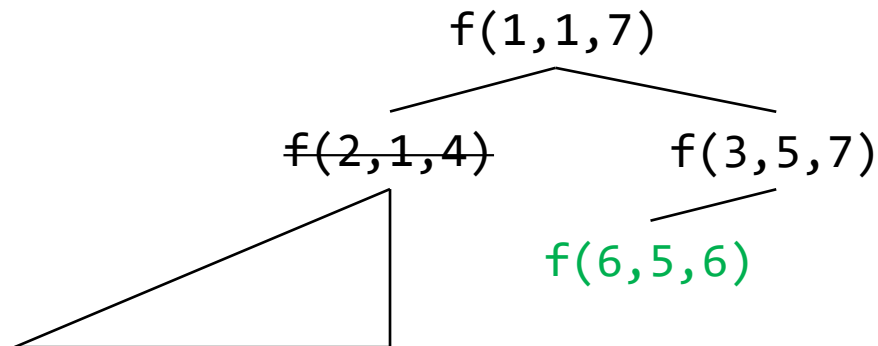
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10			4	-9	3	7		
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(12,5,5)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

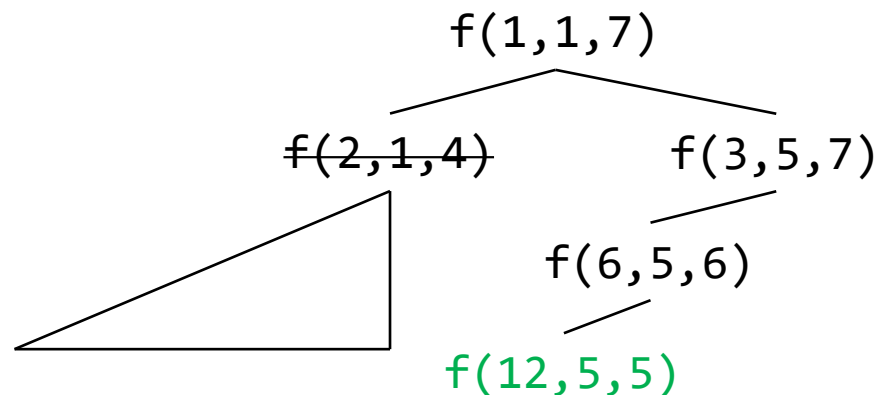
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10			4	-9	3	7	1	
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(13,6,6)
Init(12,5,5)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

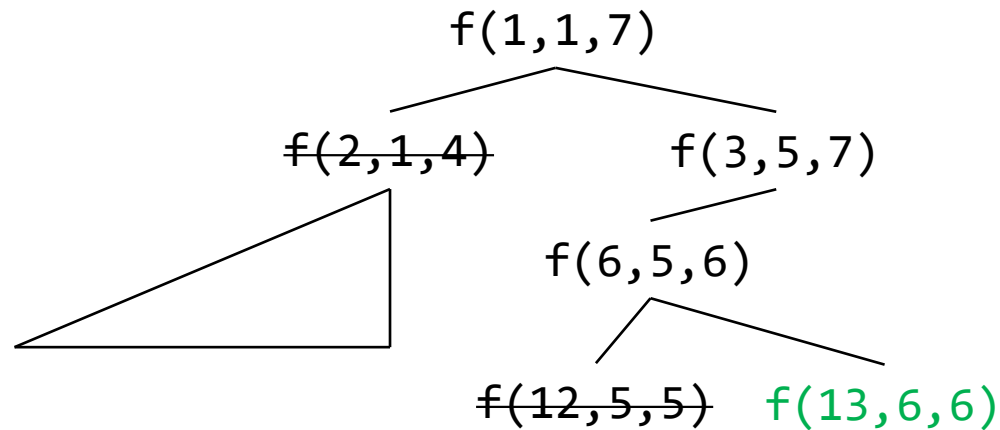
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10			4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(13,6,6)
Init(12,5,5)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

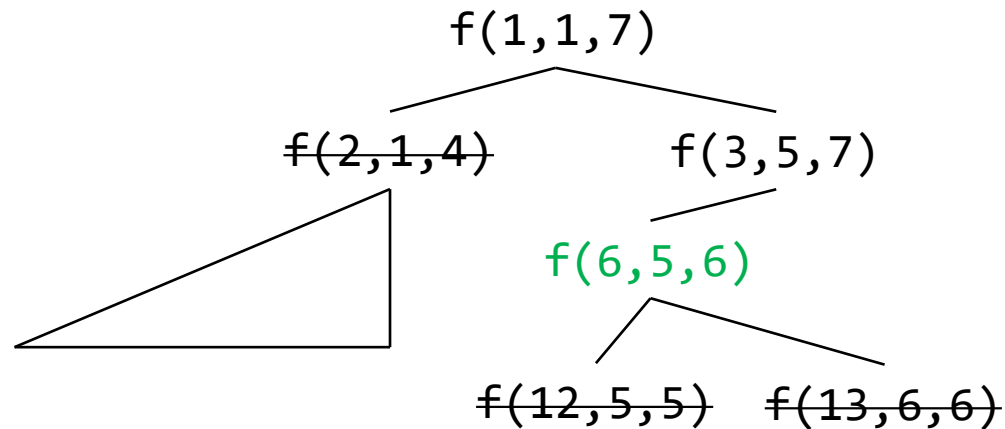
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10	1		4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(7,7,7)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

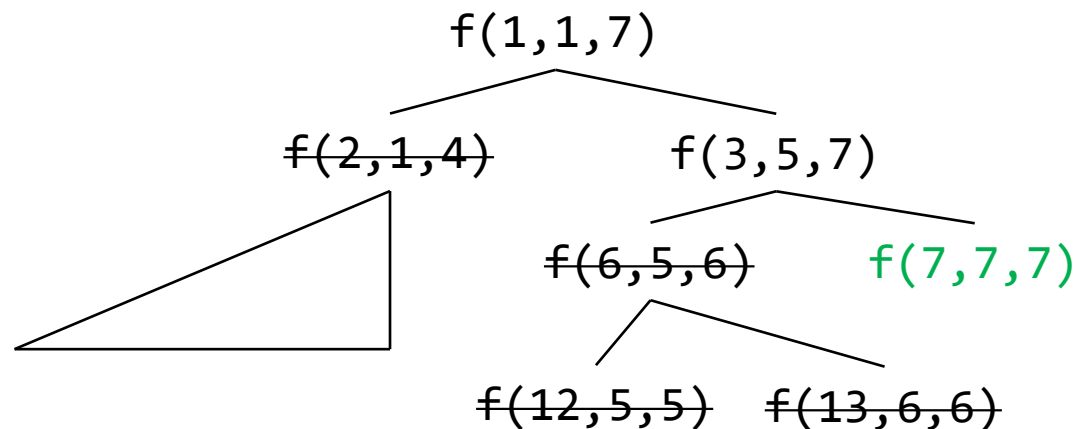
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

	5		-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(7,7,7)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

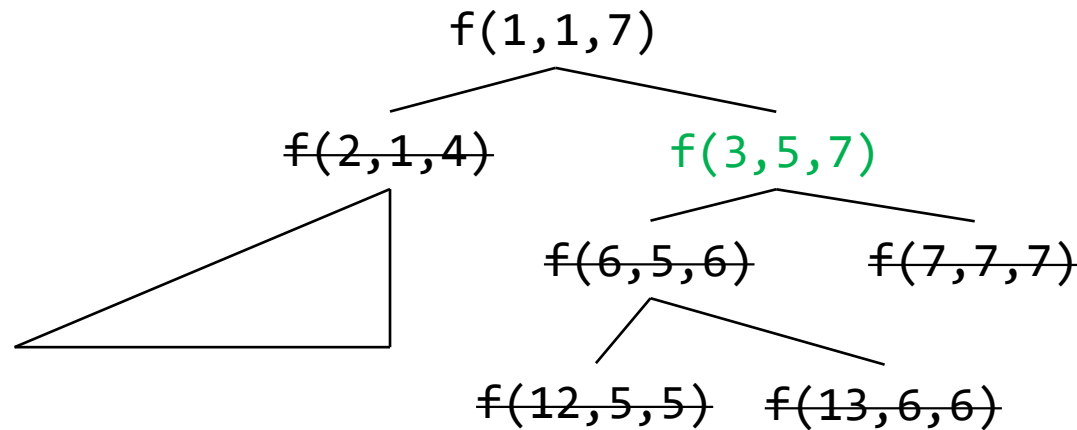
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node × 2
7     int right = (node × 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

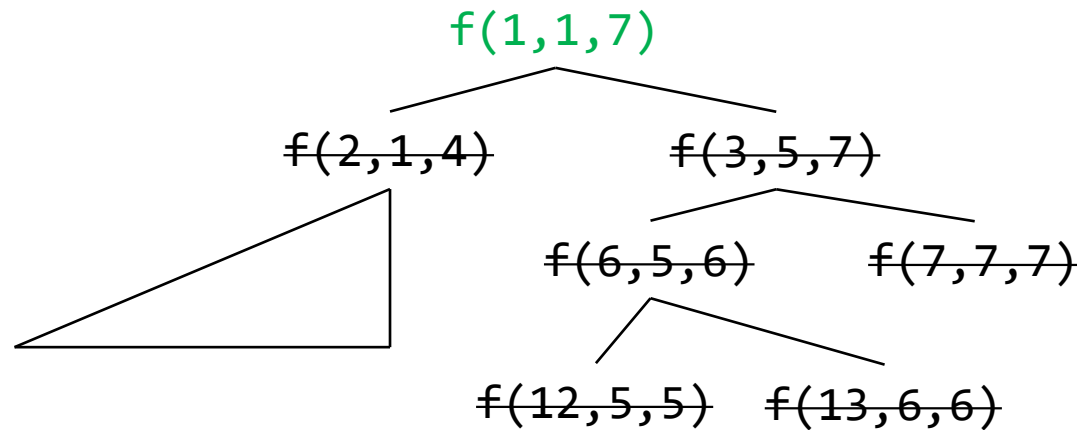
	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Init(7,7,7)
Init(6,5,6)
Init(3,5,7)
Init(1, 1, 7) (call from main function)

```

1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;
6
7     int left = node * 2
8     int right = (node * 2) + 1
9     int mid = (begin + end) / 2
10
11     init(left, begin, mid)
12     init(right, mid+1, end)
13
14     tree[node] = tree[left] + tree[right]
15 }

```



Input array (1-based index)

4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

The program will return to main menu as the recursion stack is empty and the segment tree is perfectly built!

```

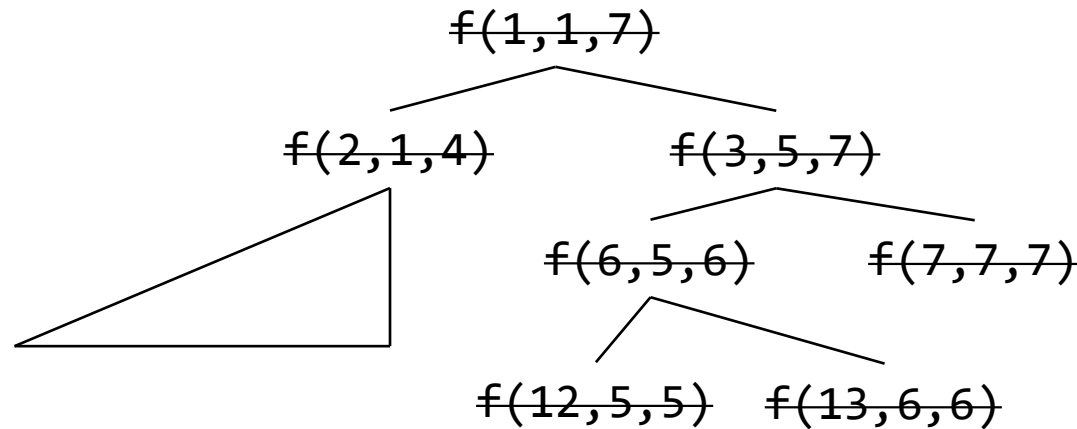
1 void init ( int node, int begin, int end) {
2     If (begin == end)
3         // leaf node found. Insert directly.
4         tree[node] = arr[begin]
5         return;

6     int left = node * 2
7     int right = (node * 2) + 1
8     int mid = (begin + end) / 2

9     init(left, begin, mid)
10    init(right, mid+1, end)

11    tree[node] = tree[left] + tree[right]
12 }

```



Input array (1-based index)

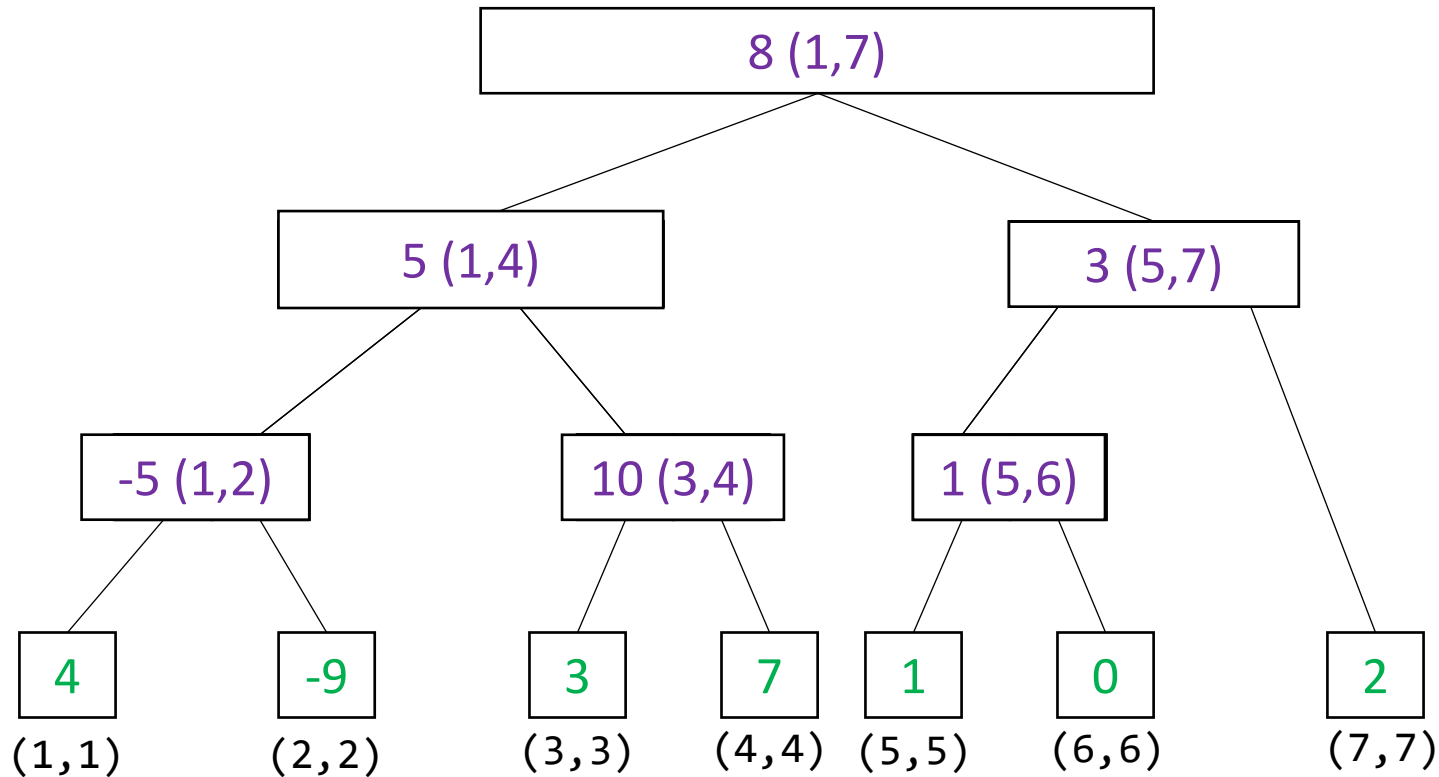
4	-9	3	7	1	0	2
1	2	3	4	5	6	7

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

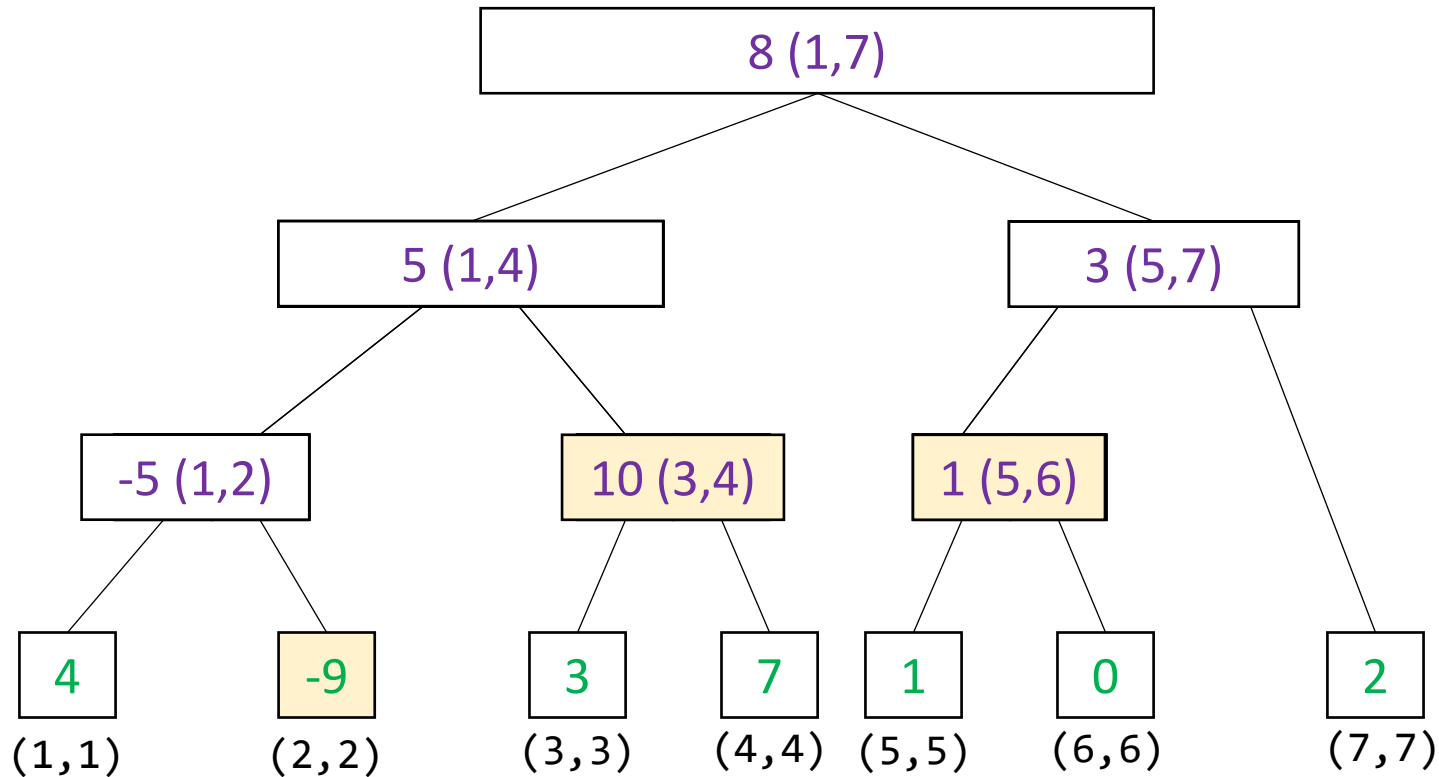


Query:

Suppose, we need the sum of the values within the range (2,6)

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



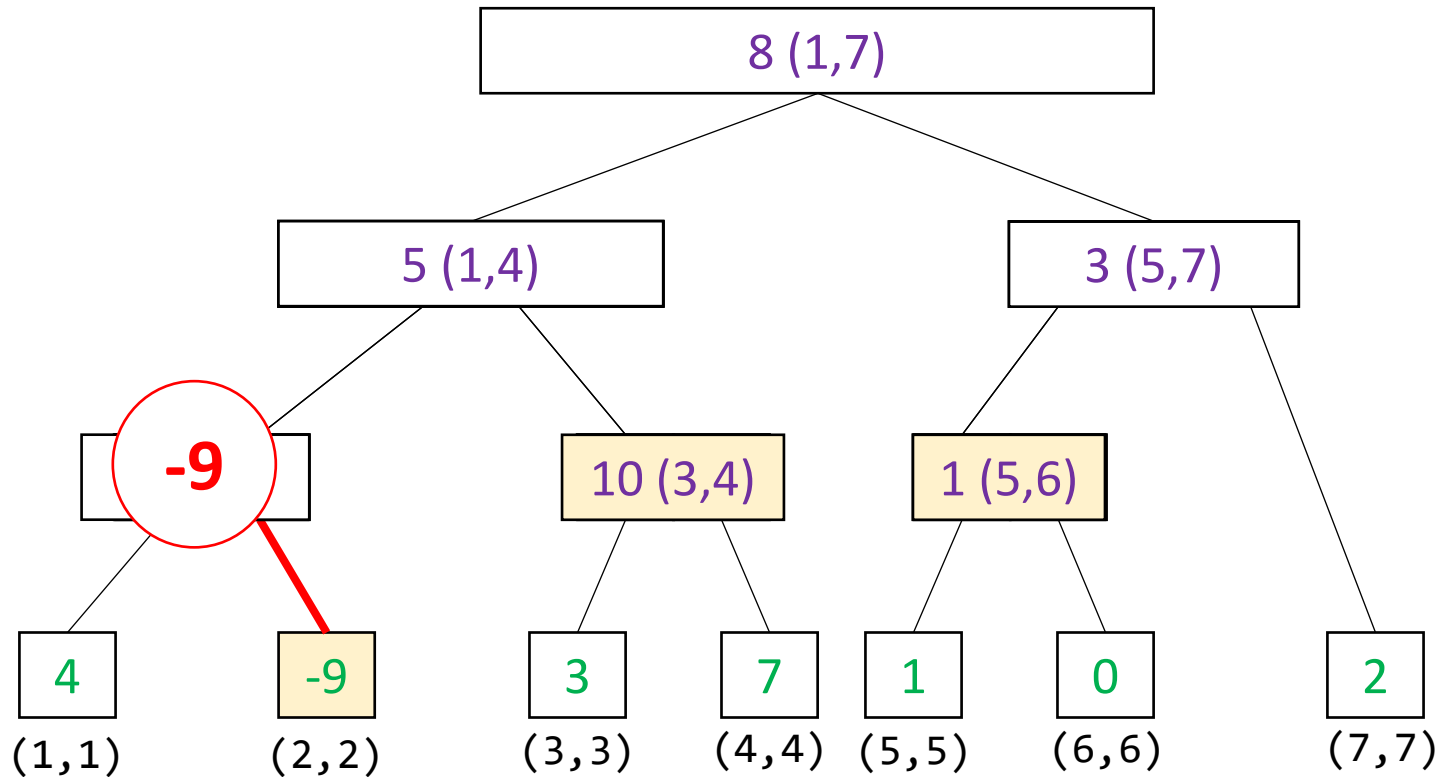
Query:

Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



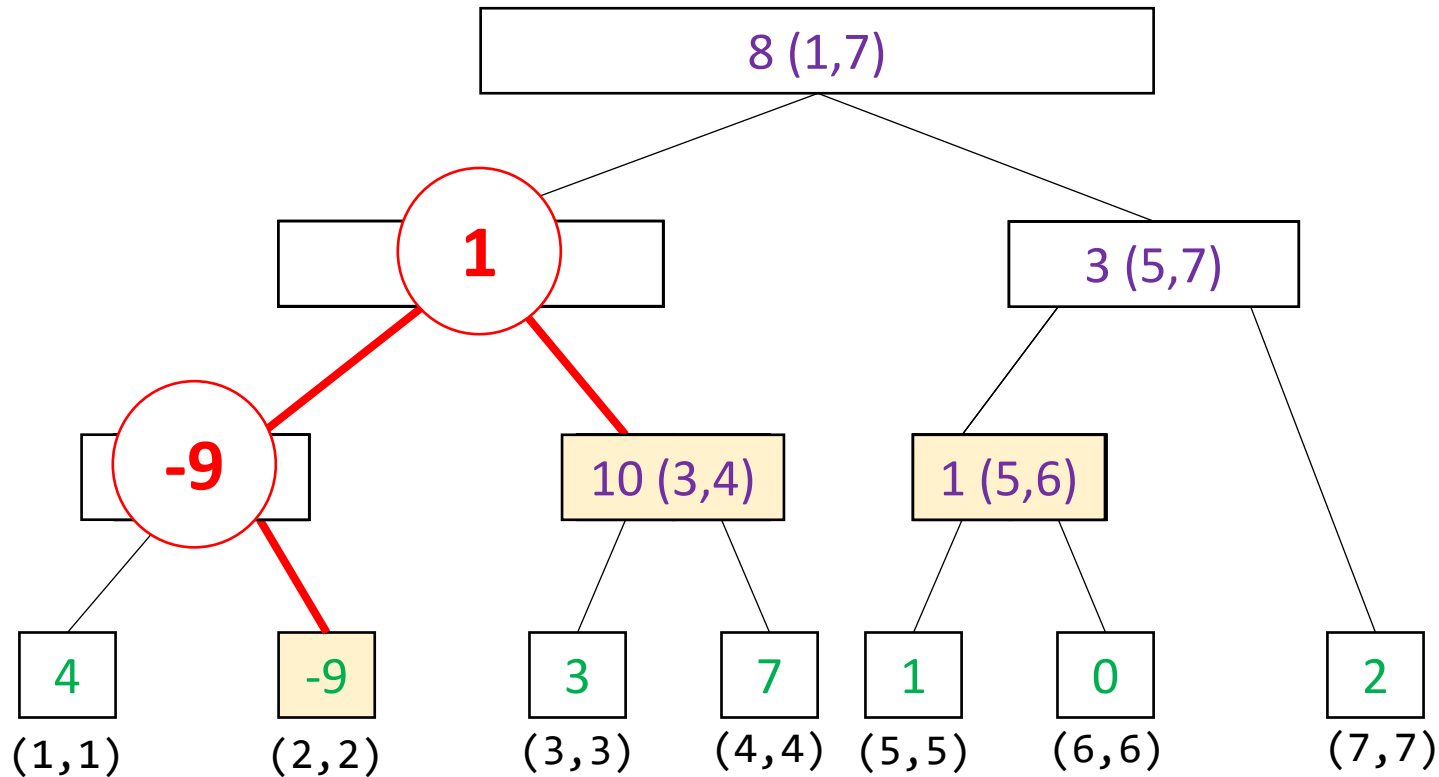
Query:

Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



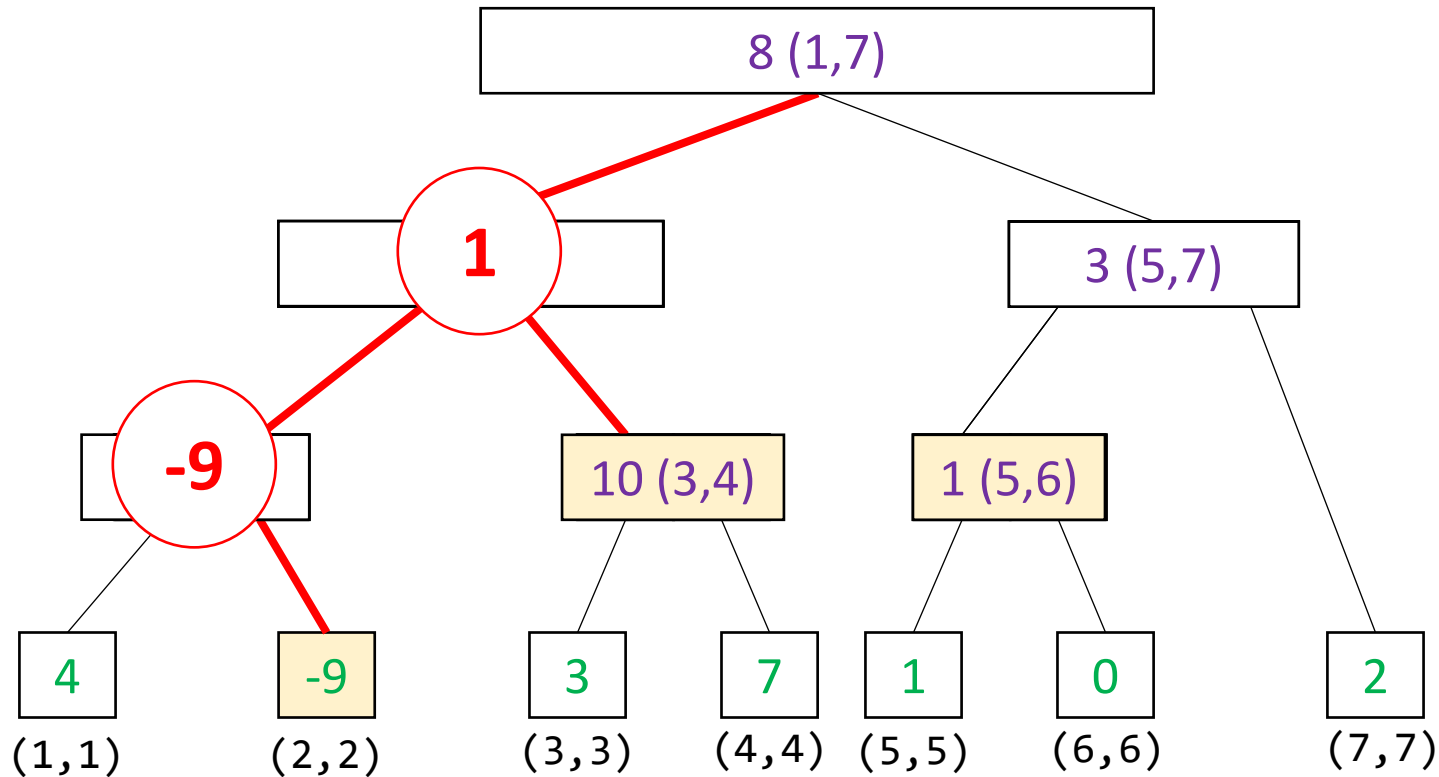
Query:

Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



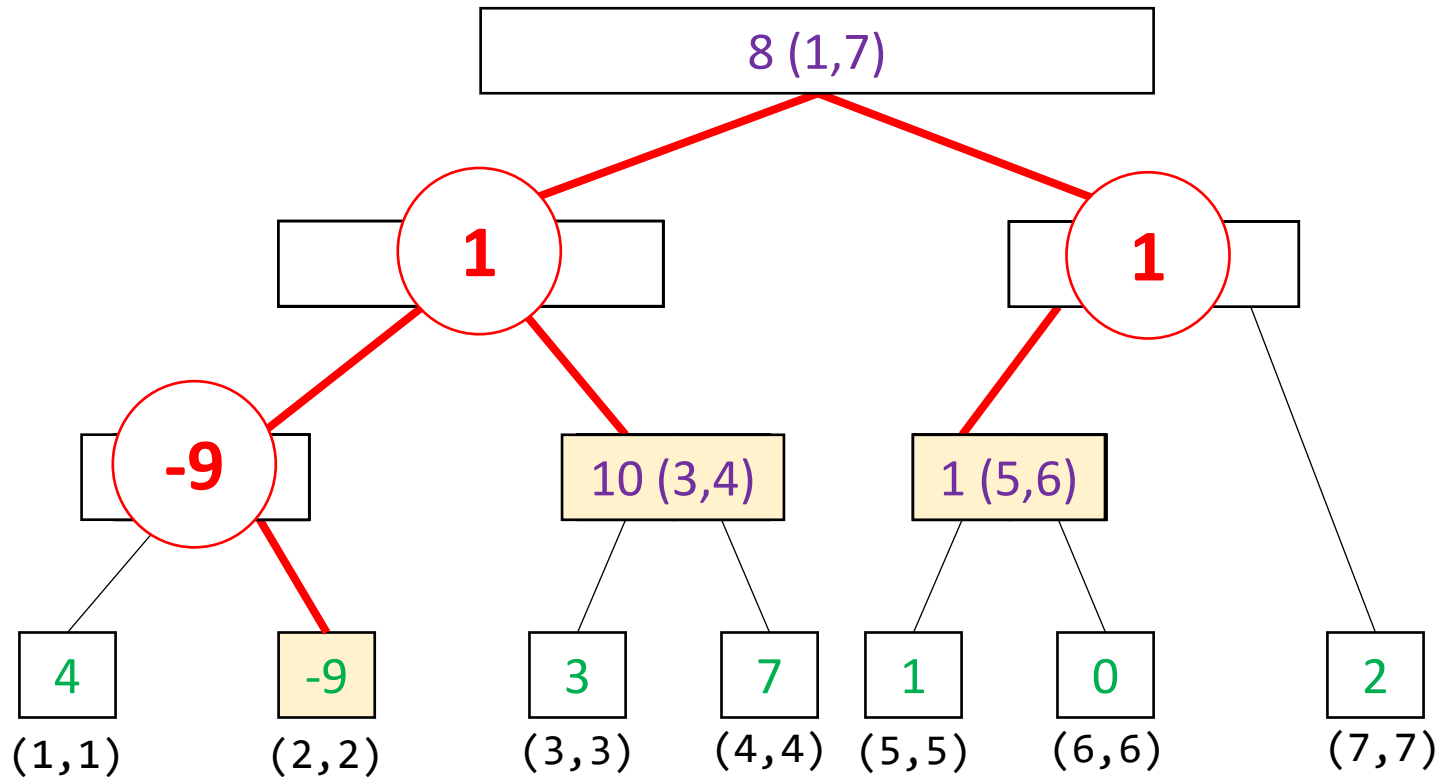
Query:

Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



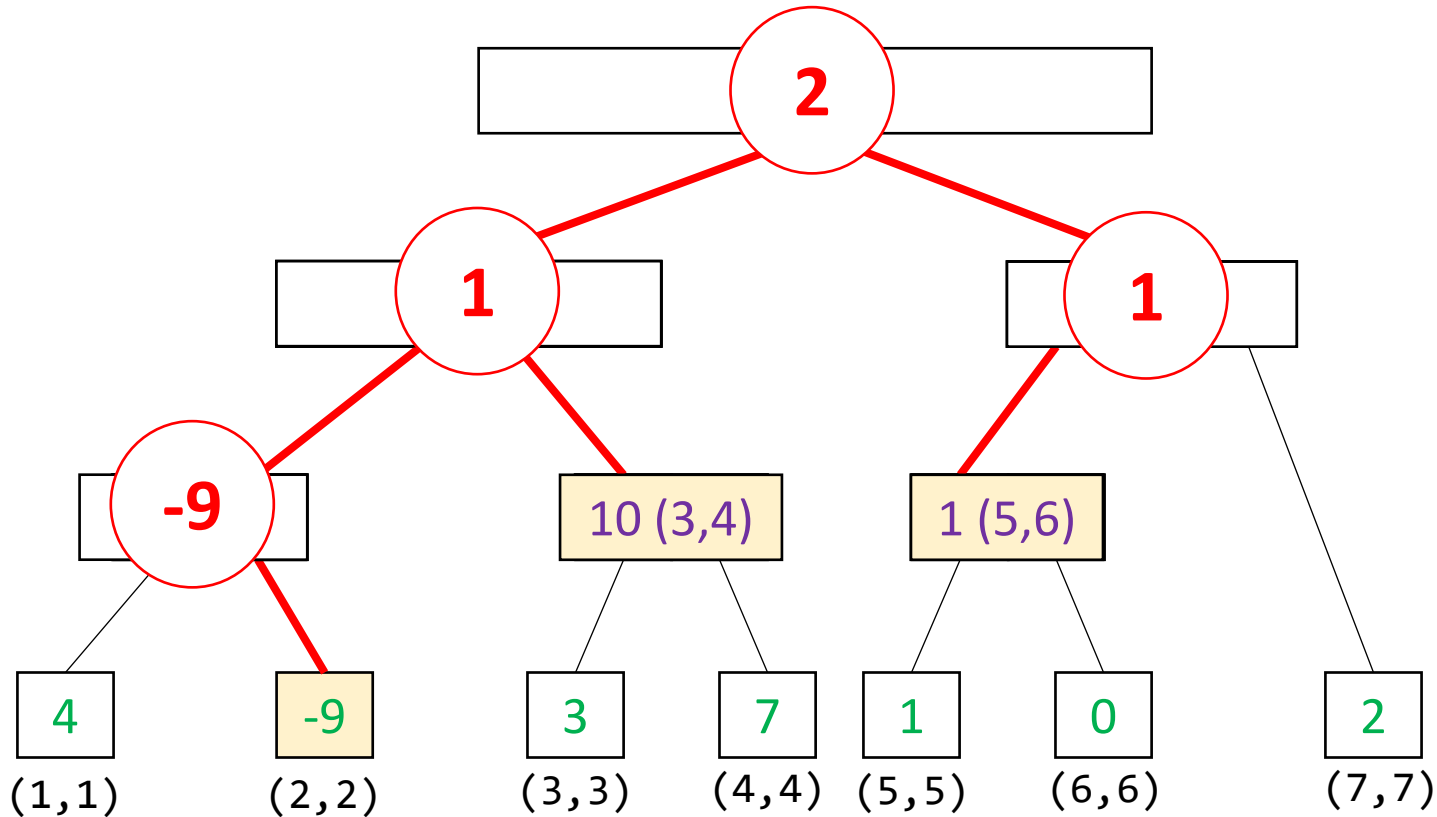
Query:

Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



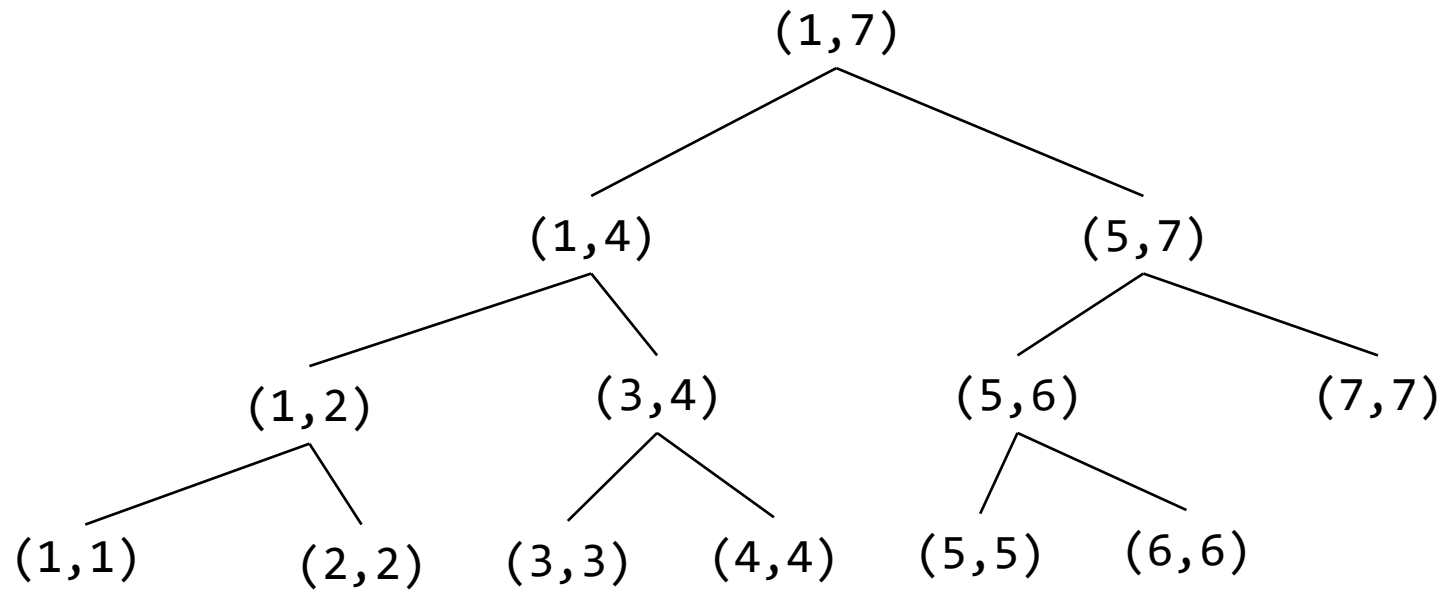
Query:

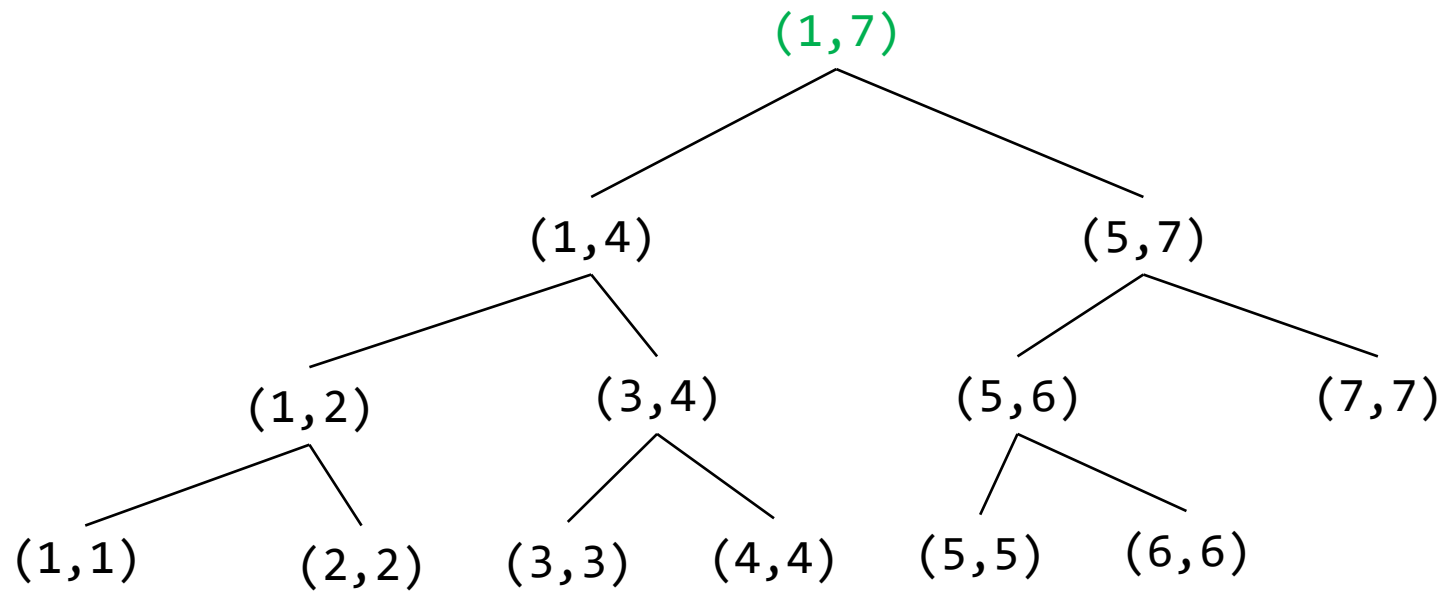
Suppose, we need the sum of the values within the range (2,6)

To do that, it's enough just to access the Yellow nodes !

How do we implement this concept?

How to query?
Need the result for
 $(i,j) = \text{Range}(2,6)$





How to query?

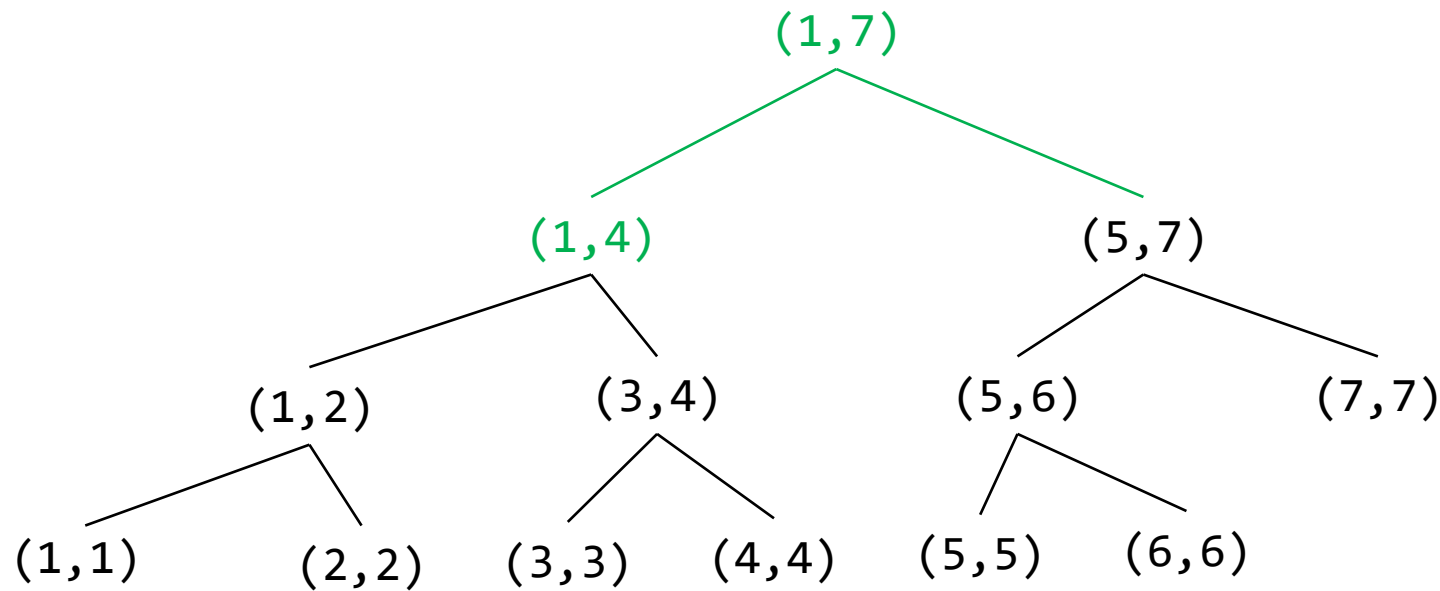
Need the result for

$(i, j) = \text{Range}(2, 6)$

- Start from the root
- Range of current node $(b, e) = (1, 7)$

This is a *partial overlap*. As current node has some extra info along with the relevant info.

For *partial overlap*, look into both left and right subtrees



How to query?

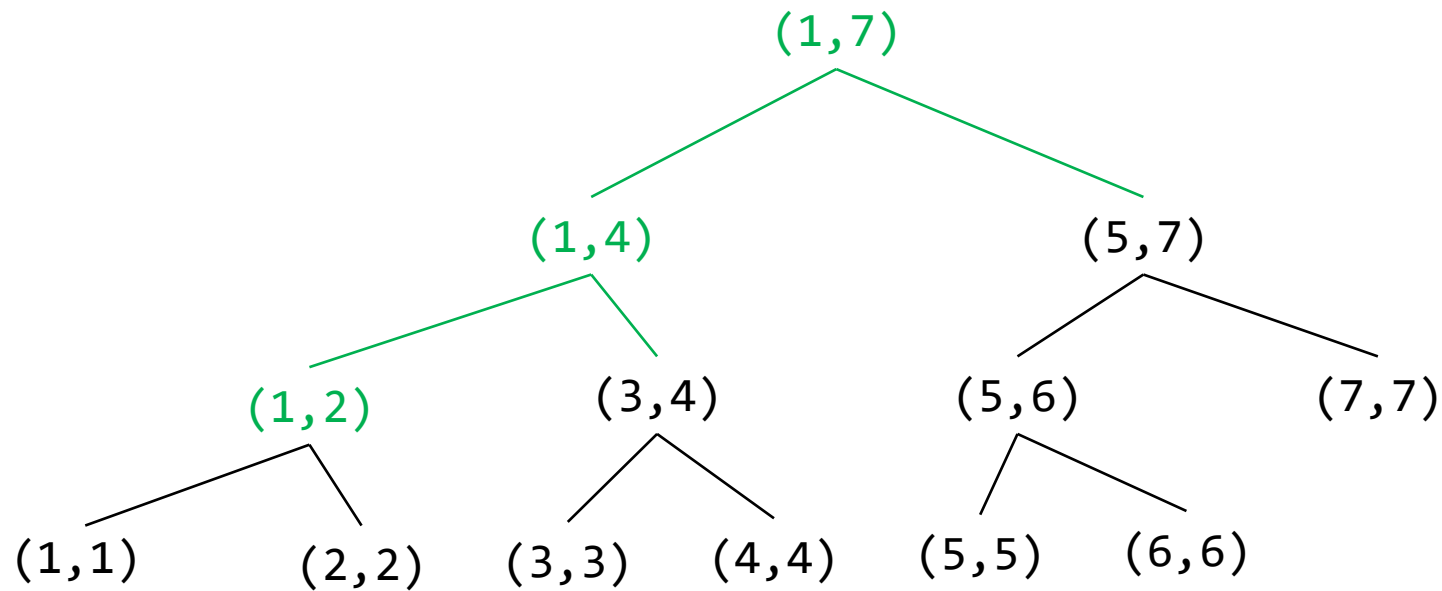
Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (1, 4)$

Partial overlap, as node contains some extra info!

Look into both left & right subtrees.



How to query?

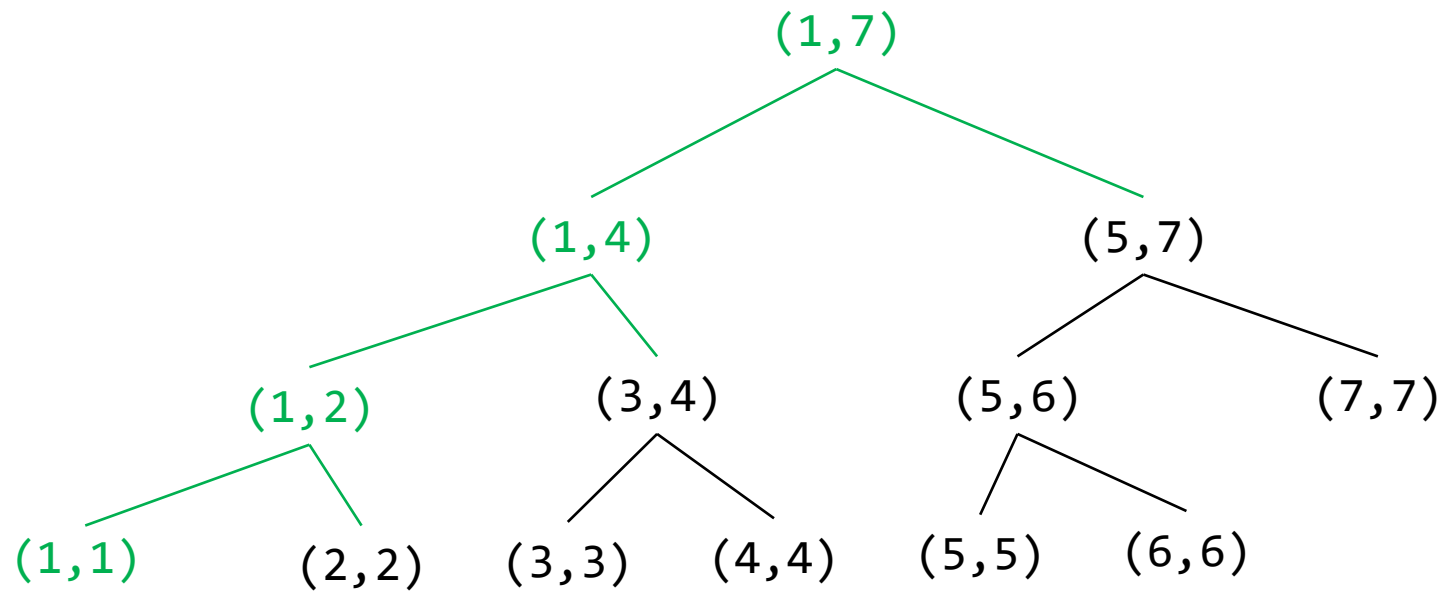
Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (1, 2)$

Partial overlap!

Look into both left & right subtrees.



How to query?

Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (1, 1)$

No overlap!

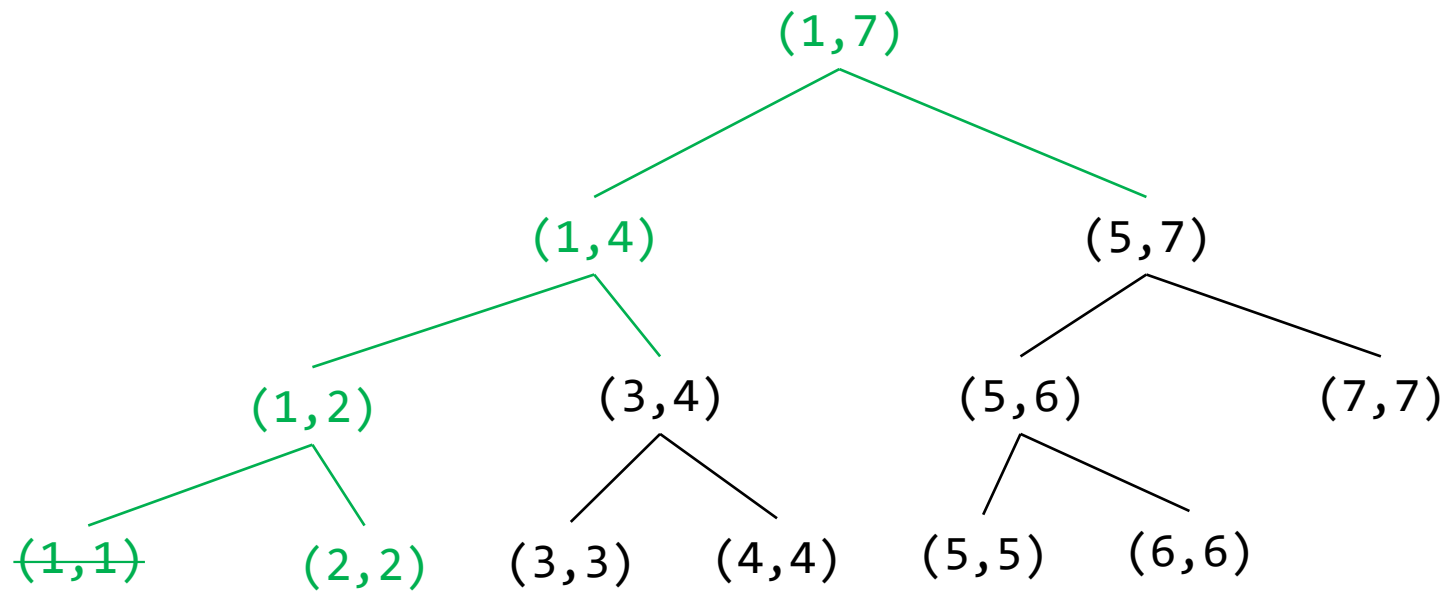
Range contains no relevant info

Return 0 (in case of sum)

Return INT_Min in case of max query

Return INT_Max for min query

Return something that doesn't affect the result!



How to query?

Need the result for

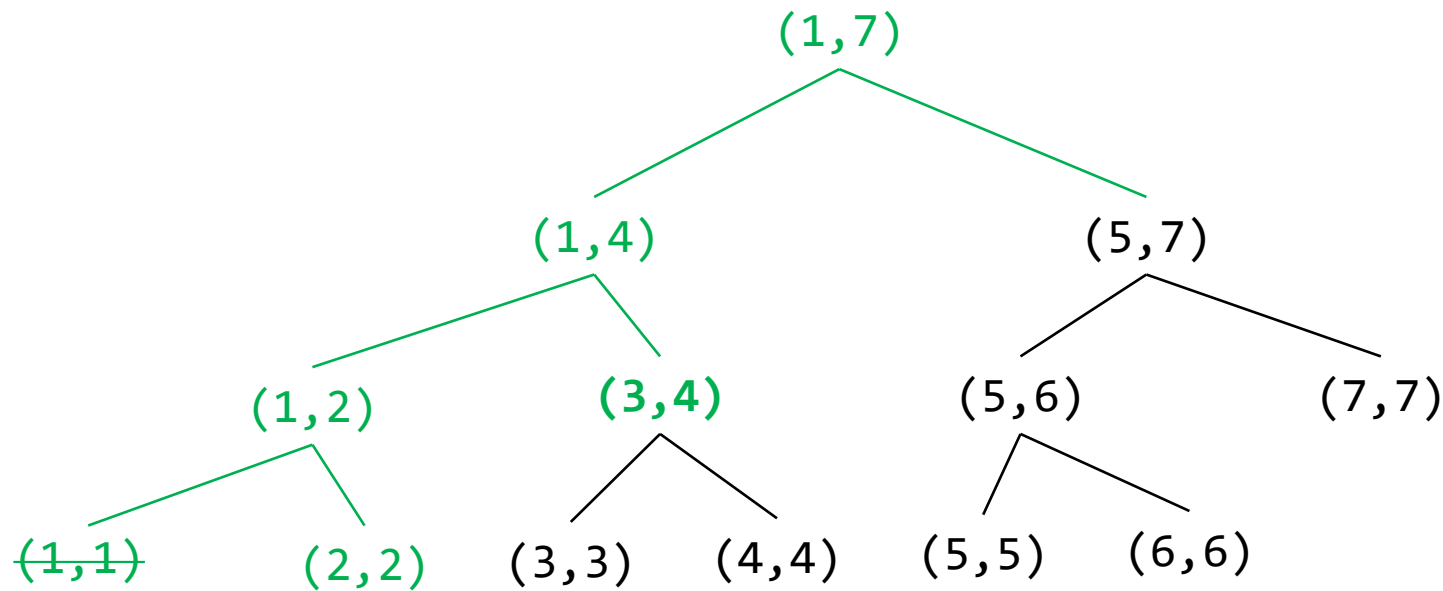
$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (2, 2)$

Full overlap!

Entire range of current node is relevant to current query

Return the result, don't divide further.



How to query?

Need the result for

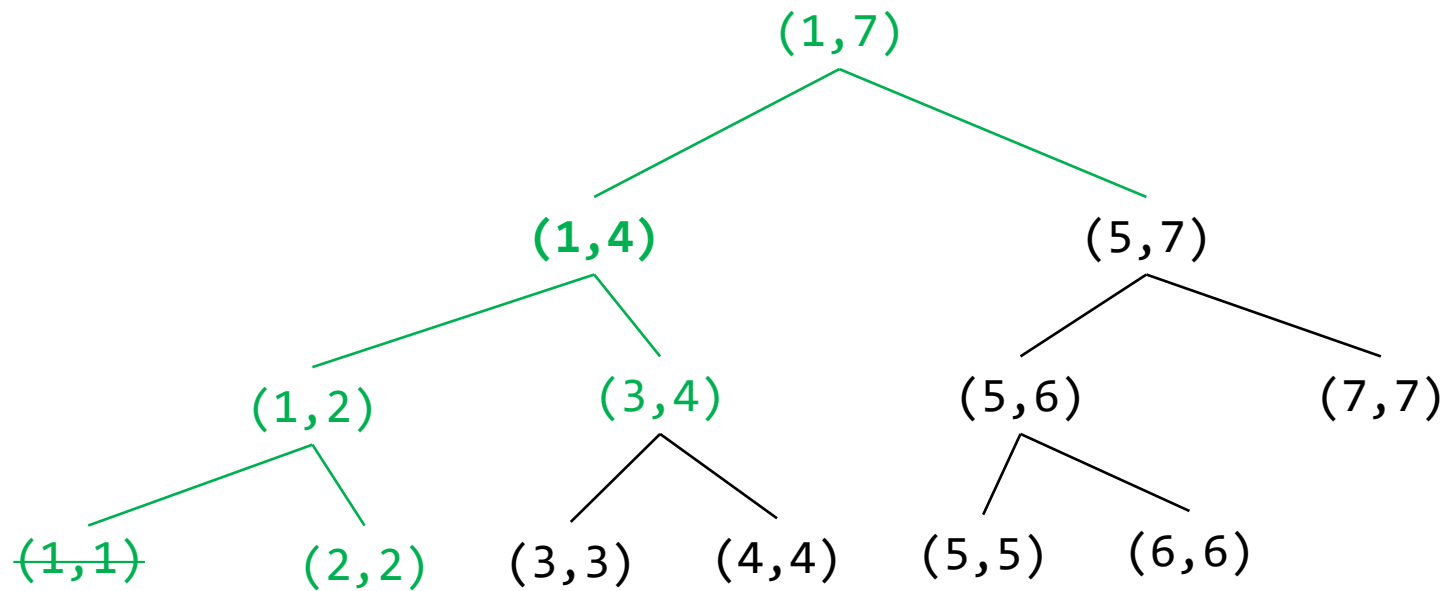
$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (3, 4)$

Full overlap!

Entire range of current node is relevant to current query

Return the result, don't divide further.



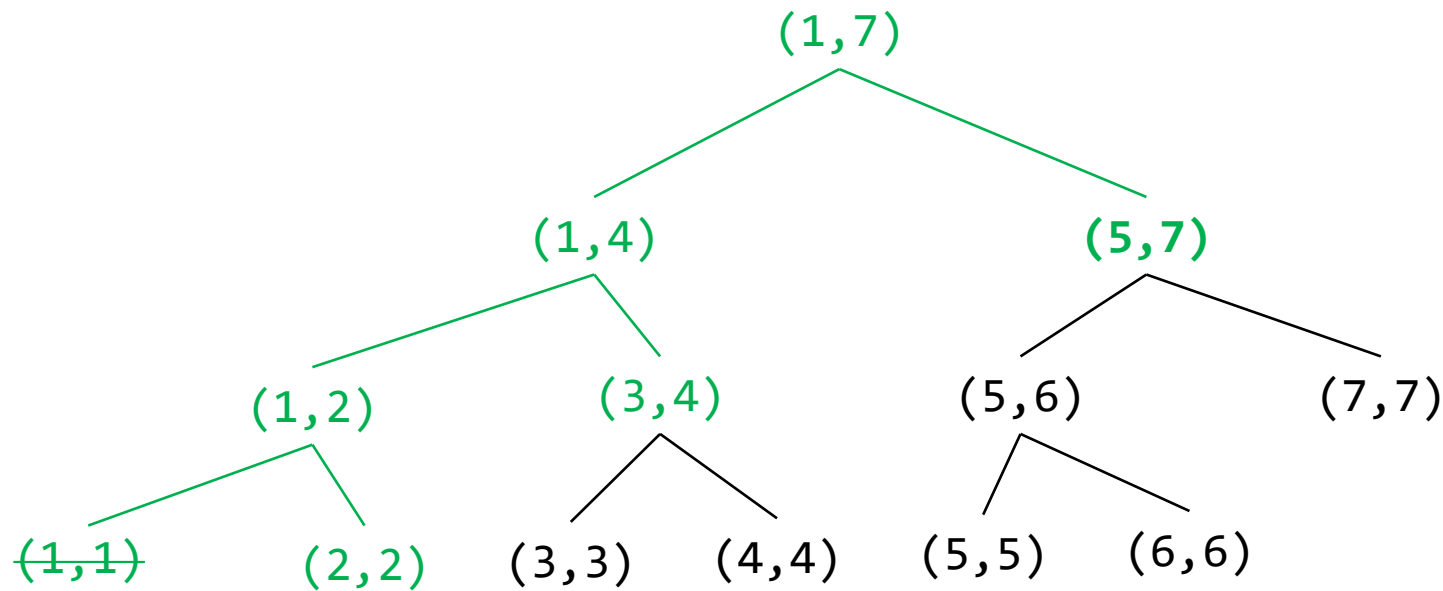
How to query?

Need the result for

$(i, j) = \text{Range}(2, 6)$

Range of current node $(b, e) = (1, 4)$

This node will combine the results from the left and right subtree and pass to root.



How to query?

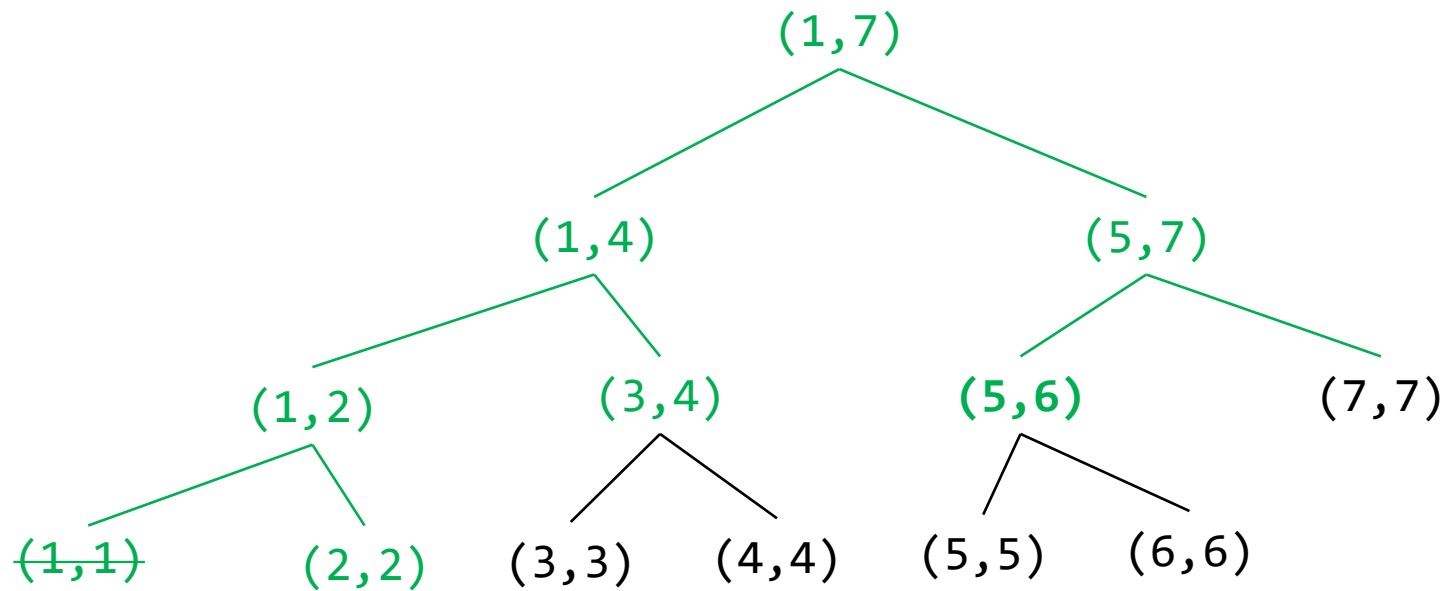
Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (5, 7)$

Partial overlap.

Divide into left & right subtrees



How to query?

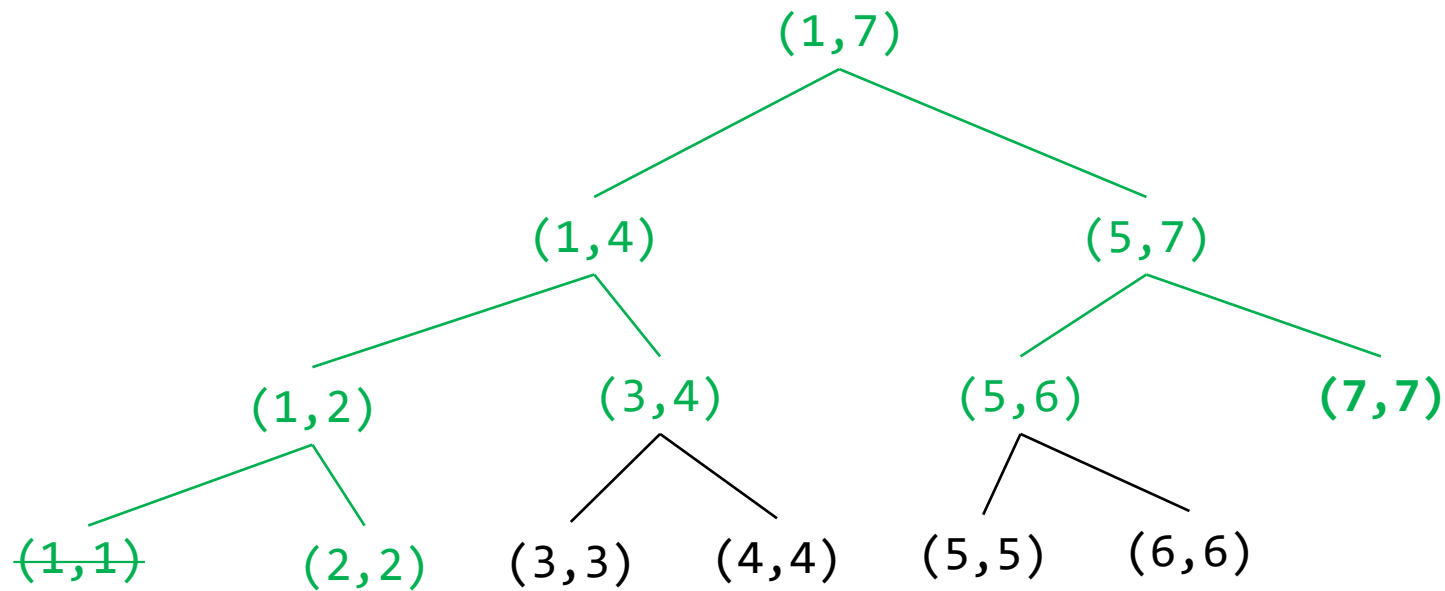
Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (5, 6)$

Full overlap.

Utilize the result. Don't divide further



How to query?

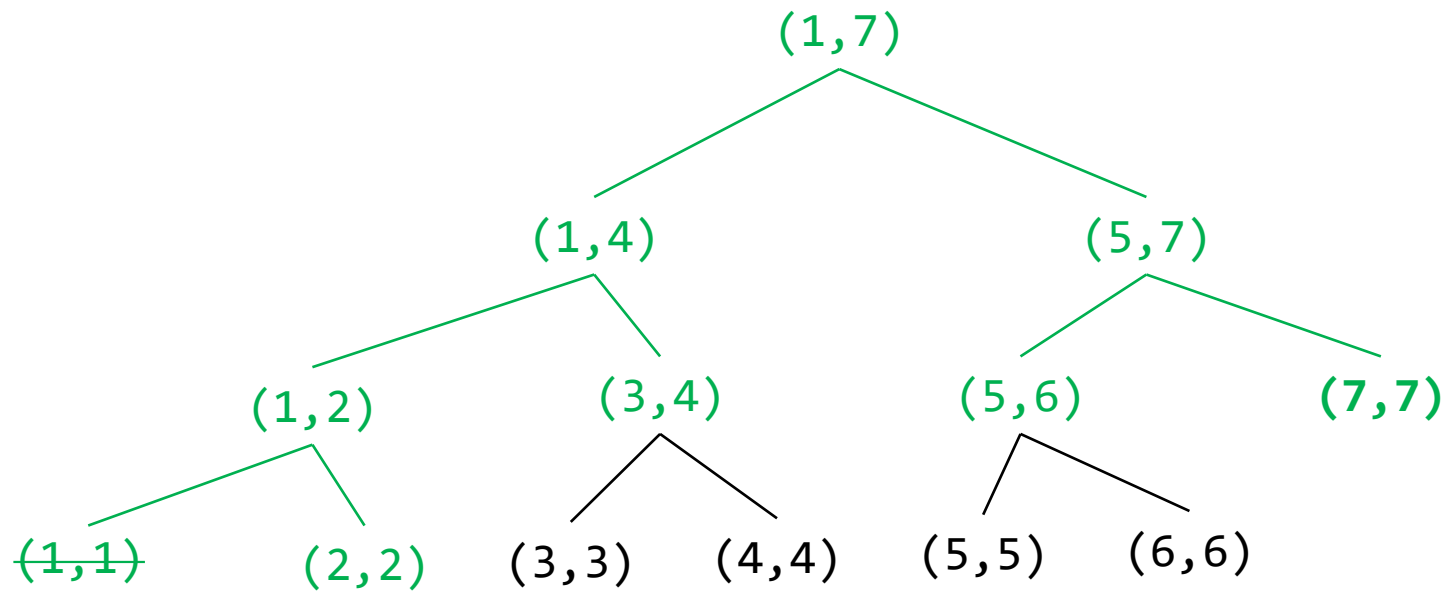
Need the result for

$(i, j) = \text{Range } (2, 6)$

Range of current node $(b, e) = (7, 7)$

No overlap.

Return

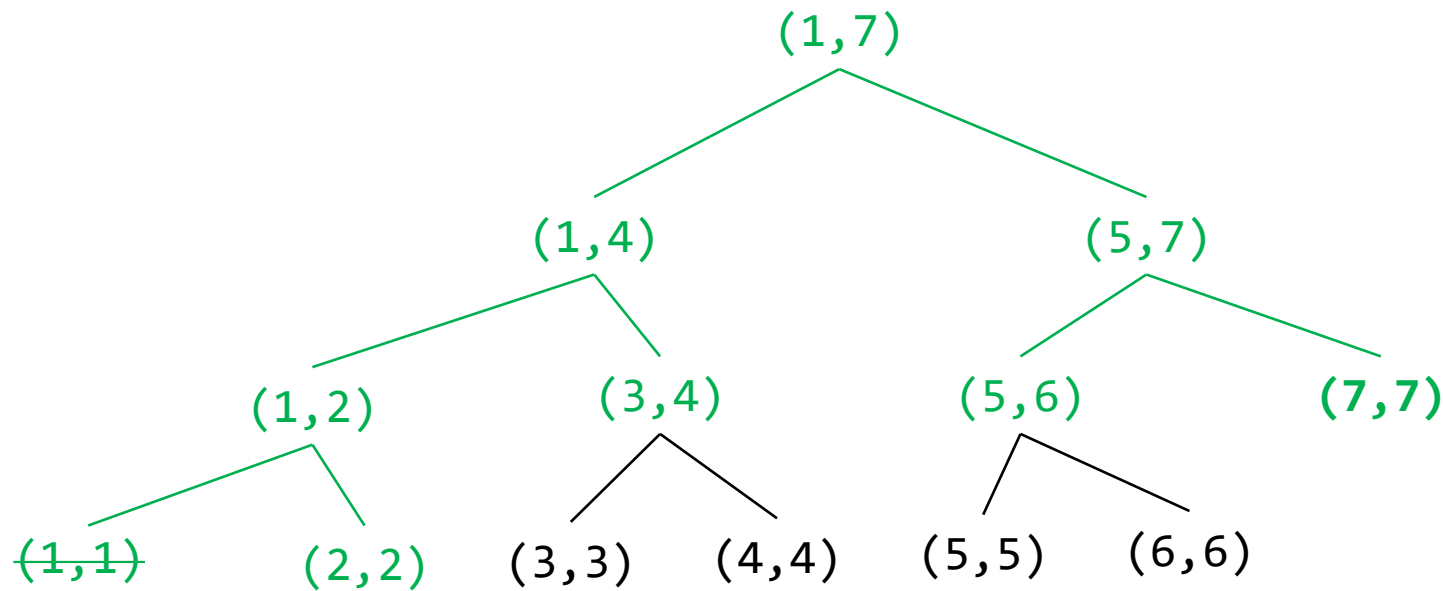


How to query?

Need the result for

$(i,j) = \text{Range } (2,6)$

What is the complexity of the overall process?



How to query?

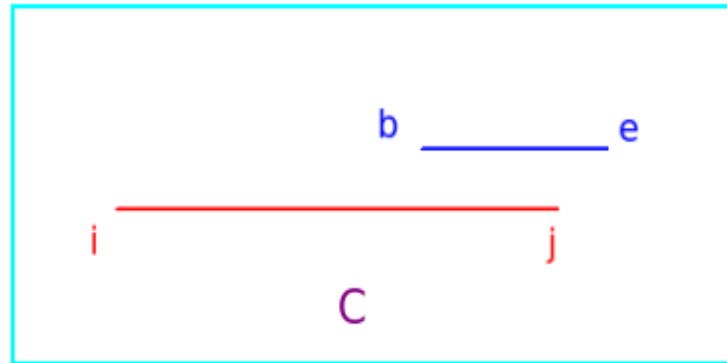
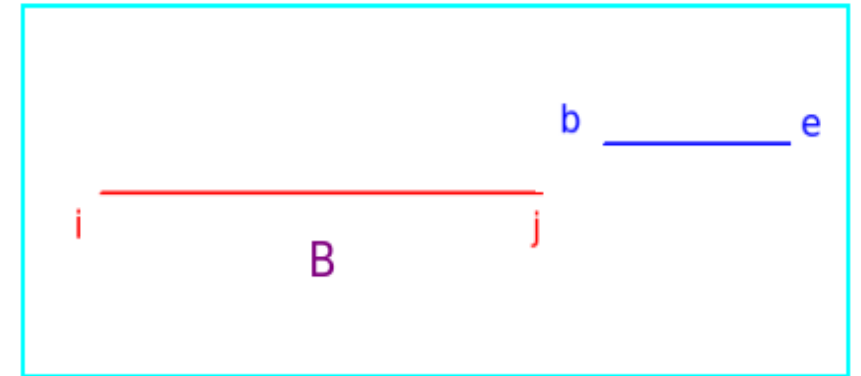
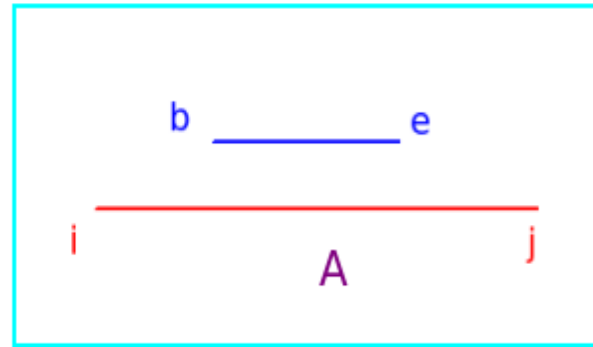
Need the result for

$(i, j) = \text{Range } (2, 6)$

What is the complexity of the overall process?

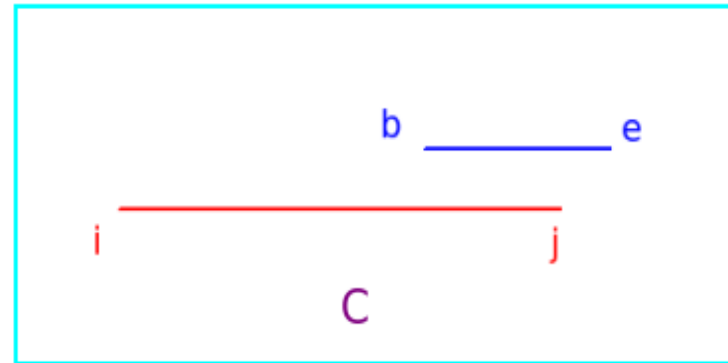
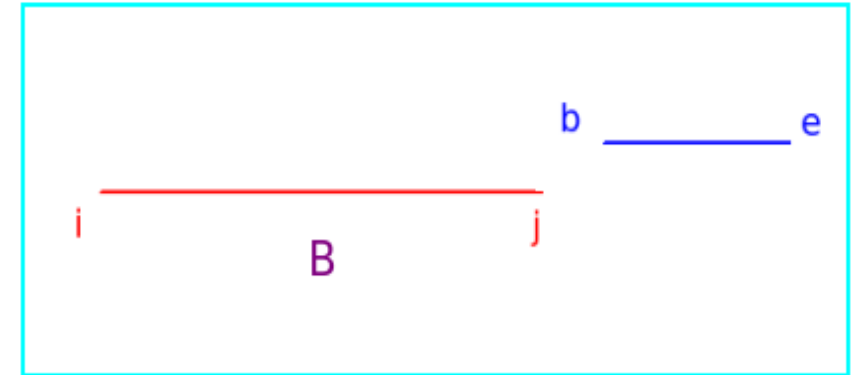
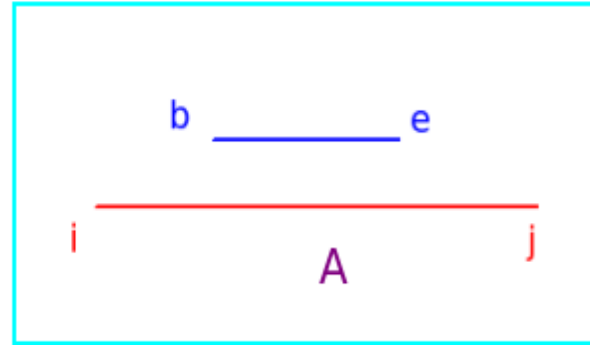
Ans: $O(\log N)$

It might be a bit misleading to see that the query algorithm is almost traversing all the paths. The effect is less visible due to the tree being very small. But as the input size grows, we'll see that entire $\log N$ paths are hardly traversed.



Query:

- Full overlap: don't dig further! Use the value.
- No overlap: Ignore
- Partial overlap: Split into two halves!



- **Case 1:** ($b \geq i$ & $e \leq j$)
 - The full segment is relevant. Return the value of the node.

- **Case 1:** $(b \geq i \ \& \ e \leq j)$
 - The full segment is relevant. Return the value of the node.
- **Case 2:** $(i > e \ || \ j < b)$
 - This segment doesn't contribute to our query.
 - Return 0, if working with sum. (Not contributing to the sum.)
 - Return INT_MAX, if working with finding MIN from a given range.
 - Return INT_MIN, if working with finding MAX from a given range.

- **Case 1:** $(b \geq i \ \& \ e \leq j)$
 - The full segment is relevant. Return the value of the node.
- **Case 2:** $(i > e \ || \ j < b)$
 - This segment doesn't contribute to our query.
 - Return 0, if working with sum. (Not contributing to the sum.)
 - Return INT_MAX, if working with finding MIN from a given range.
 - Return INT_MIN, if working with finding MAX from a given range.
- **Case 3:** Case1 or Case 2 not True
 - Need to split further and look for relevant sections in both left & right subtrees.

```
1 int query_sum (int node, int b, int e, int i, int j) {
```

```
2
```

```
3
```

*Here (b, e) is the range represented by the
current node (we start from the root)*

```
4
```

```
5
```

(i, j) is the query range

```
6
```

```
7
```

```
8
```

```
9
```

```
10
```

```
11
```

```
12 }
```

```
1 int query_sum (int node, int b, int e, int i, int j) {  
2     if(i>e || j<b)  
3         return 0                //out of range  
4  
5  
6  
7  
8  
9  
10  
11  
12 }
```

```

1 int query_sum (int node, int b, int e, int i, int j) {
2     if(i>e || j<b)
3         return 0                //out of range
4     if(b>=i && e<=j)
5         return tree[node]        //full segment is relevant
6
7
8
9
10
11
12 }

```

```

1 int query_sum (int node, int b, int e, int i, int j) {
2     if(i>e || j<b)
3         return 0                //out of range
4     if(b>=i && e<=j)
5         return tree[node]        //full segment is relevant
6     int left= node*2
7     int right= node*2 + 1
8     int mid= (b + e)/2
9
10
11
12 }

```



```

1 int query_sum (int node, int b, int e, int i, int j) {
2     if(i>e || j<b)
3         return 0                //out of range
4     if(b>=i && e<=j)
5         return tree[node]       //full segment is relevant
6     int left= node*2
7     int right= node*2 + 1
8     int mid= (b + e)/2
9     int p1 = query(left, b, mid, i, j)
10    int p2 = query(right, mid+1, e, i, j)
11
12 }

```

```

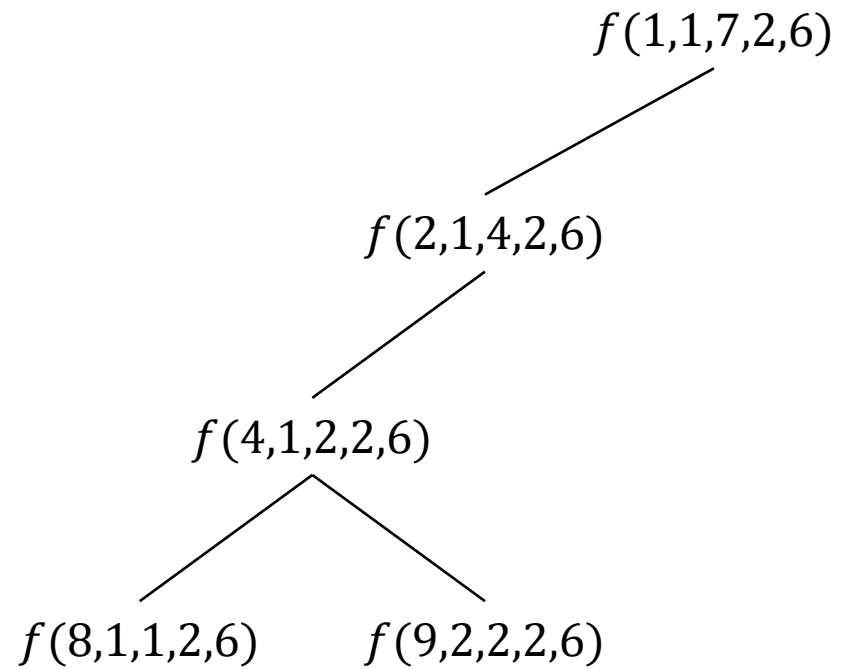
1 int query_sum (int node, int b, int e, int i, int j) {
2     if(i>e || j<b)
3         return 0                //out of range
4     if(b>=i && e<=j)
5         return tree[node]        //full segment is relevant
6     int left= node*2
7     int right= node*2 + 1
8     int mid= (b + e)/2
9     int p1 = query(left, b, mid, i, j)
10    int p2 = query(right, mid+1, e, i, j)
11    return p1+p2
12 }

```

How to query?

Need the result for

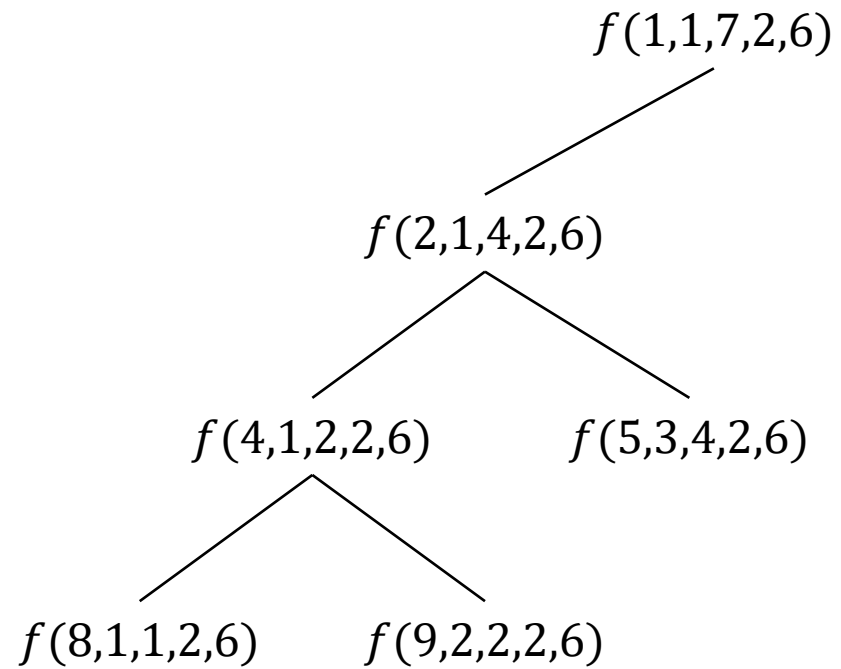
$(i, j) = \text{Range } (2, 6)$



How to query?

Need the result for

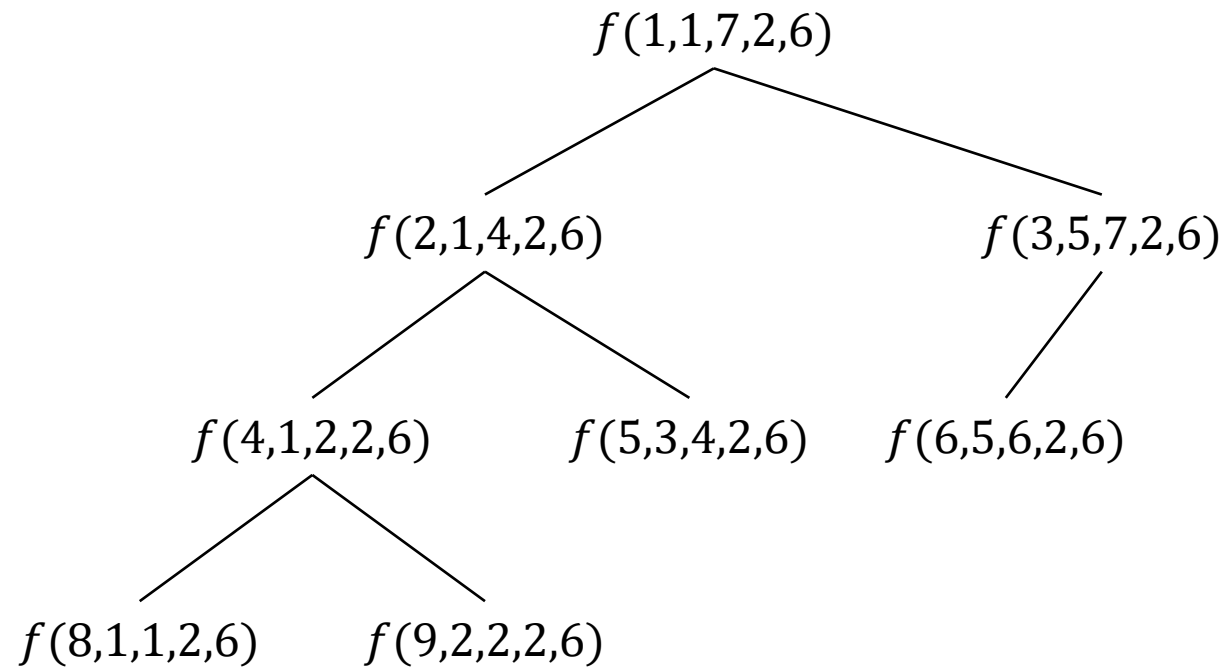
$(i, j) = \text{Range } (2, 6)$



How to query?

Need the result for

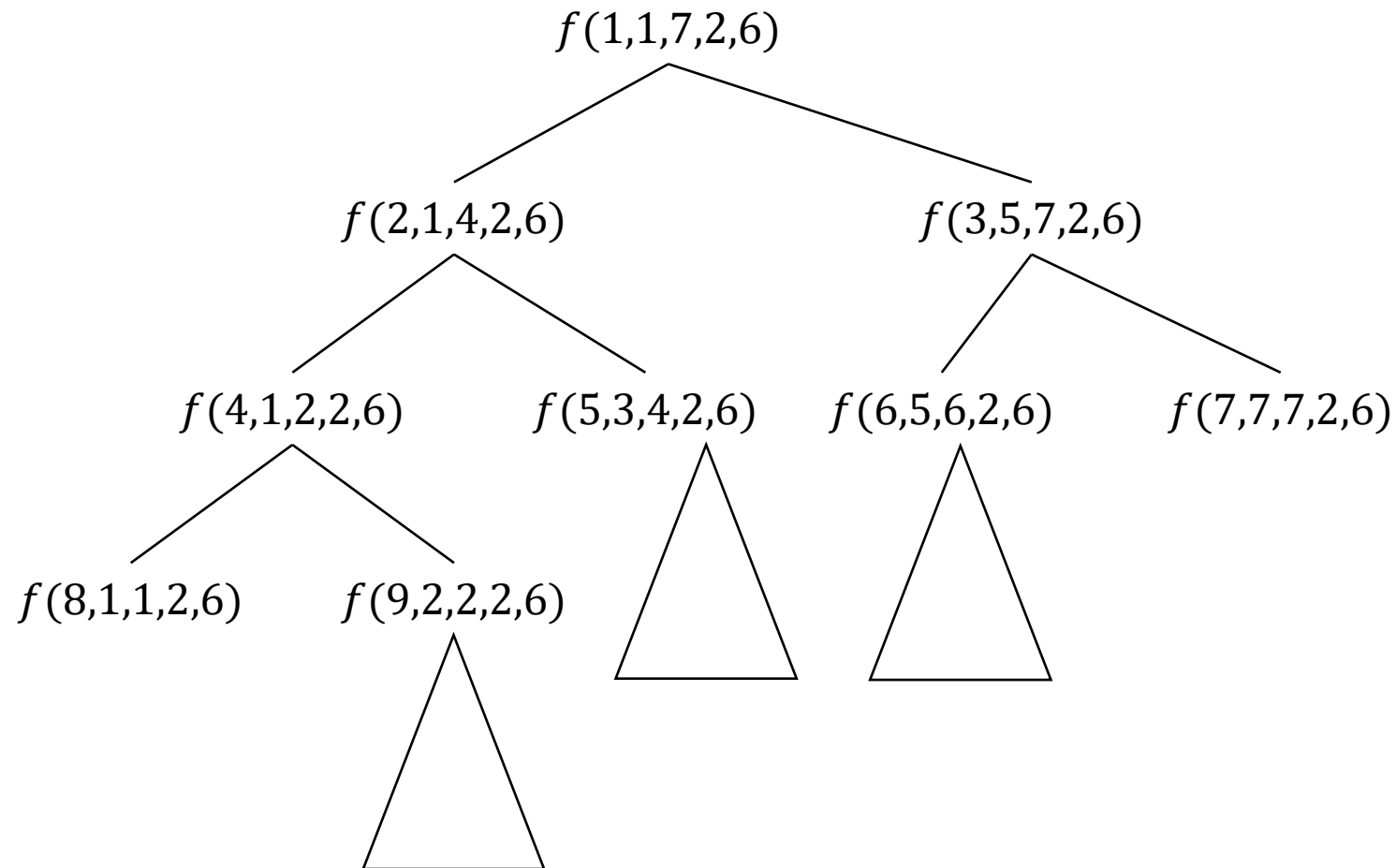
$(i, j) = \text{Range } (2, 6)$



How to query?

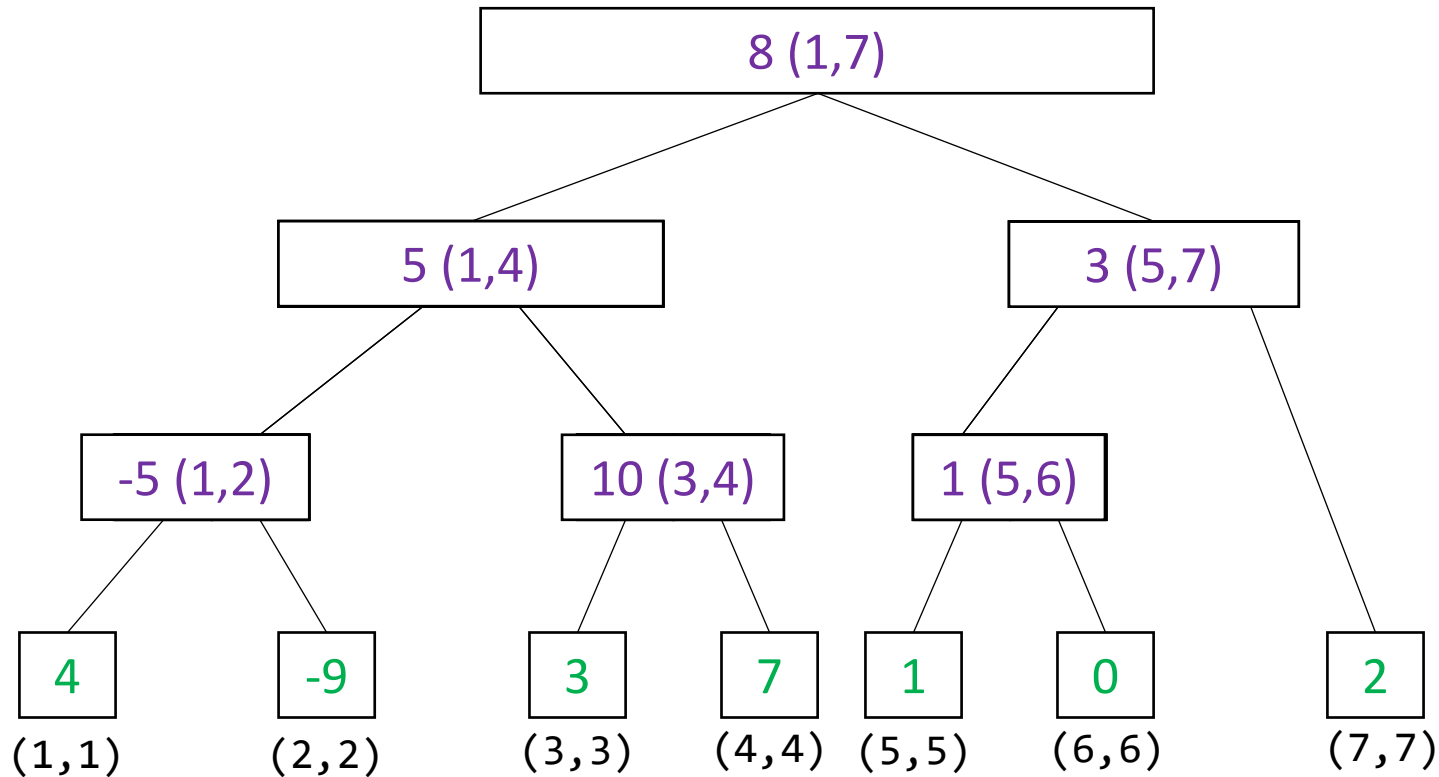
Need the result for

$(i, j) = \text{Range } (2, 6)$



Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

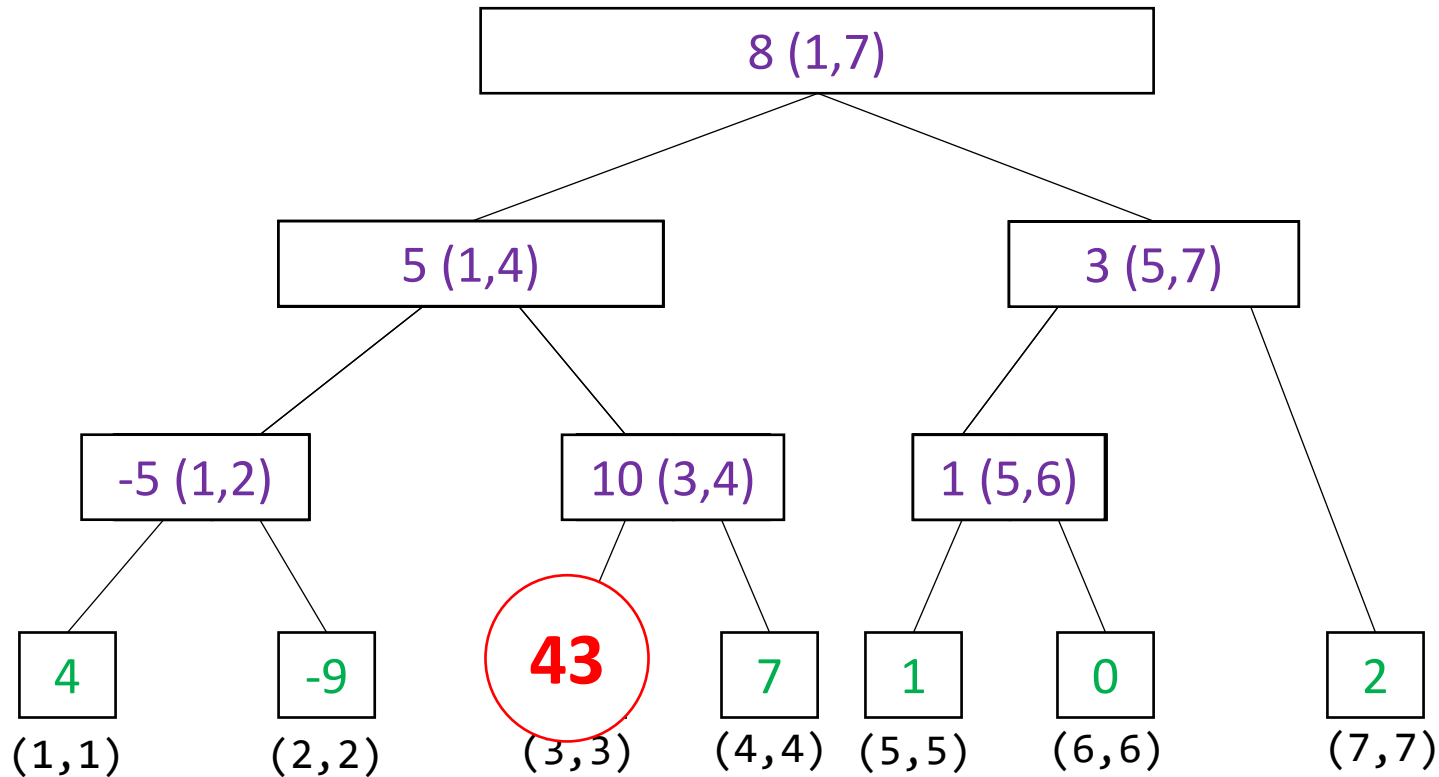


Update:

Suppose, we need to update the value represented from index (3,3). New value will be 43.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

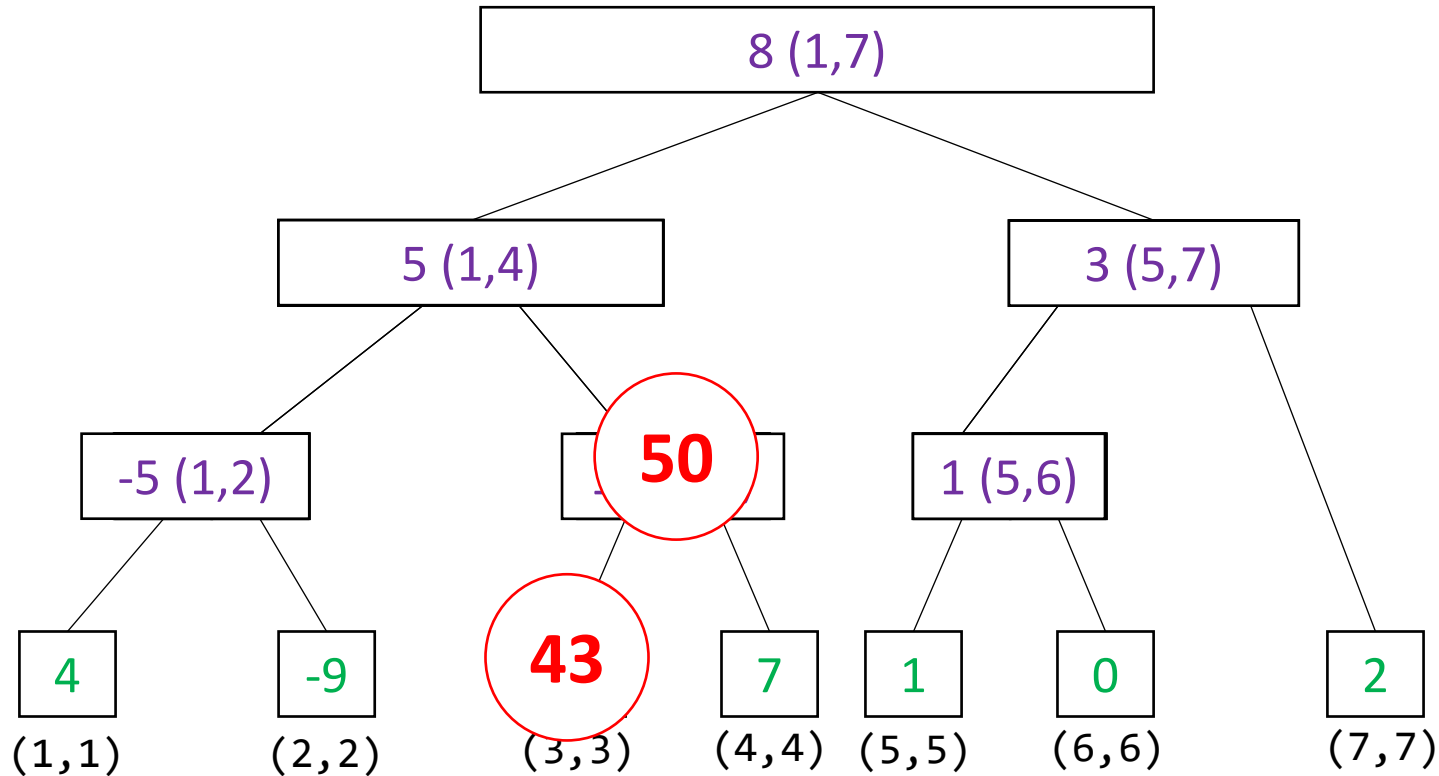


Update:

Suppose, we need to update the value represented from index (3,3). New value will be 43.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

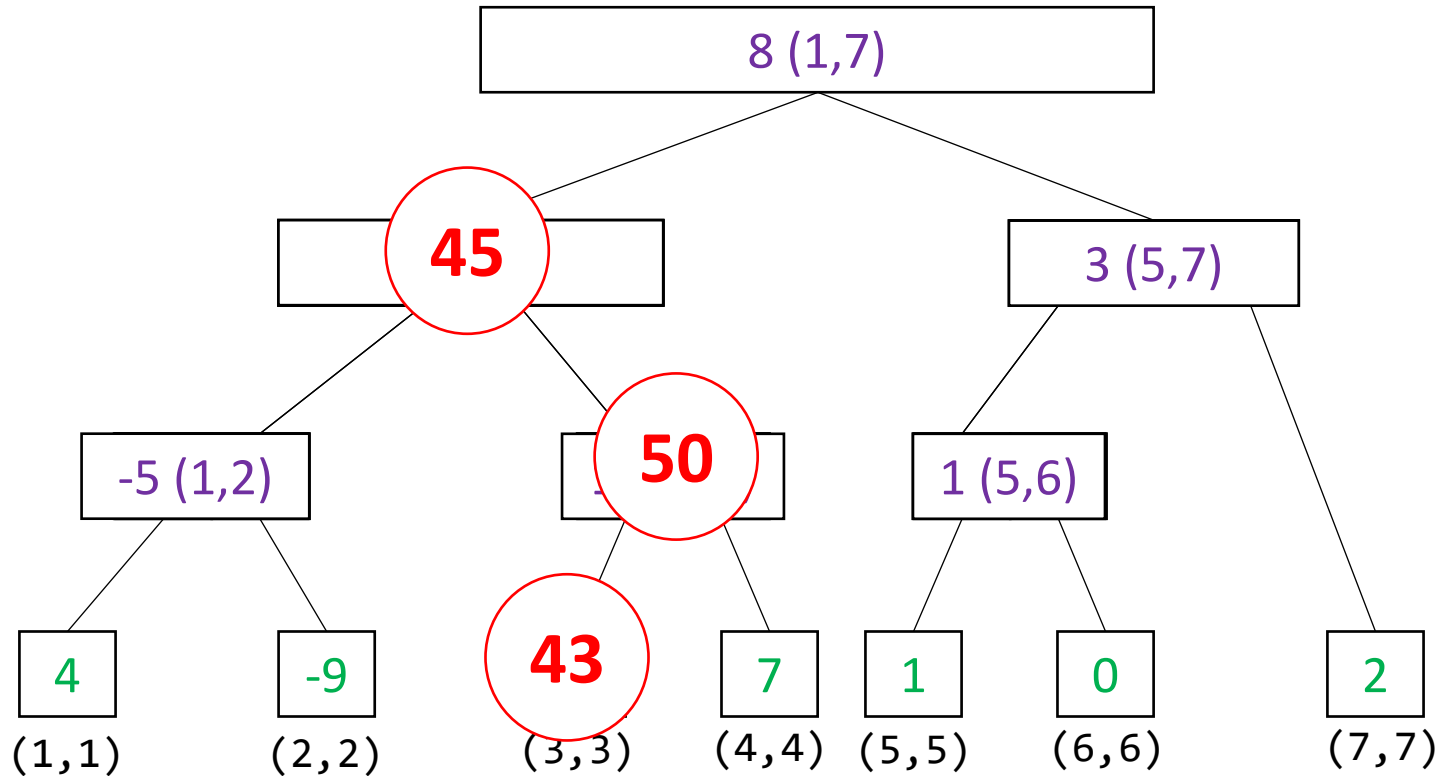


Update:

Suppose, we need to update the value represented from index (3,3). New value will be 43.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

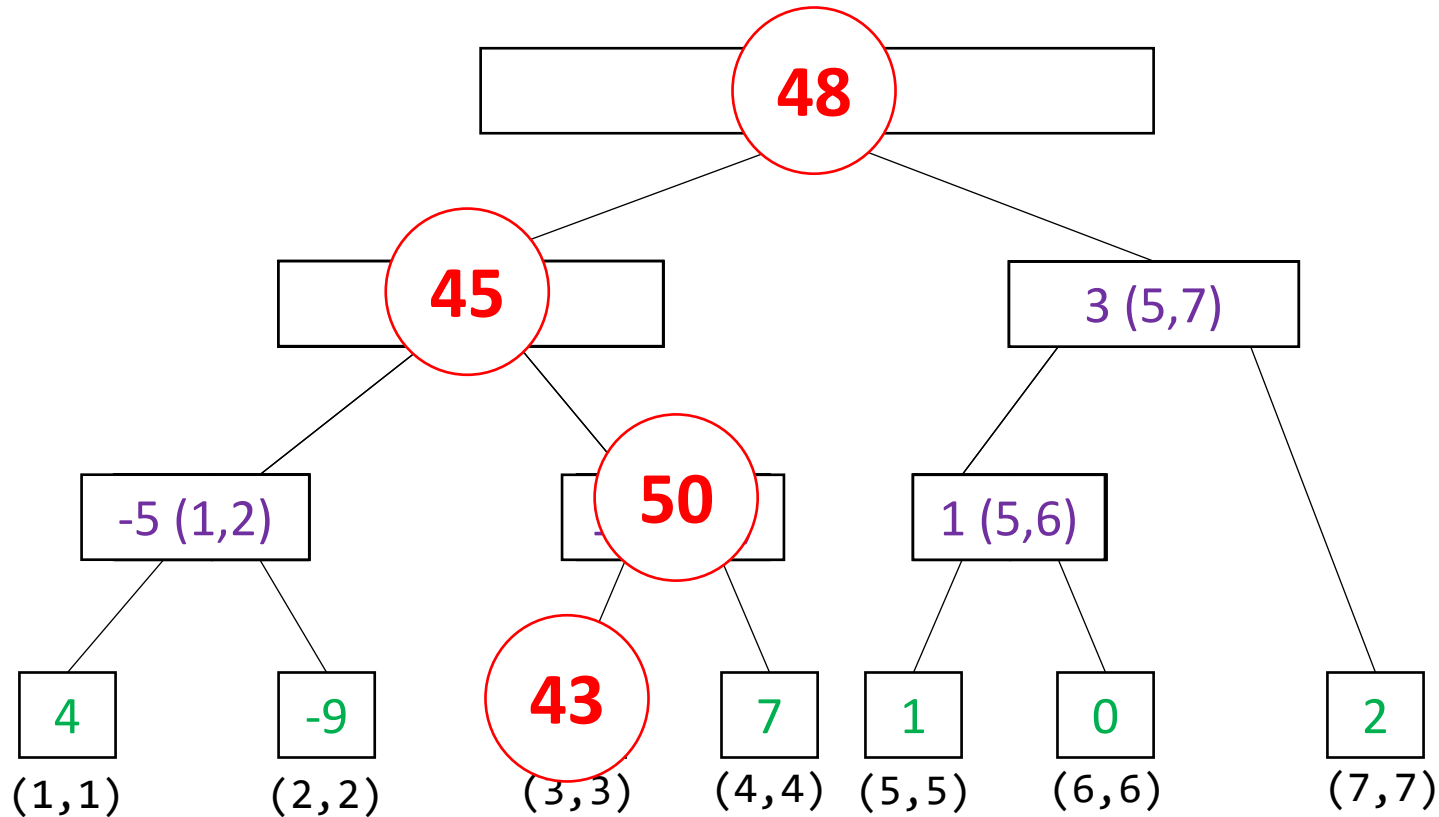


Update:

Suppose, we need to update the value represented from index (3,3). New value will be 43.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

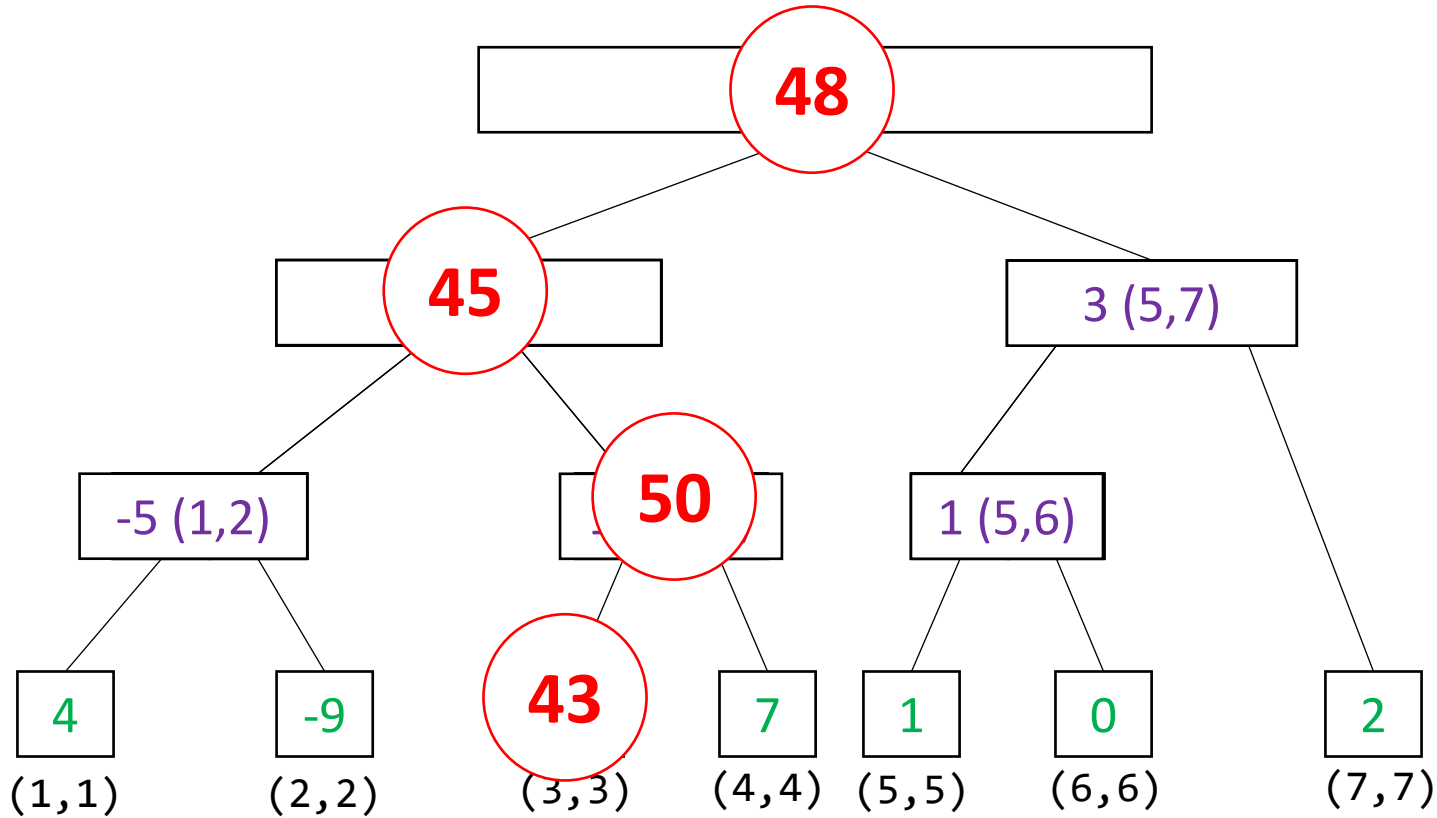


Update:

Suppose, we need to update the value represented from index (3,3). New value will be 43.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

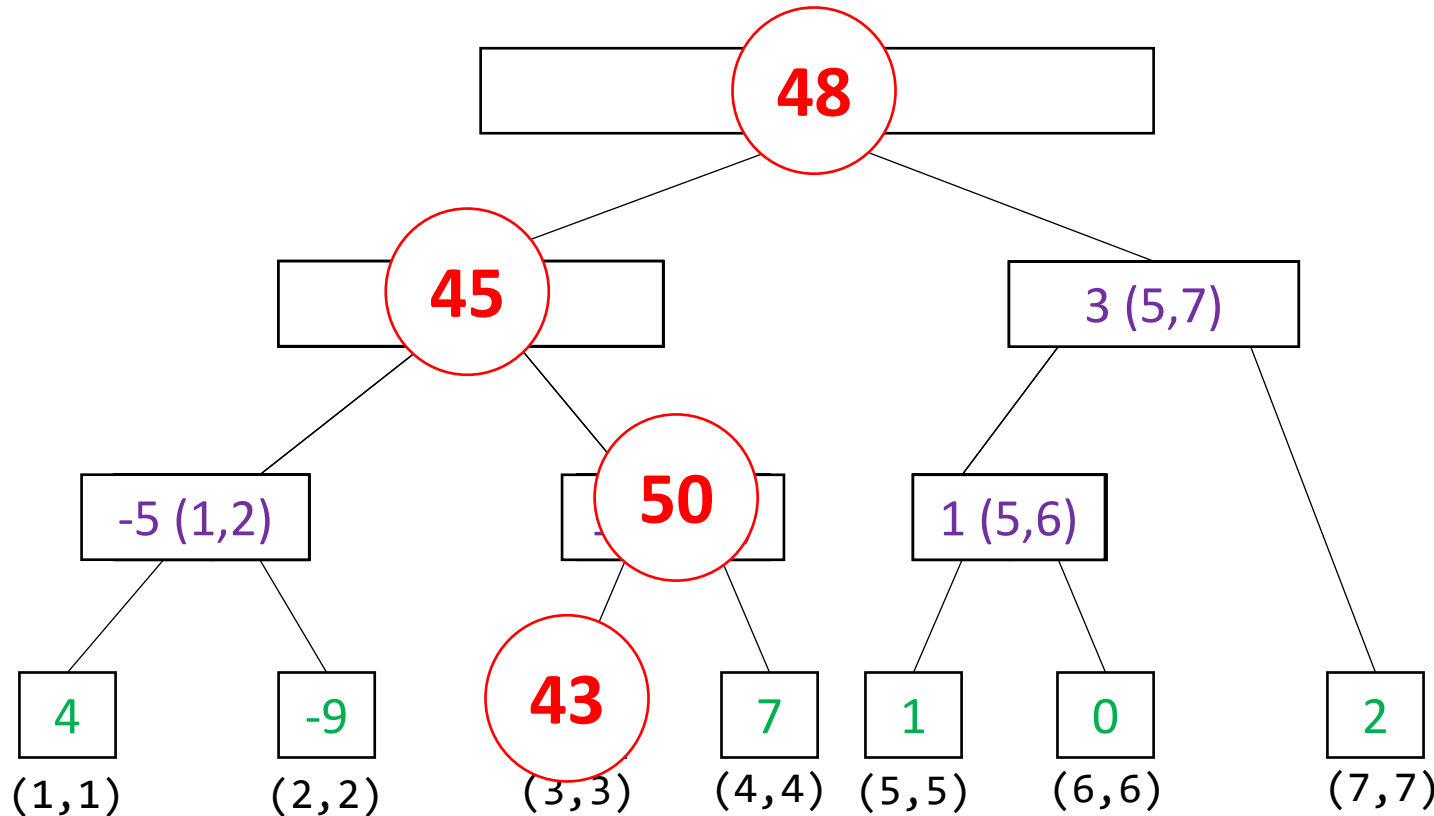


Update:

Suppose, we need the update the value represented from *index* (3,6).
Add 5 with each value.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



Update:

Suppose, we need to update the value represented from *index (3,6)*.

Add 5 with each value.

Adding 5 with index-3, then index 4, then 5, then 6.

Very Inefficient.

Use the lazy-propagation technique.

```
1 int update (int node, int b, int e, int i, int newValue) {  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13 }
```

```
1 int update (int node, int b, int e, int i, int newValue) {  
2     if(i>e || i<b)  
3         return          //out of range  
  
4  
5  
6  
  
7  
8  
9  
  
10  
11  
  
12  
13 }
```

```
1 int update (int node, int b, int e, int i, int newValue) {  
2     if(i>e || i<b)  
3         return                //out of range  
  
4     if(b==i && e==i)  
5         tree[node] = newValue //position is found  
6         return  
  
7  
8  
9  
  
10  
11  
  
12  
13 }
```



```

1 int update (int node, int b, int e, int i, int newValue) {
2     if(i>e || i<b)
3         return                //out of range
4
5     if(b==i && e==i)
6         tree[node] = newValue //position is found
7         return
8
9     int left= node*2
10    int right= node*2 + 1
11    int mid= (b + e)/2
12
13 }

```

```

1 int update (int node, int b, int e, int i, int newValue) {
2     if(i>e || i<b)
3         return                //out of range
4
5     if(b==i && e==i)
6         tree[node] = newValue //position is found
7         return
8
9     int left= node*2
10    int right= node*2 + 1
11    int mid= (b + e)/2
12
13    update(left, b, mid, i, newValue)
14    update(right, mid+1, e, i, newValue)
15
16 }

```

```

1 int update (int node, int b, int e, int i, int newValue) {
2     if(i>e || i<b)
3         return                //out of range
4
5     if(b==i && e==i)
6         tree[node] = newValue //position is found
7         return
8
9     int left= node*2
10    int right= node*2 + 1
11    int mid= (b + e)/2
12
13    update(left, b, mid, i, newValue)
14    update(right, mid+1, e, i, newValue)
15
16    tree[node] = tree[left] + tree[right]
17 }

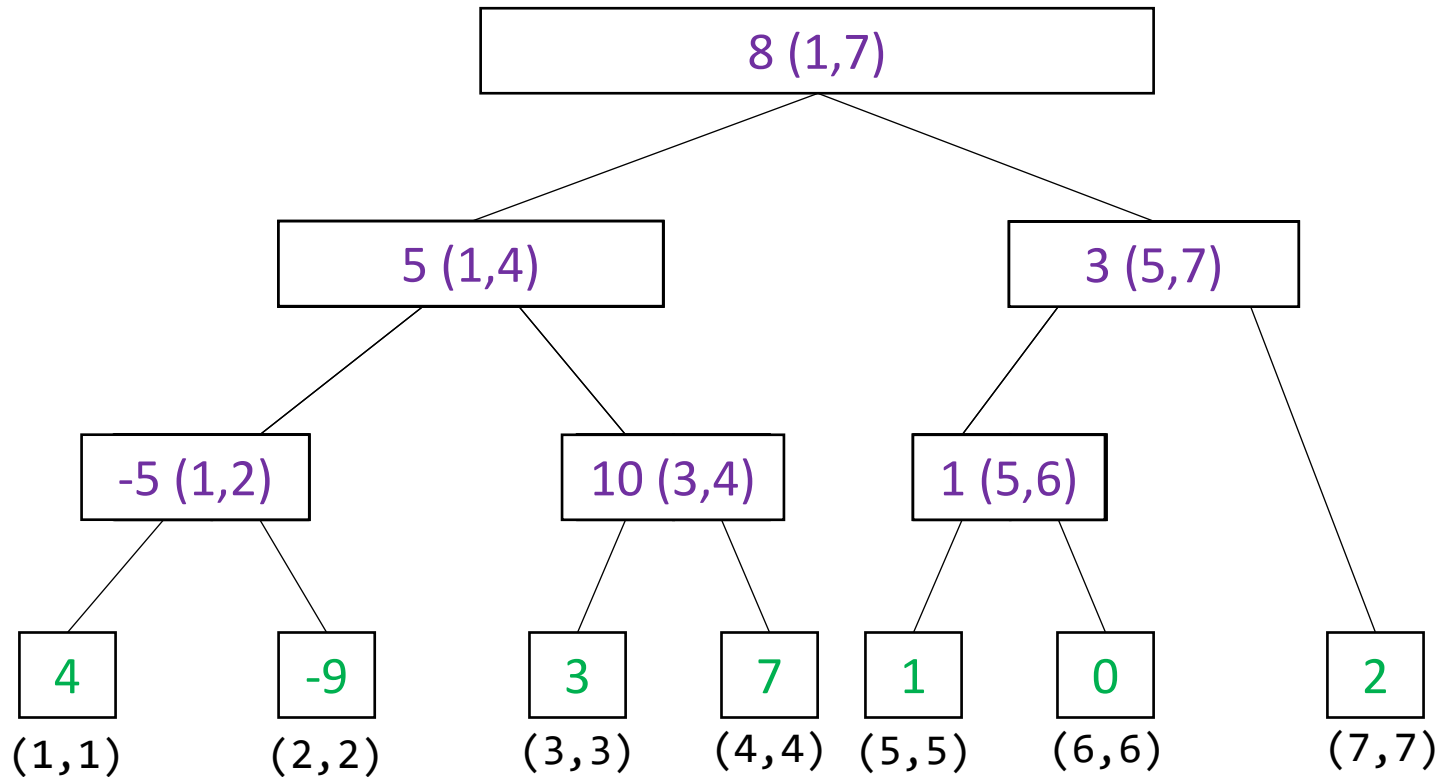
```

Analysis:

- Segment tree splits the array into two portions.
- But the portioning can be done more than the height of the tree. For each QUERY the complexity will be $O(\log N)$.
- The init function goes to each node once, resulting in a complexity of $O(N \log N)$.

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



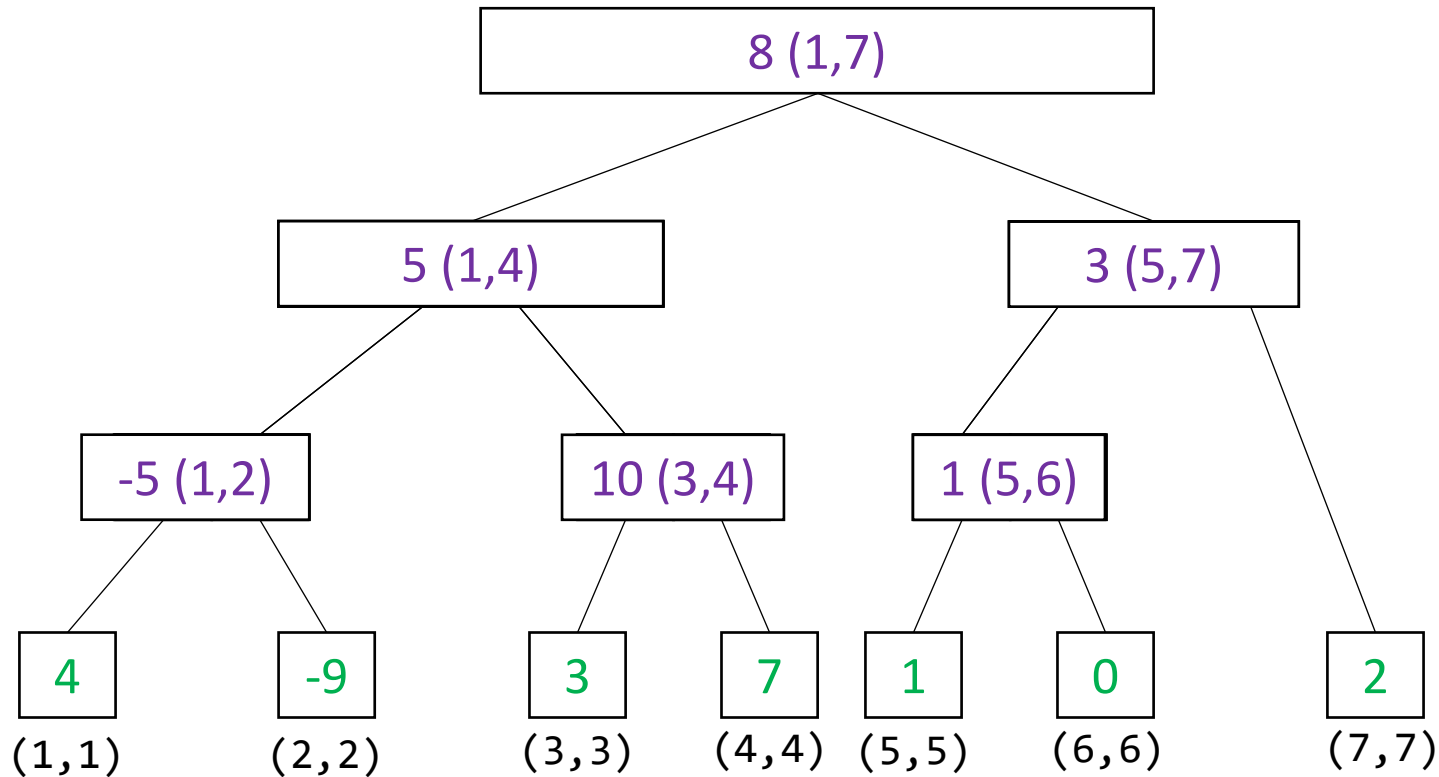
Update:

Add 5 to all values within range [1,4].

How?

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



Update:

Add 5 to all values within range [1,4].

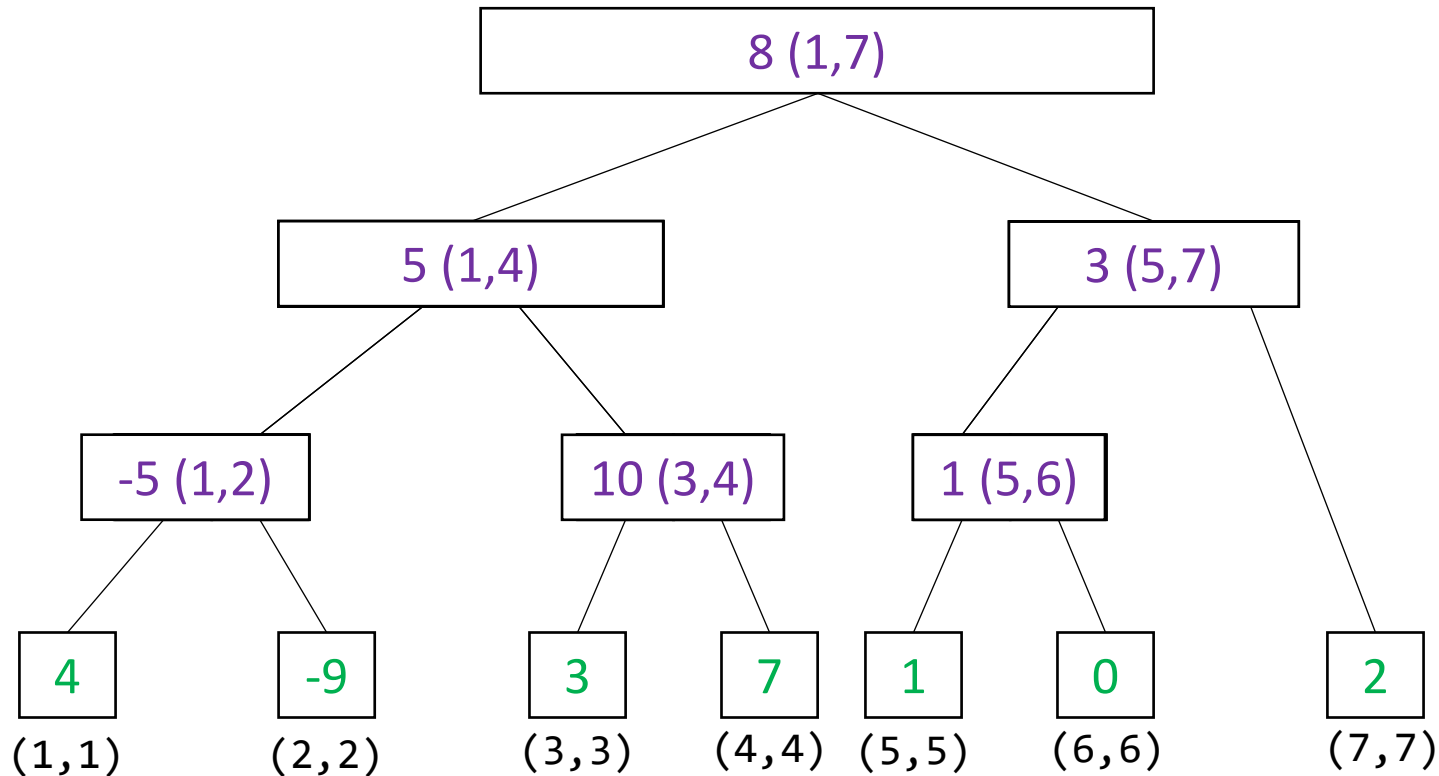
How?

There are 4 numbers in this range
 $(e - b + 1) = (4 - 1 + 1) = 4$

We can apply point update 4 times and update one at a time!

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



Update:

Add 5 to all values within range [1,4].

How?

There are 4 numbers in this range

$$(e - b + 1) = (4 - 1 + 1) = 4$$

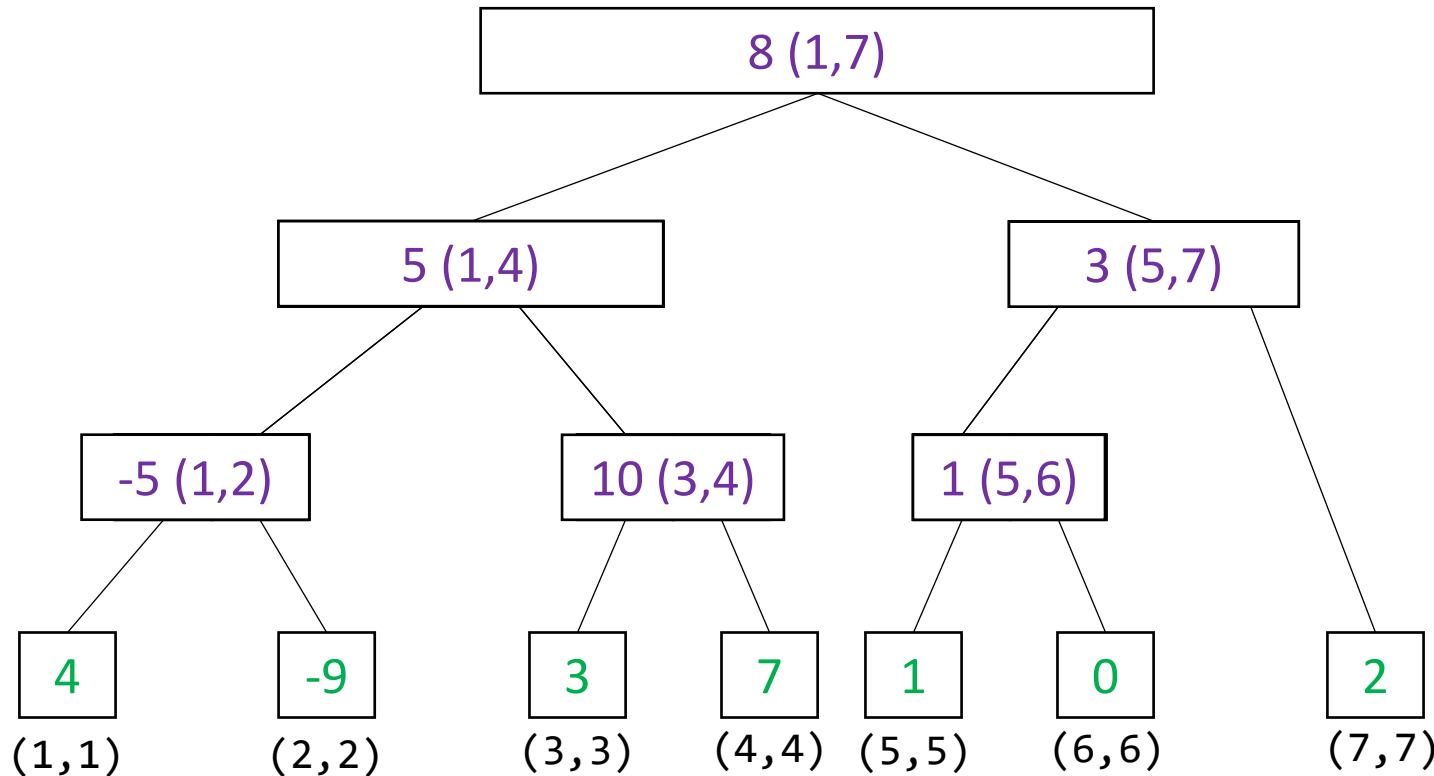
We can apply point update 4 times and update one at a time!

Costly idea!

For N numbers in the range, update complexity is $O(N \log N)$

Resulting Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



Lazy propagation technique can bring this complexity to $O(\log N)$

Update:

Add 5 to all values within range [1,4].

How?

There are 4 numbers in this range

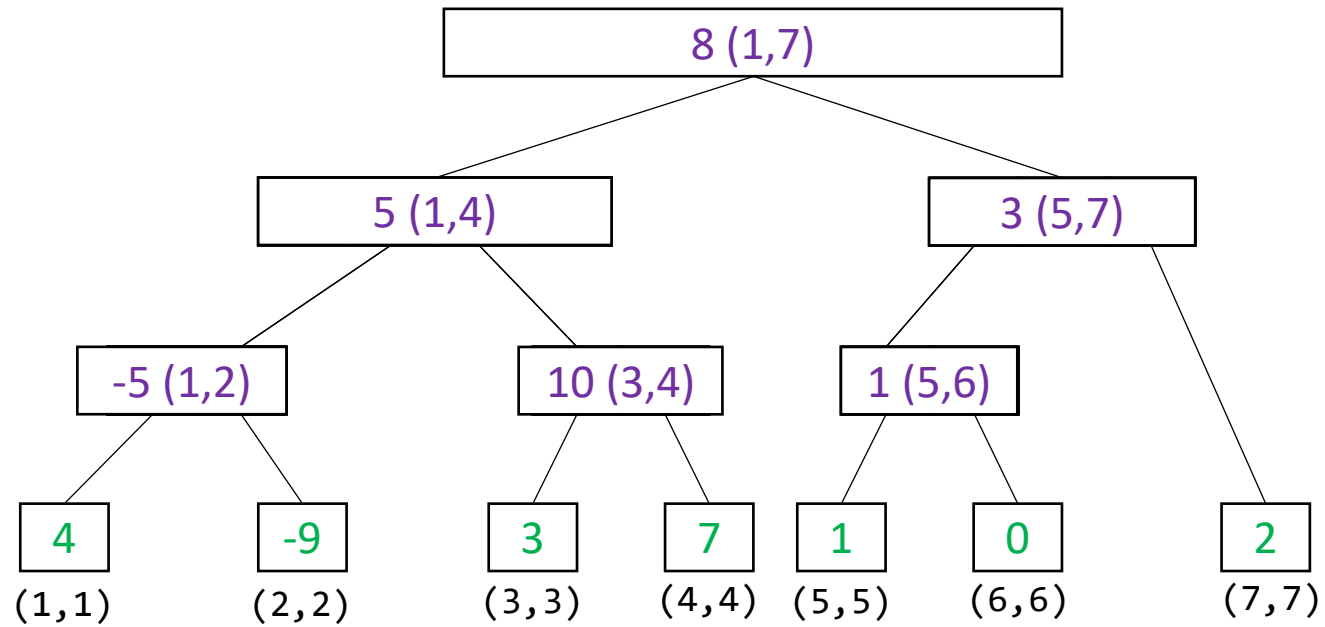
$$(e - b + 1) = (4 - 1 + 1) = 4$$

We can apply point update 4 times and update one at a time!

Costly idea!

For N numbers in the range, update complexity is $O(N \log N)$

Lazy propagation is a trick that lets a segment tree handle range updates efficiently, without pushing the update all the way down to all affected leaves immediately.

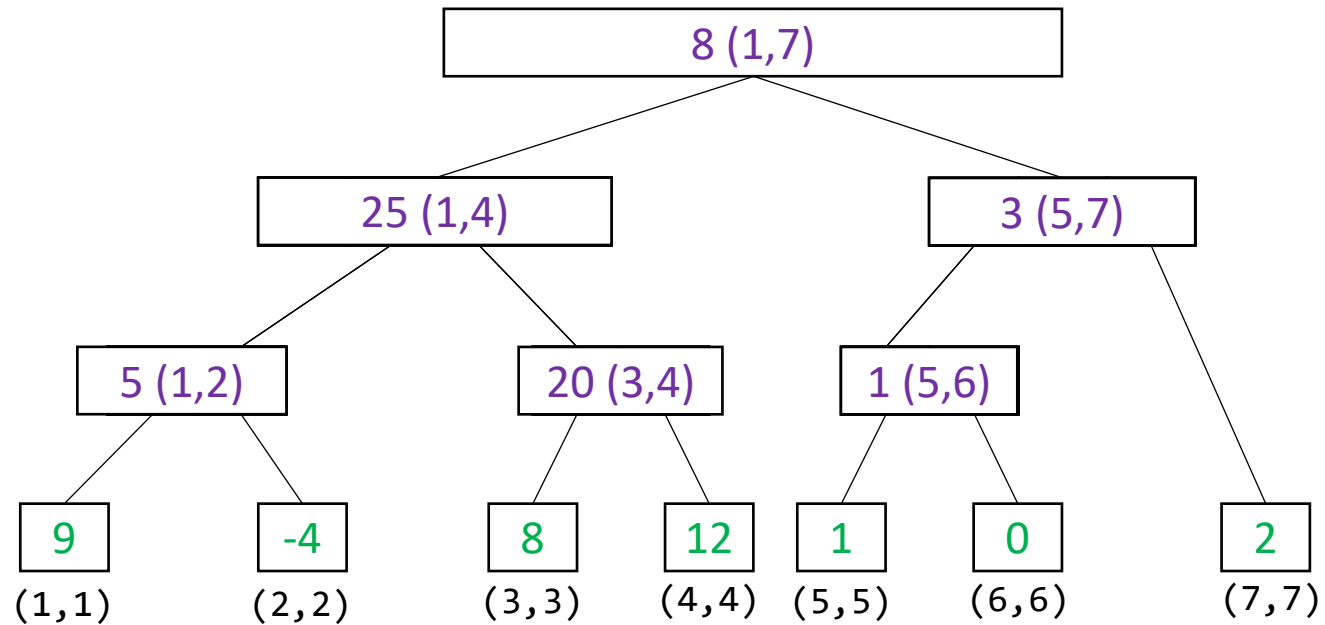


Concept of Lazy-propagation:

Task: Increment [1,4] by 5.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

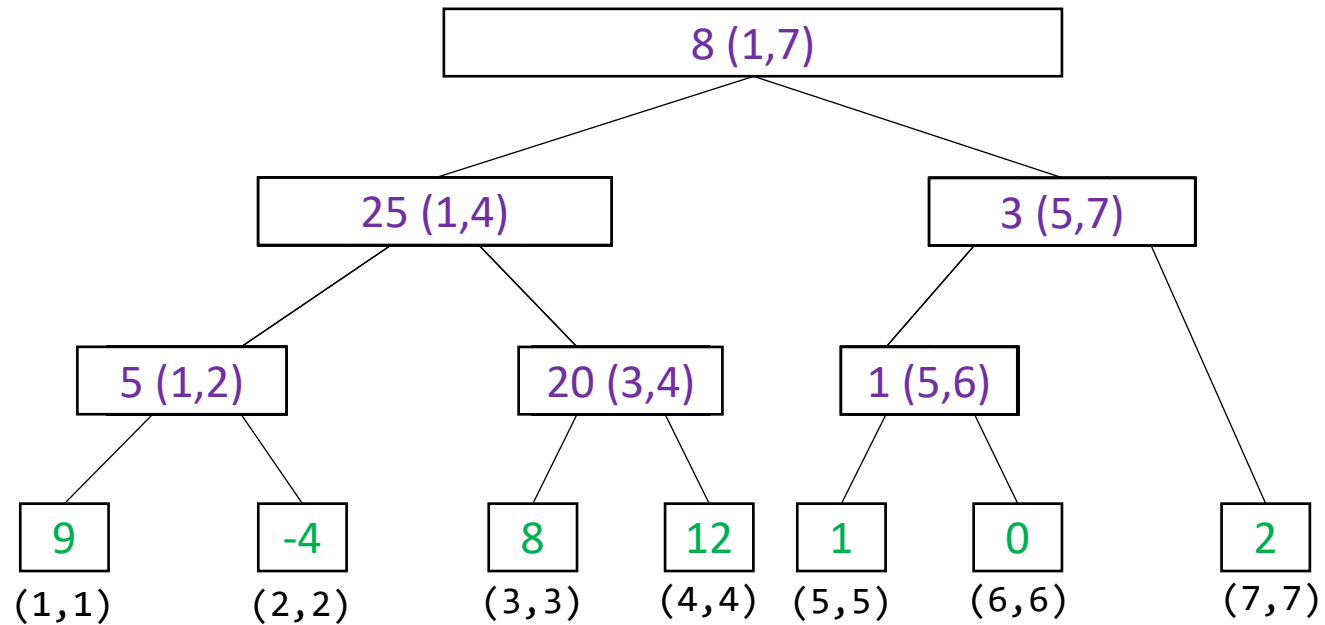


Concept of Lazy-propagation:

Task: Increment [1,4] by 5.

Solution 1:

Update index 1,2,3,4 and reflect in the tree.

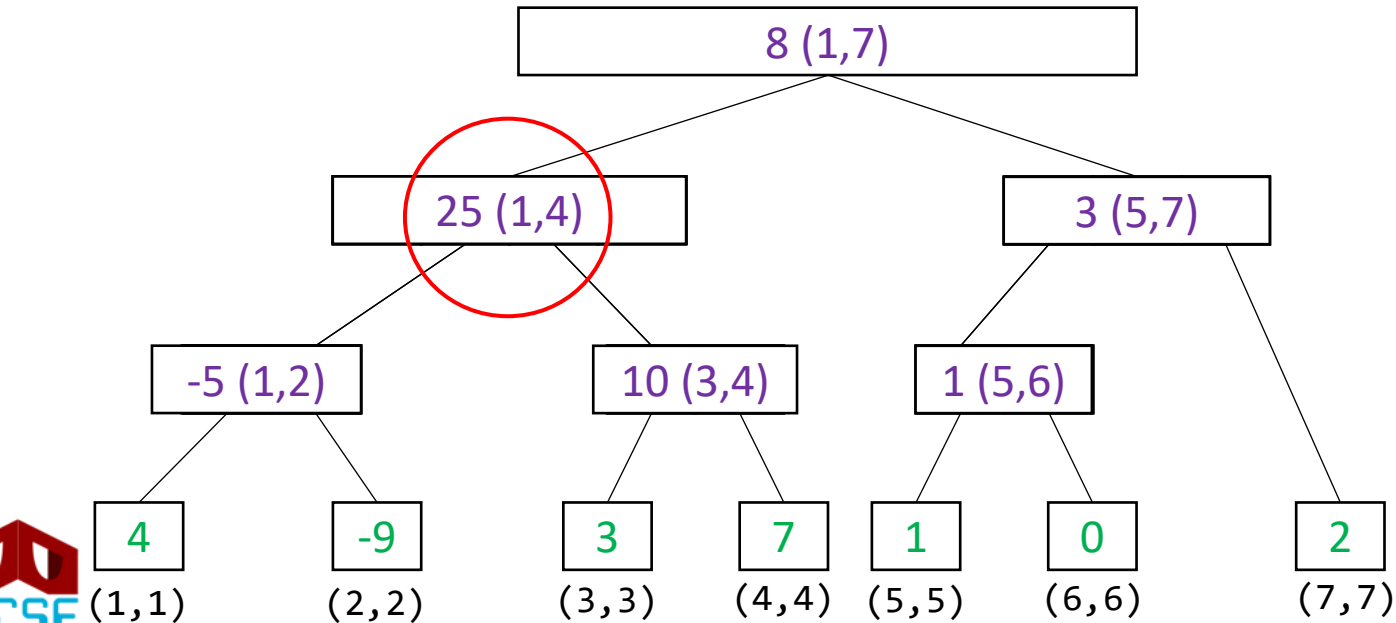


Concept of Lazy-propagation:

Task: Increment $[1,4]$ by 5.

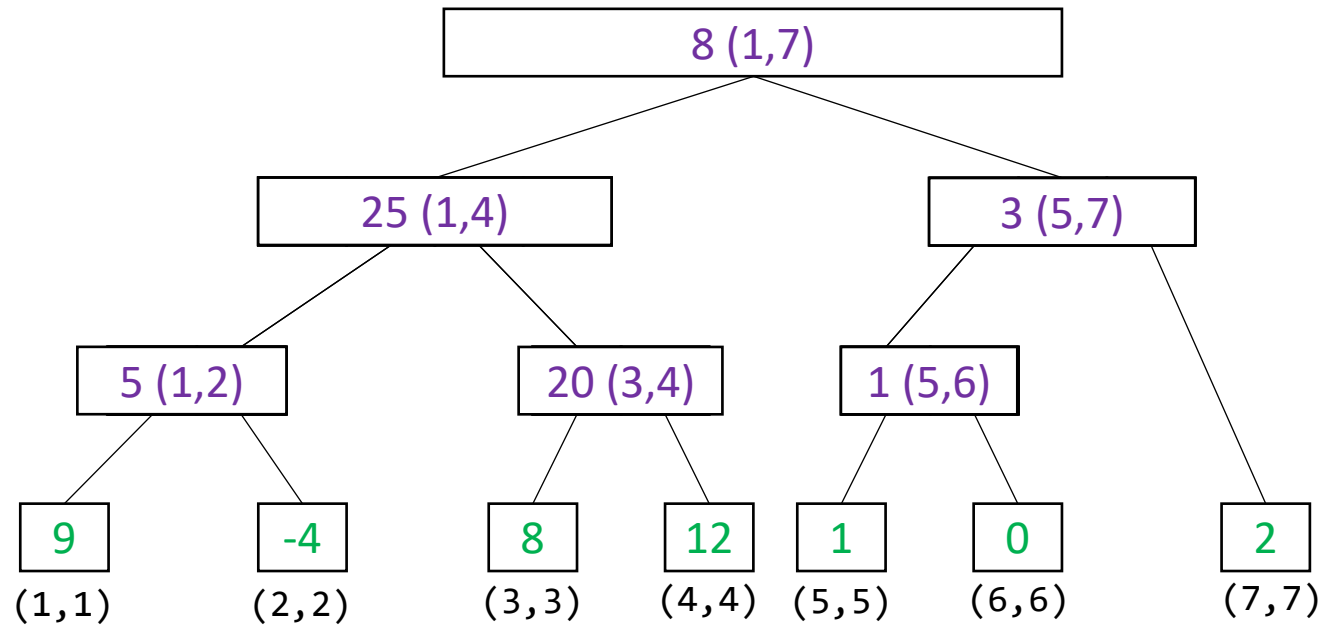
Solution 1:

Update index 1,2,3,4 and reflect in the tree.



Solution 2:

Only update index-2 of Segment tree!

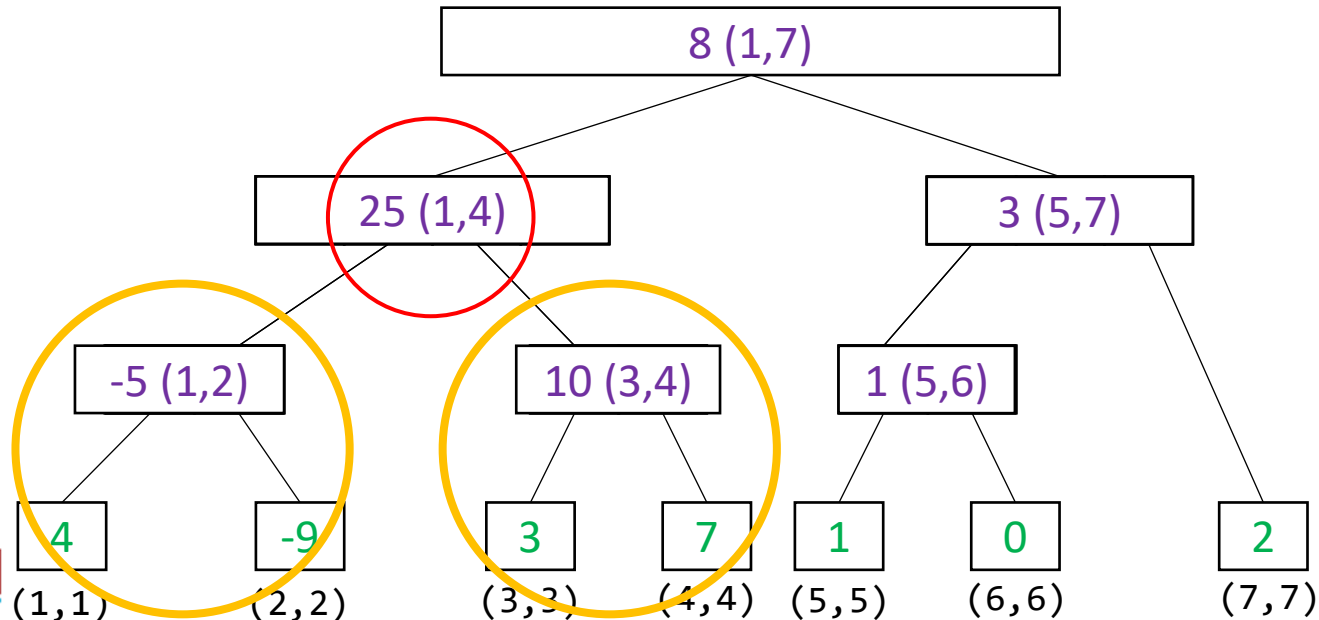


Concept of Lazy-propagation:

Task: Increment $[1,4]$ by 5.

Solution 1:

Update index 1,2,3,4 and reflect in the tree.

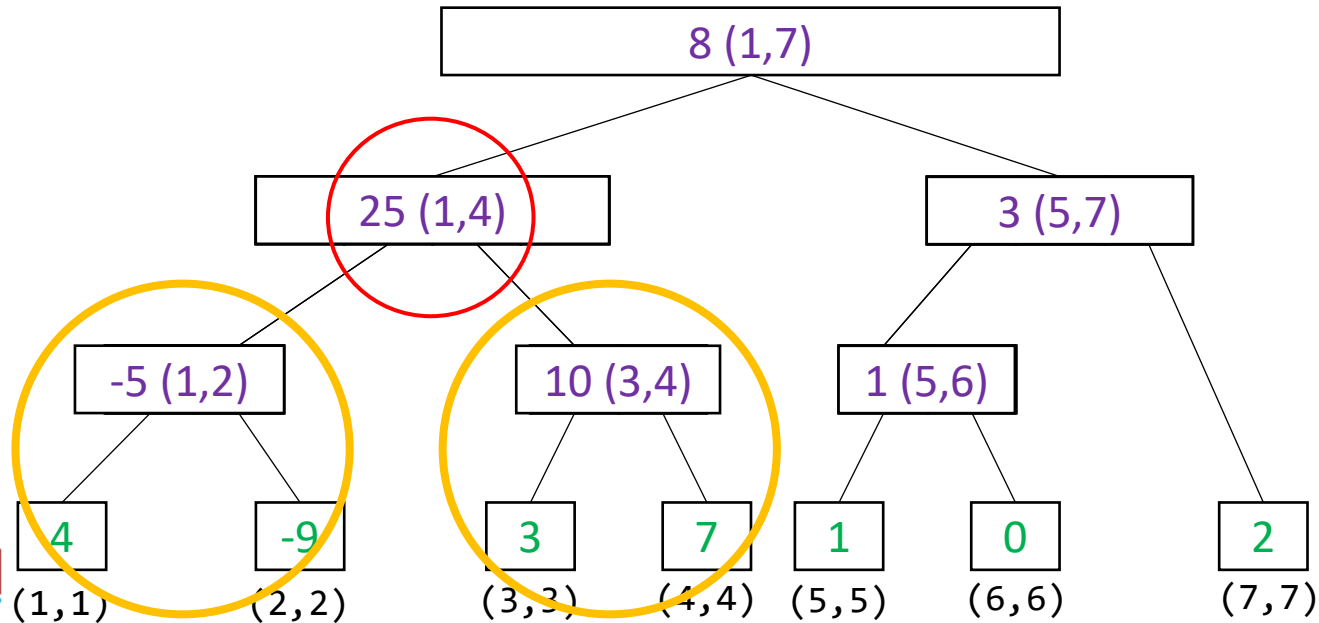


Solution 2:

Only update index-2 of Segment tree!

Welcome to the world of lazy propagation!

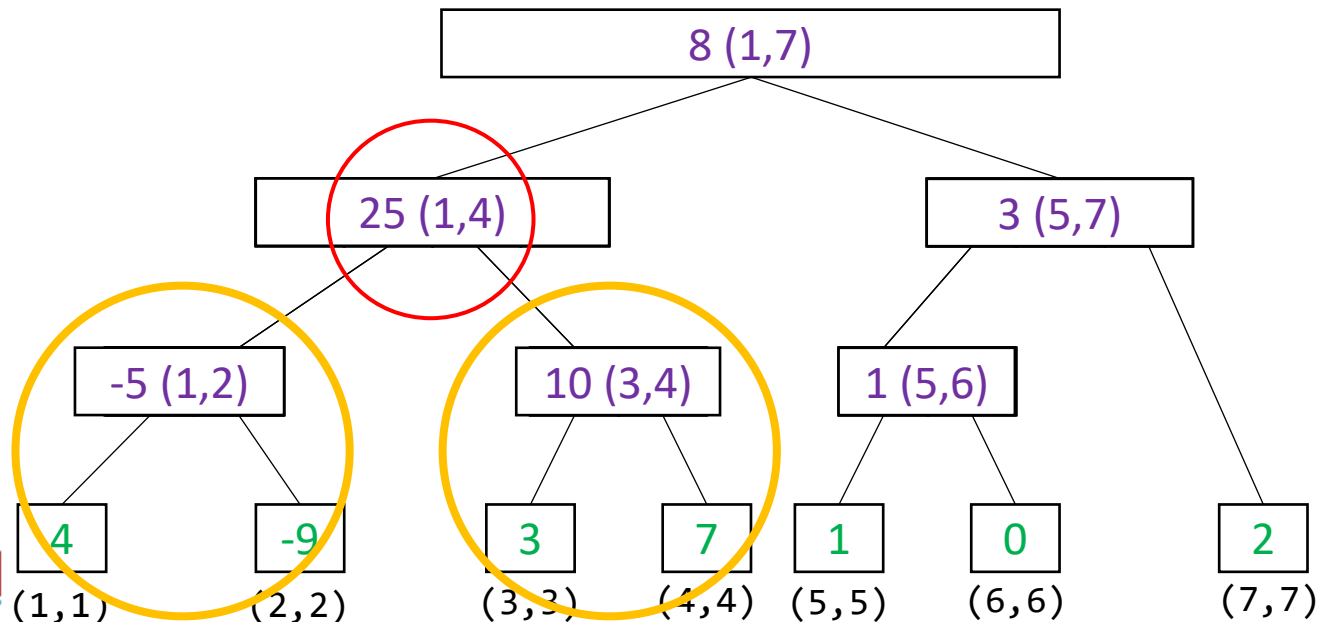
Keep procrastinating as long as you can.



Solution 2:
Only update index-2 of Segment tree!

*Welcome to the world of lazy propagation!
Keep procrastinating as long as you can.*

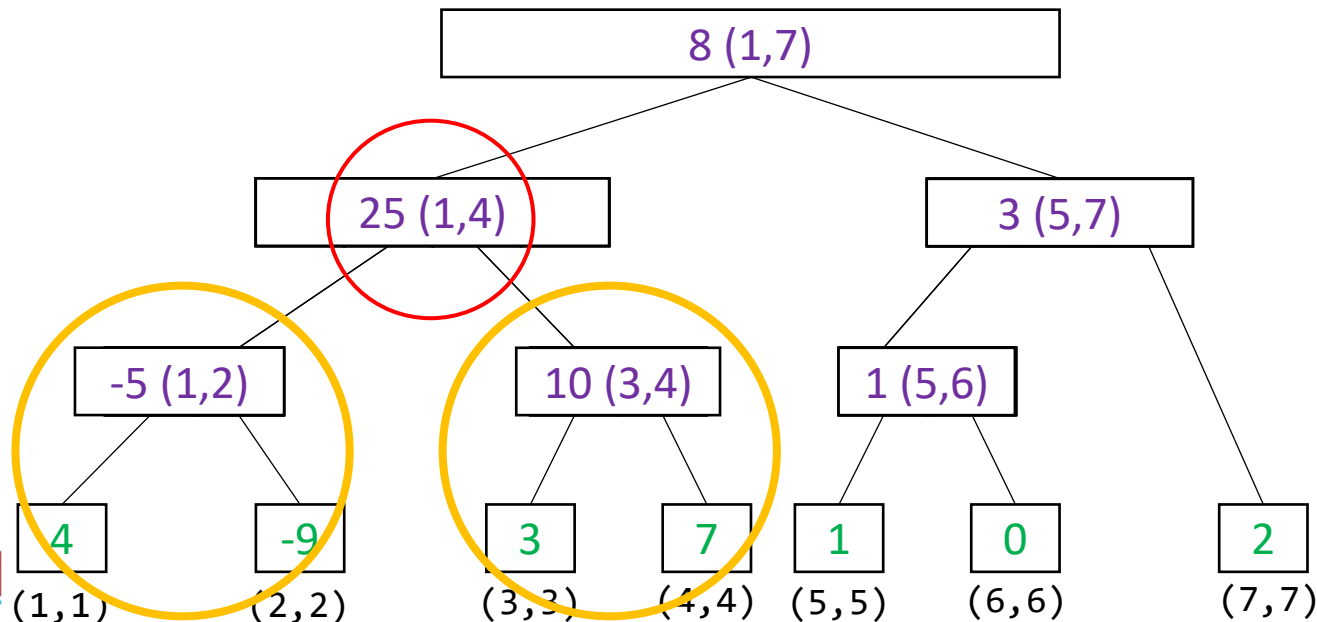
*We'll keep track of the pending tasks separately.
Next time, when we need to visit Node 4, 5, 8, or 9, we'll update them accordingly.*



Solution 2:
Only update index-2 of Segment tree!

*Welcome to the world of lazy propagation!
Keep procrastinating as long as you can.*

*We'll keep track of the pending tasks separately.
Next time, when we need to visit Node 4, 5, 8, or 9, we'll update them accordingly.*

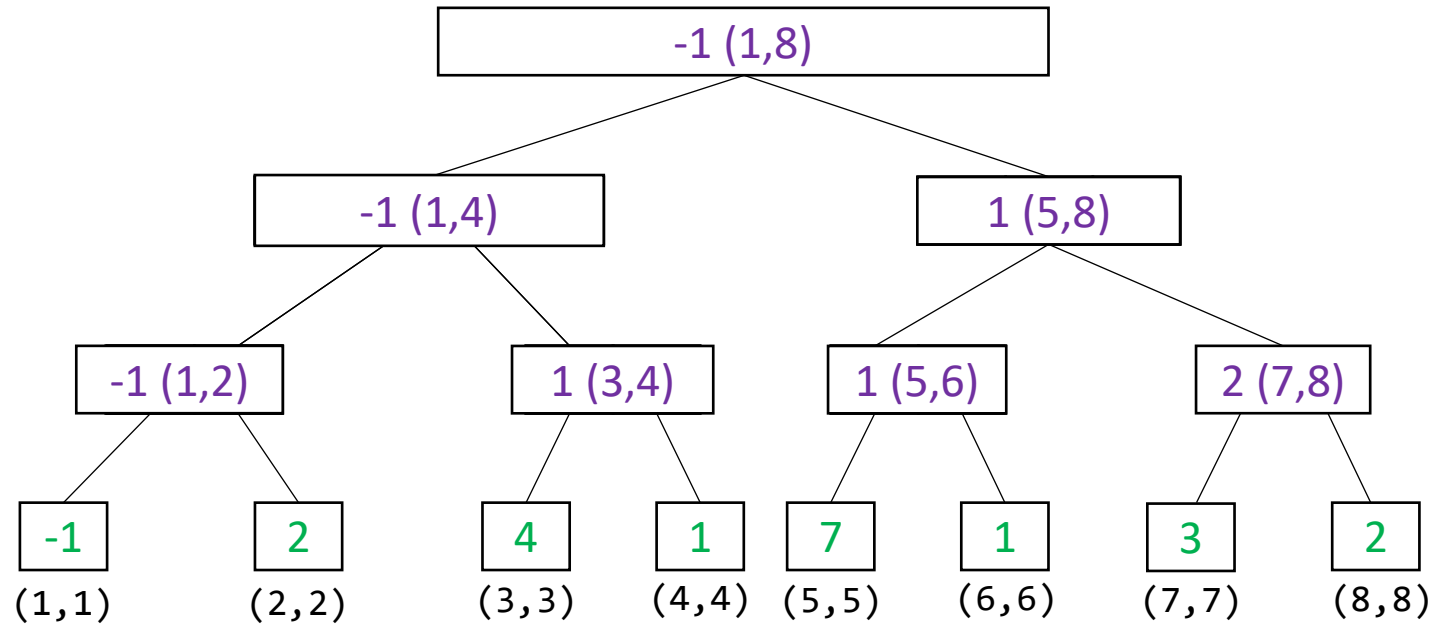


*Solution 2:
Only update index-2 of the
Segment tree!*

*Notice: Here we added 20 with 5,
as the range has 4 numbers.*

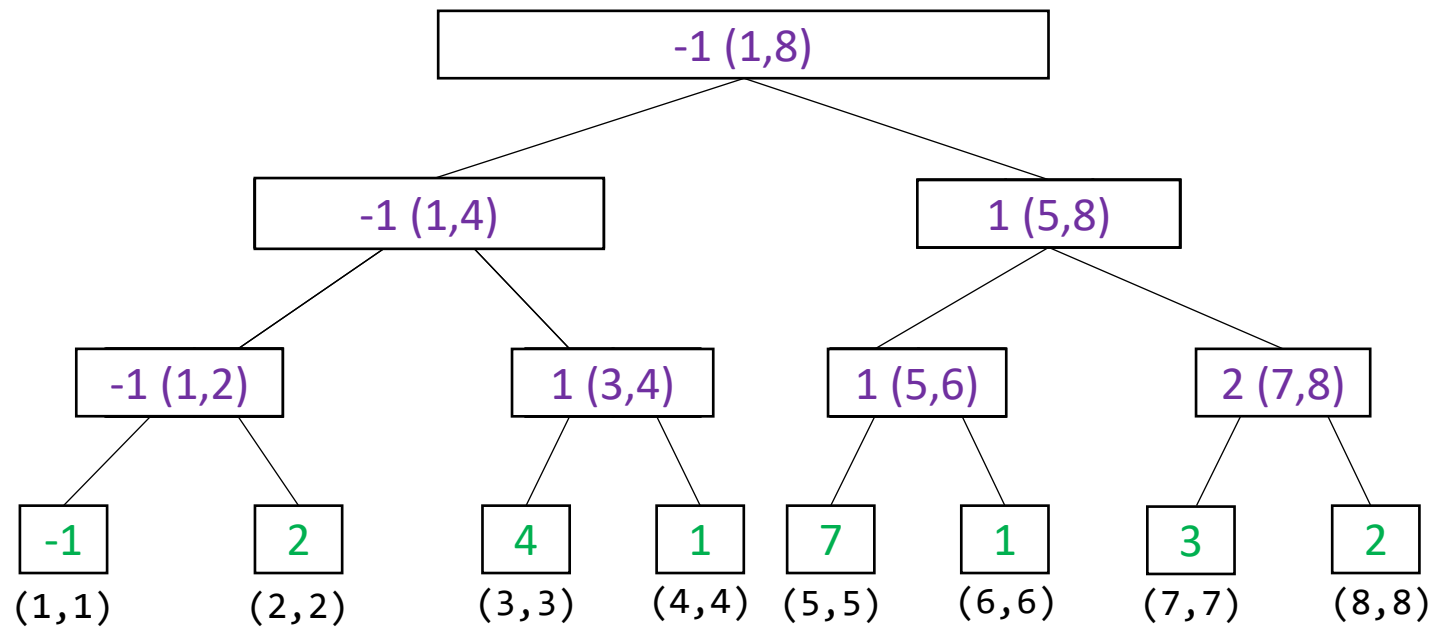
Will you always do this?

Min Segment tree



Segment Tree

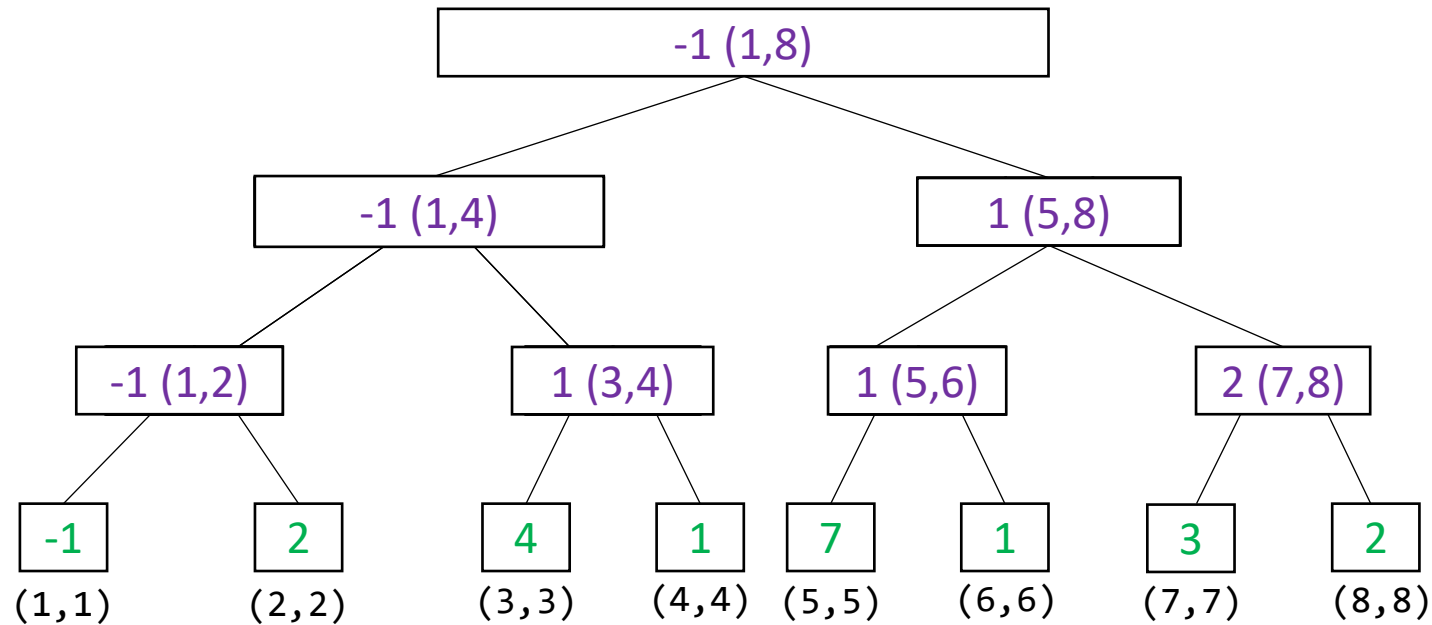
-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

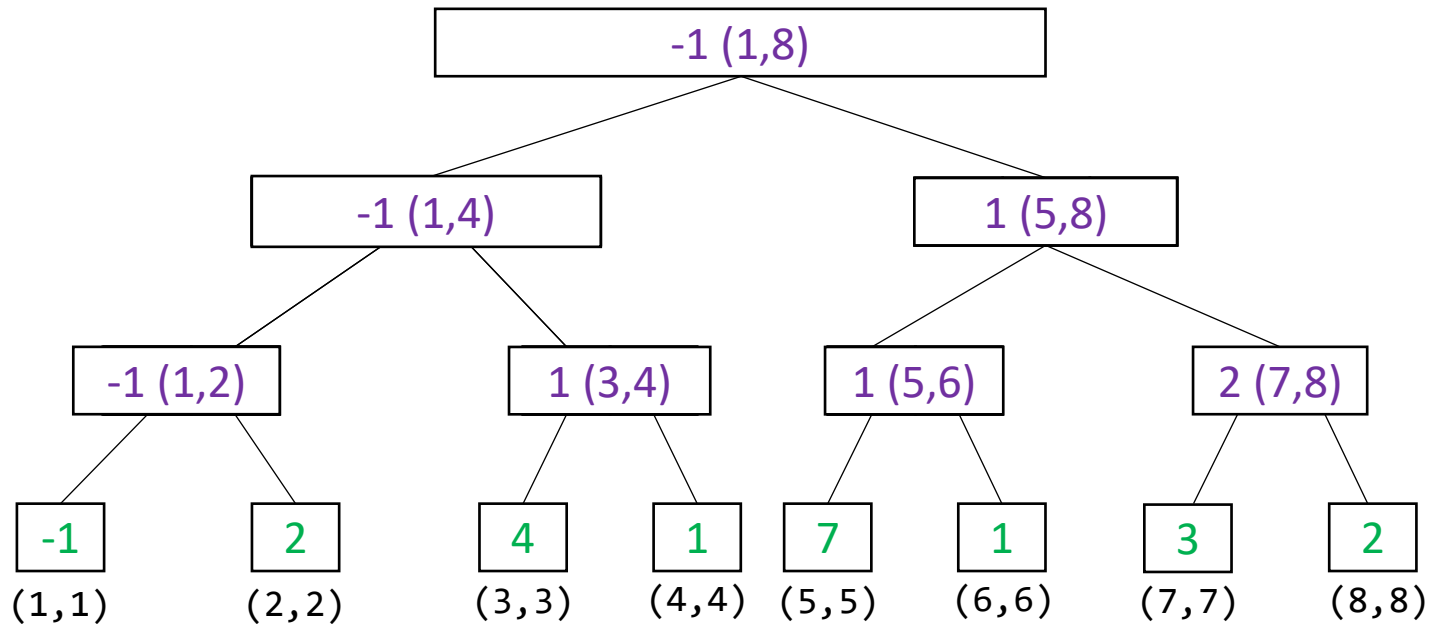
- Declare a parallel tree/array to keep track of the pending tasks. (Initially, all 0)

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

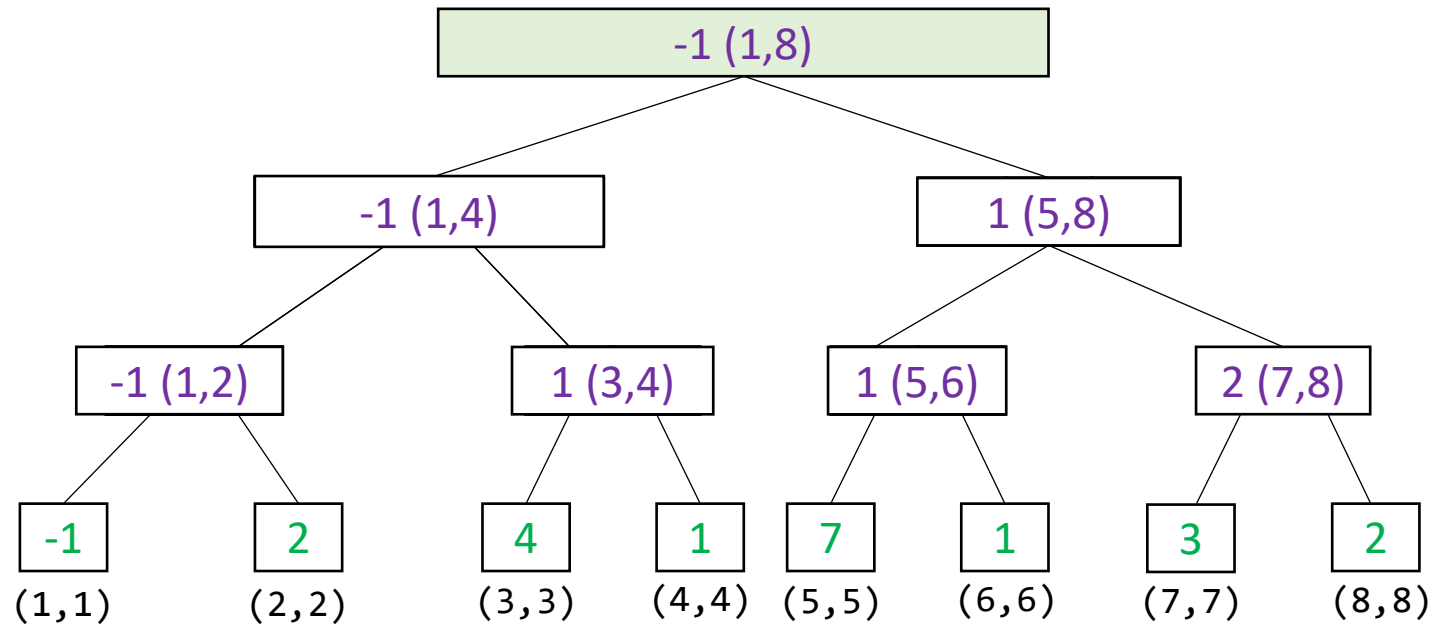
- Declare a parallel tree/array to keep track of the pending tasks. (Initially, all 0)
- Before considering a node, check if it's updated (meaning corresponding lazy-value=0. (If not, update it.)

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

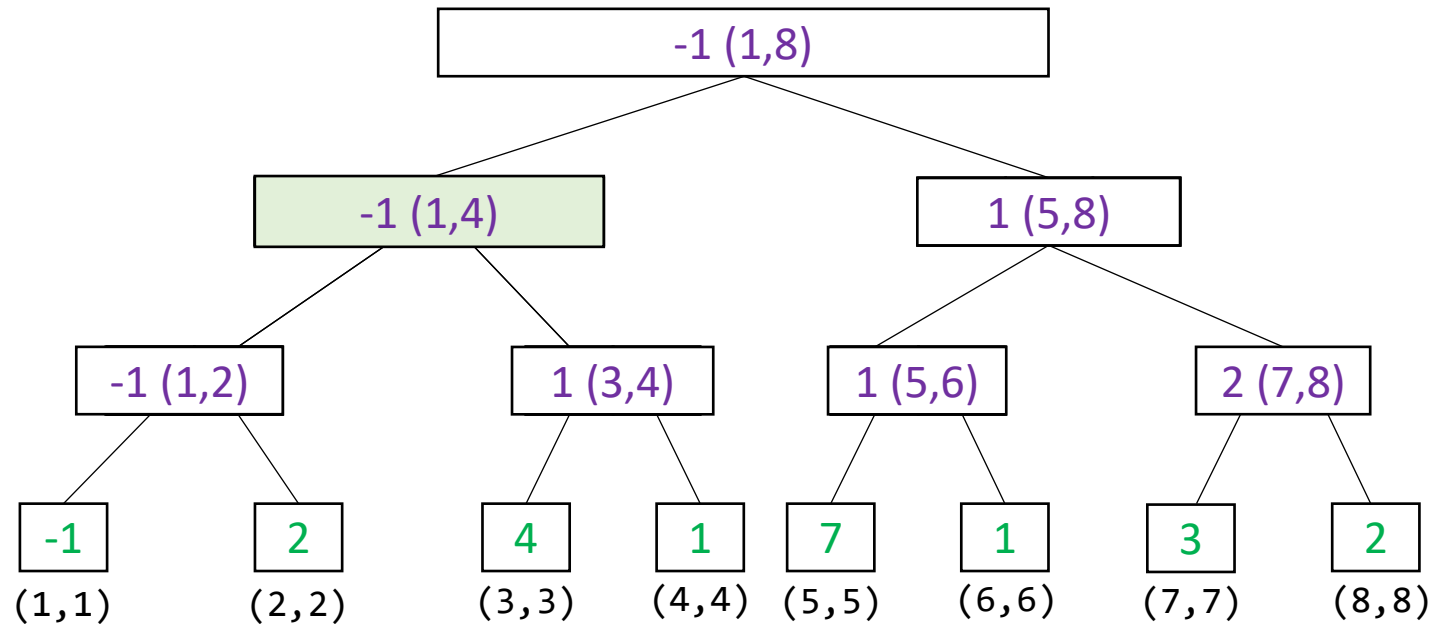
- Start at root [1,8]
- Partial overlap
- Check left and right subtree

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

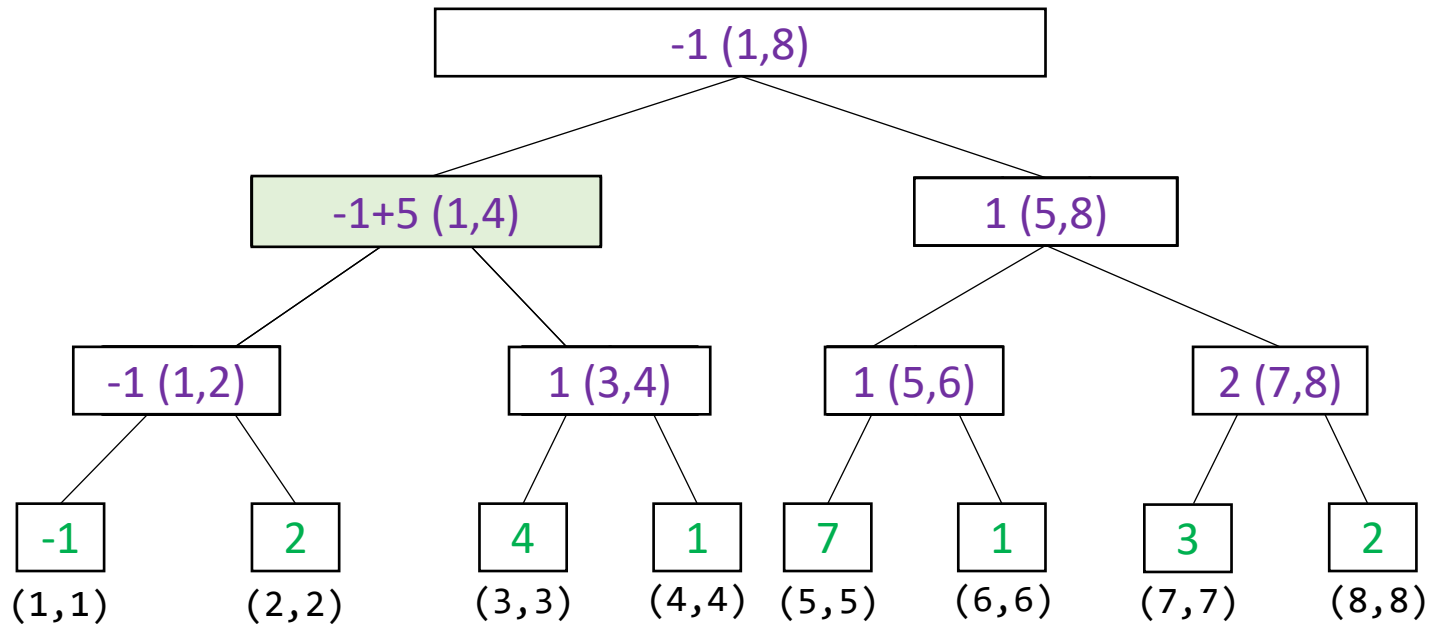
- Full overlap. Update this node, don't dig further.
- Keep track of the pending task in the lazy tree

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

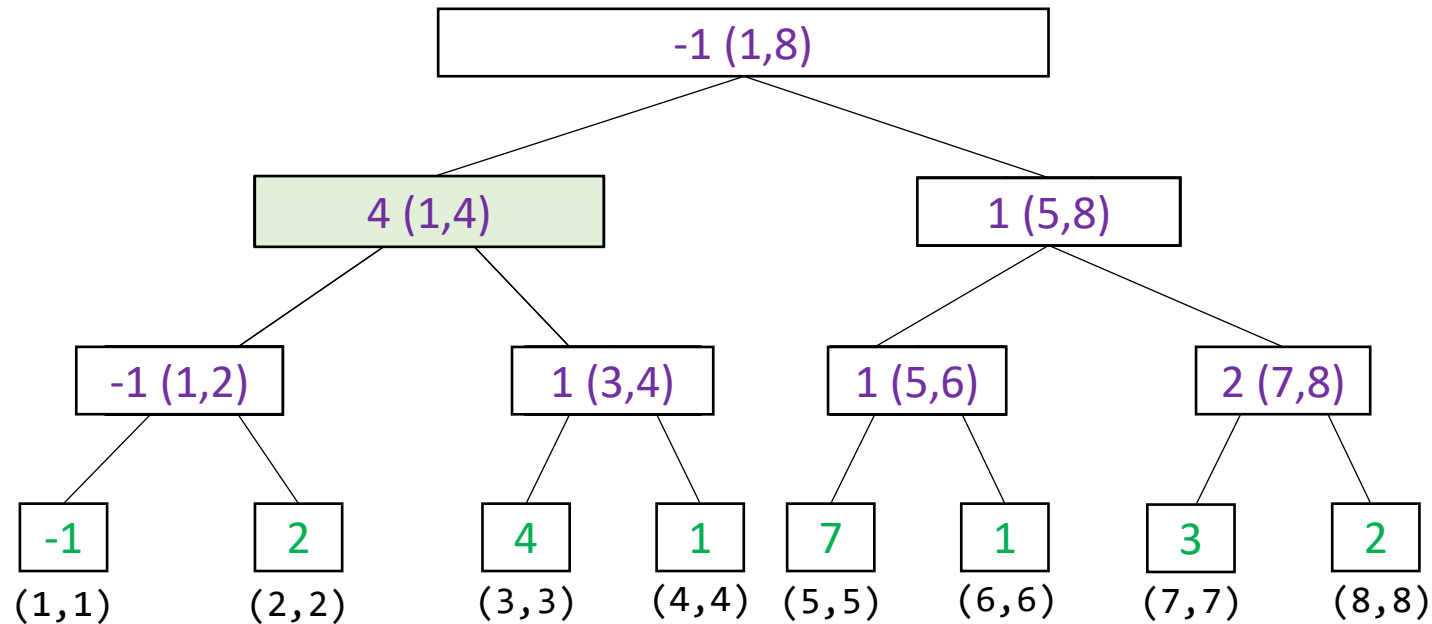
- As this is a min operation, we add 5 to the node (despite there are 4 items in this range).
- If it was sum operation, we would add $4*5=20$. Think why?

Segment Tree

-1	-1	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

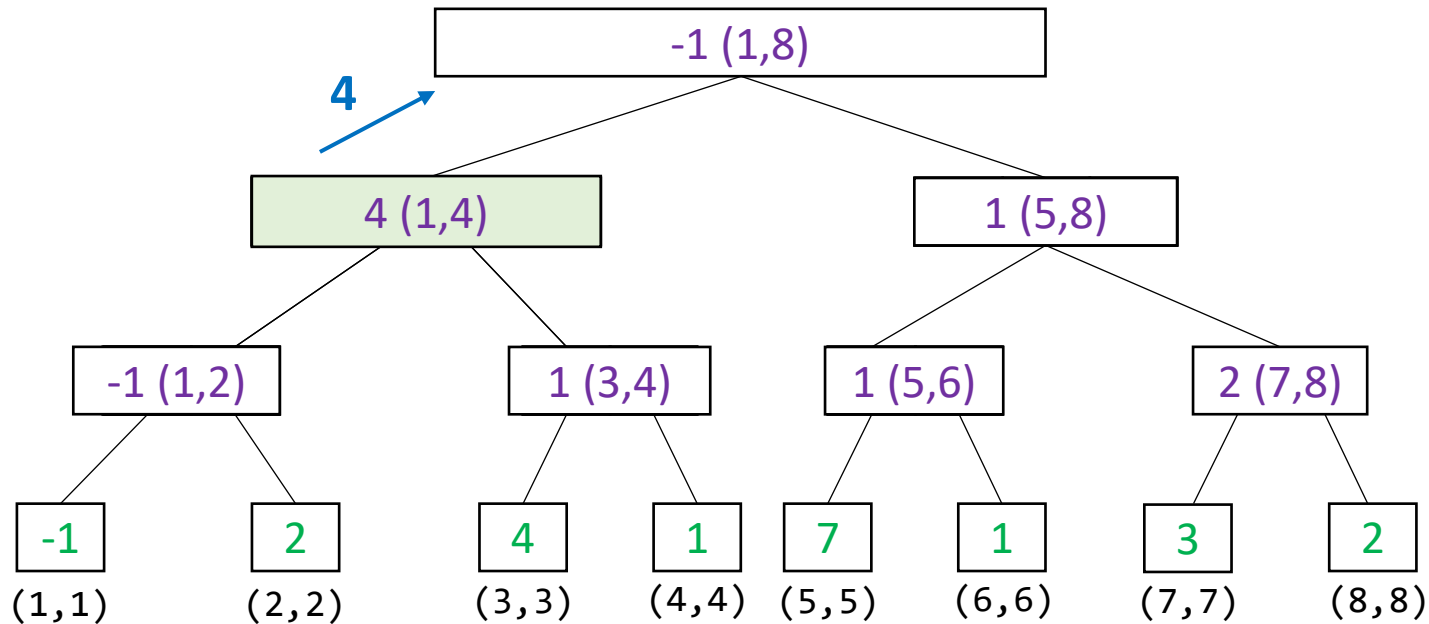
- After updating the node, we update the corresponding children of the lazy tree.
- If somehow, we need to visit them in the future, we'll update them!

Segment Tree

-1	4	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	5	5	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

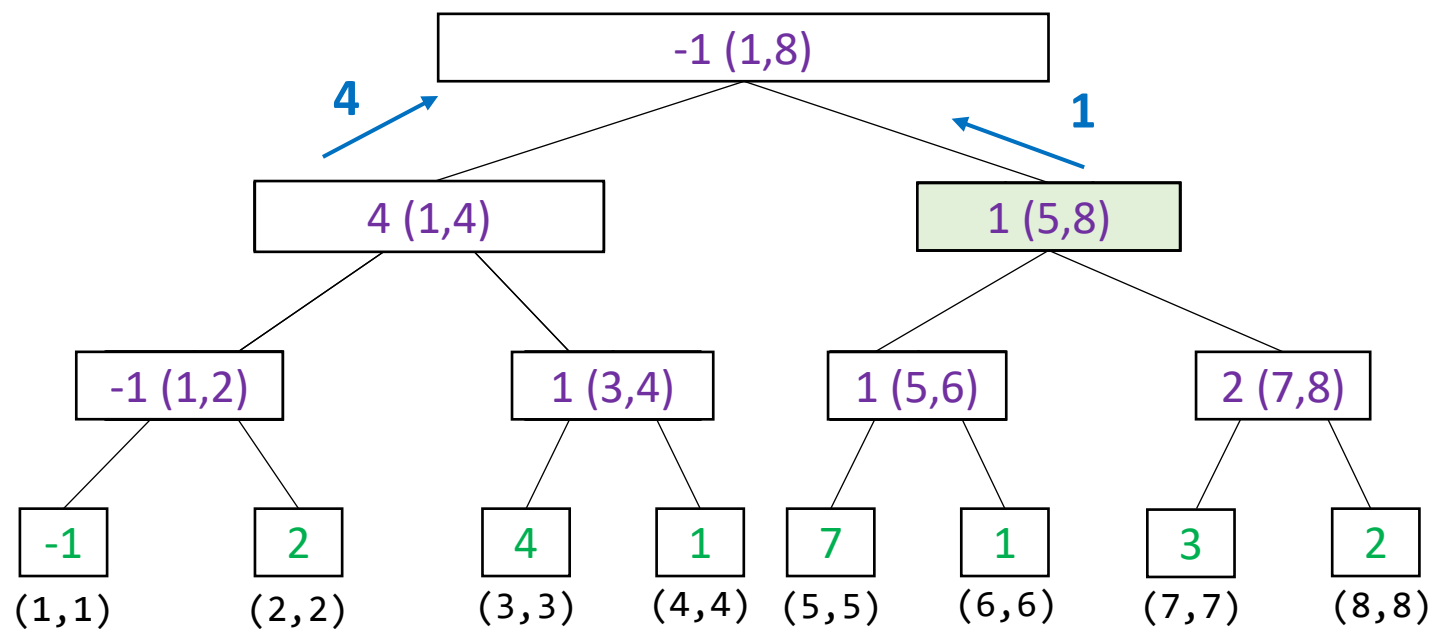
- After updating the node, we update the corresponding children of the lazy tree.
- If somehow, we need to visit them in the future, we'll update them!

Segment Tree

-1	4	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	5	5	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Increment [1,4] by 5

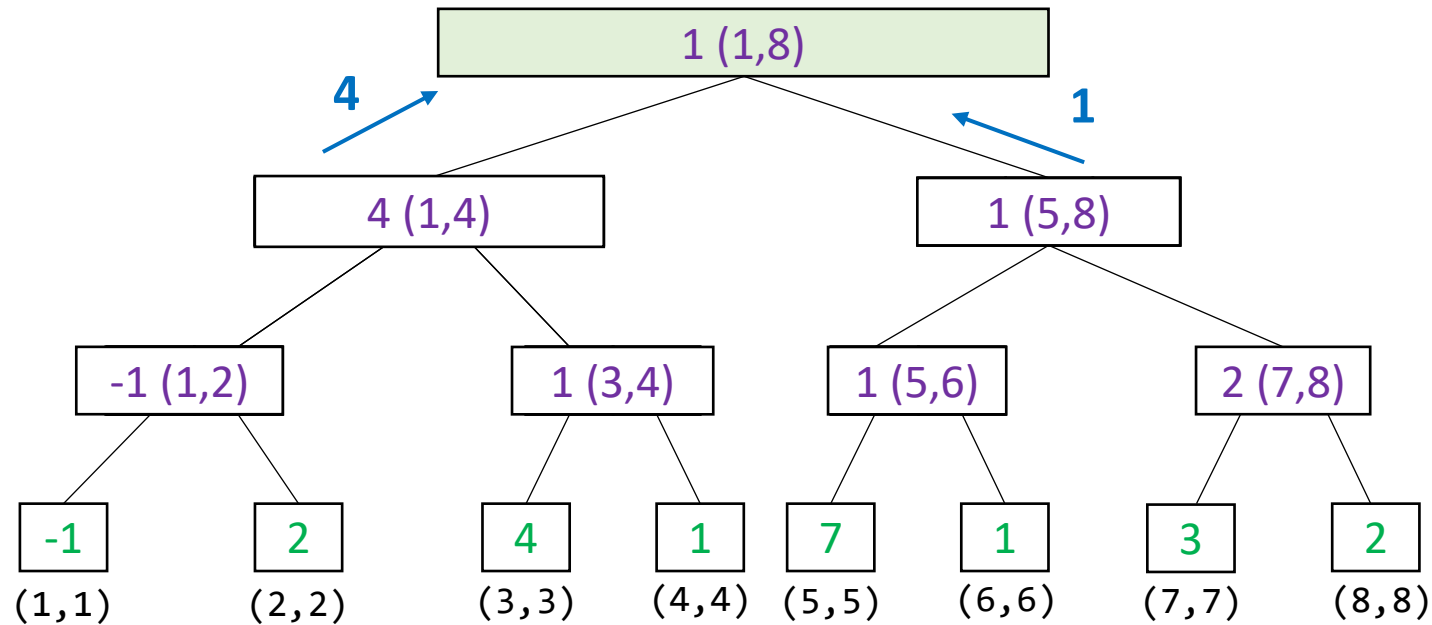
- Go to [5,8]
- No overlap
- Return value

Segment Tree

-1	4	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	5	5	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



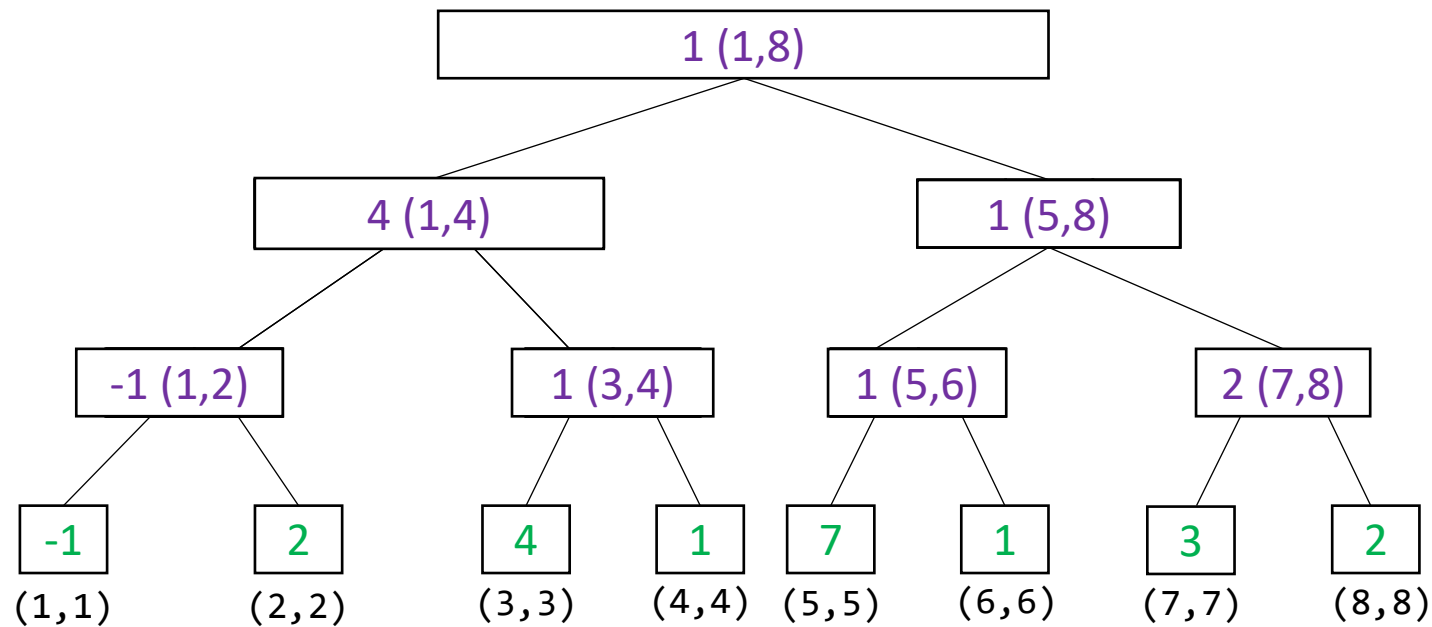
Increment [1,4] by 5
i. Finally, root is updated

Segment Tree

1	4	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	5	5	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



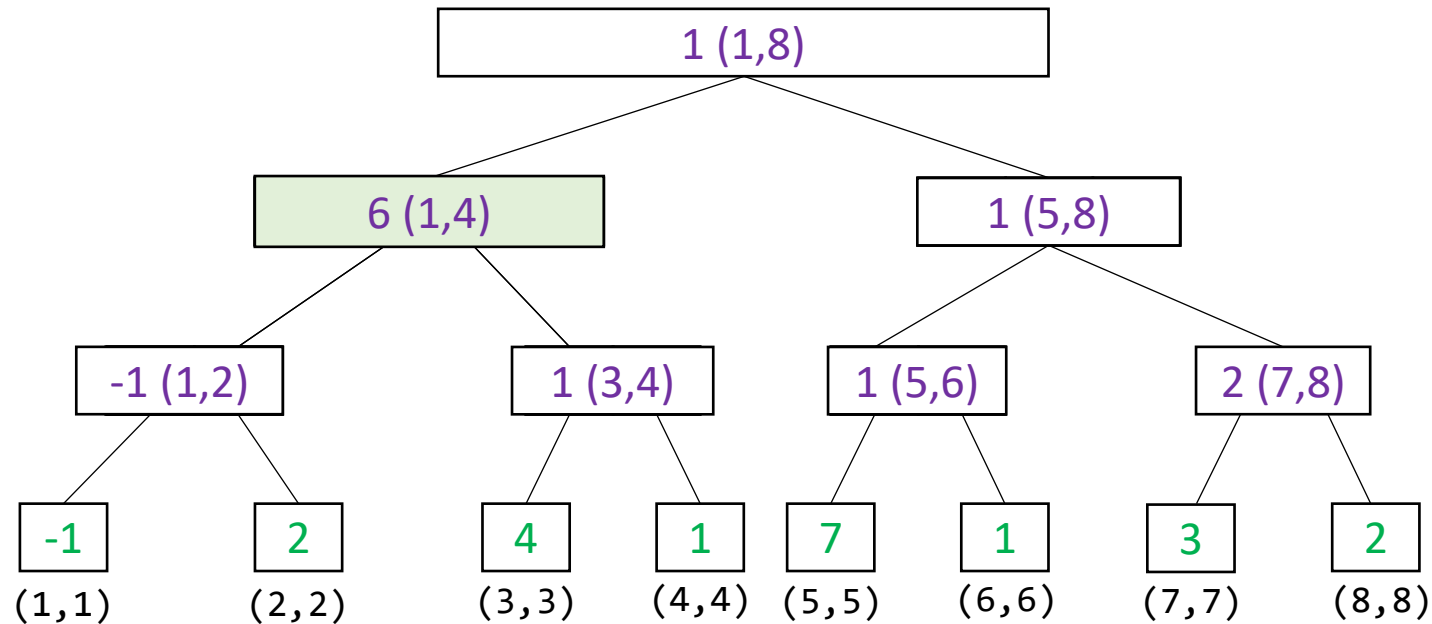
Now, Increment [1,4] by 2

Segment Tree

1	4	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	5	5	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [1,4] by 2

Same process, just [1,4] will be updated.

Corresponding children updated in lazy tree.

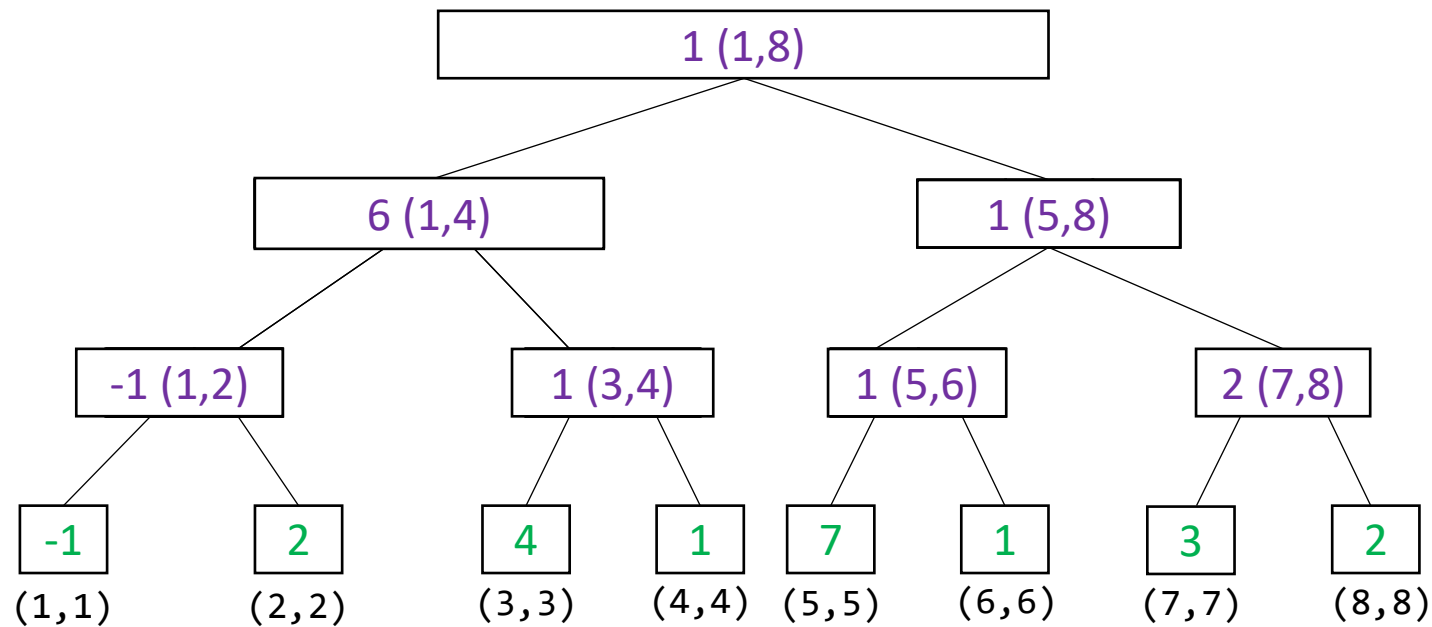
No change in root as $\max(6,1)=1$

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	7	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



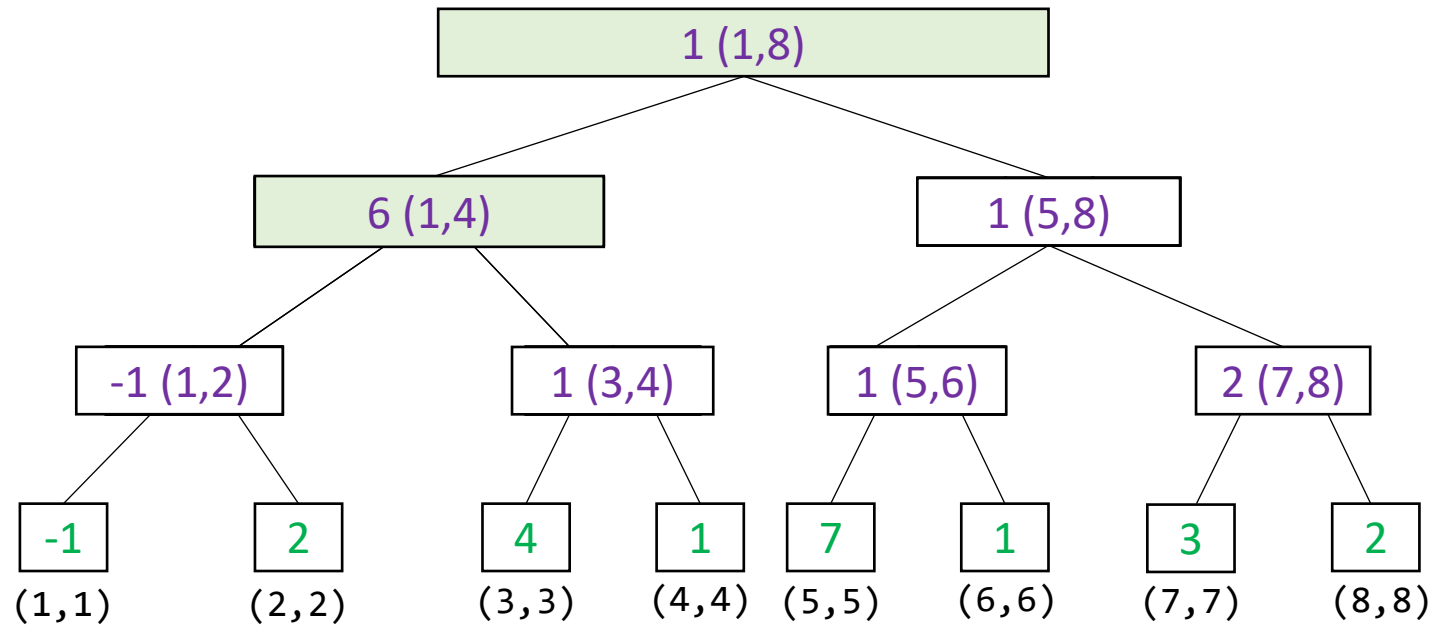
Now, Increment [2,2] by 3

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	7	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

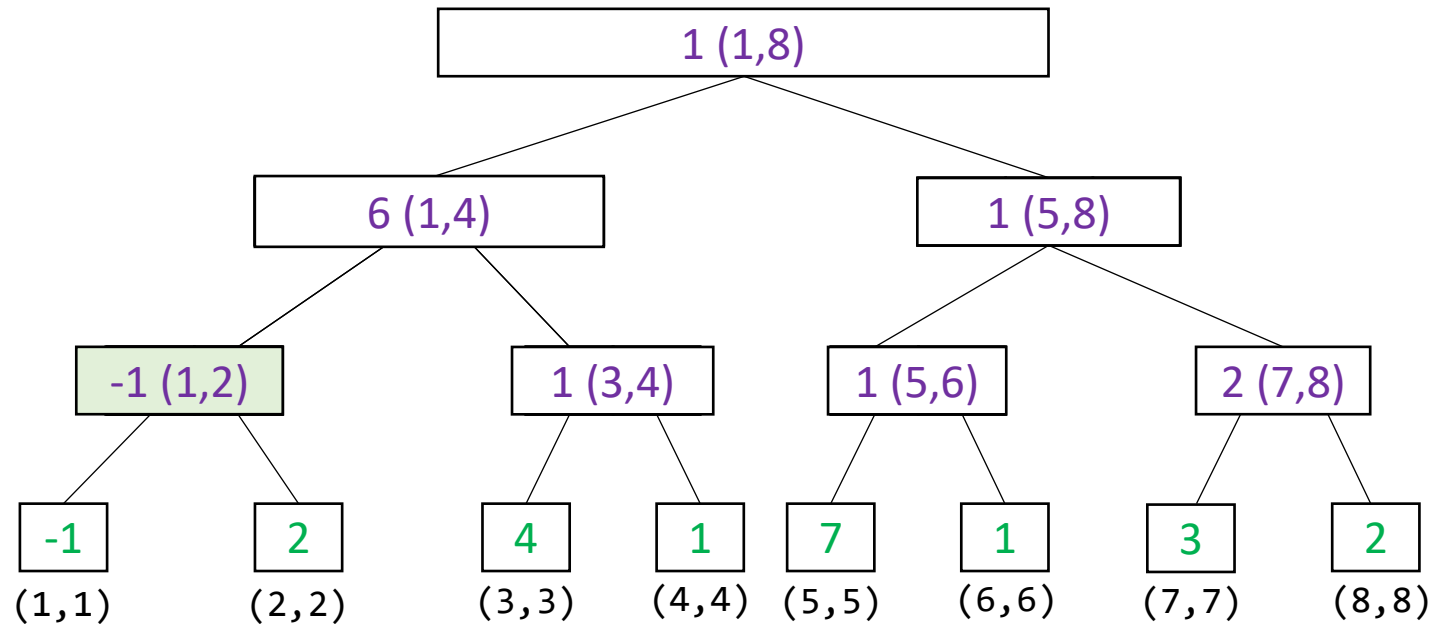
[1,8] – node is updated, partial overlap
[1,4] – node is updated, partial overlap

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	7	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

[1,8] – node is updated, partial overlap

[1,4] – node is updated, partial overlap

[1,2] – pending task in corresponding location (idx-4) of lazy tree.

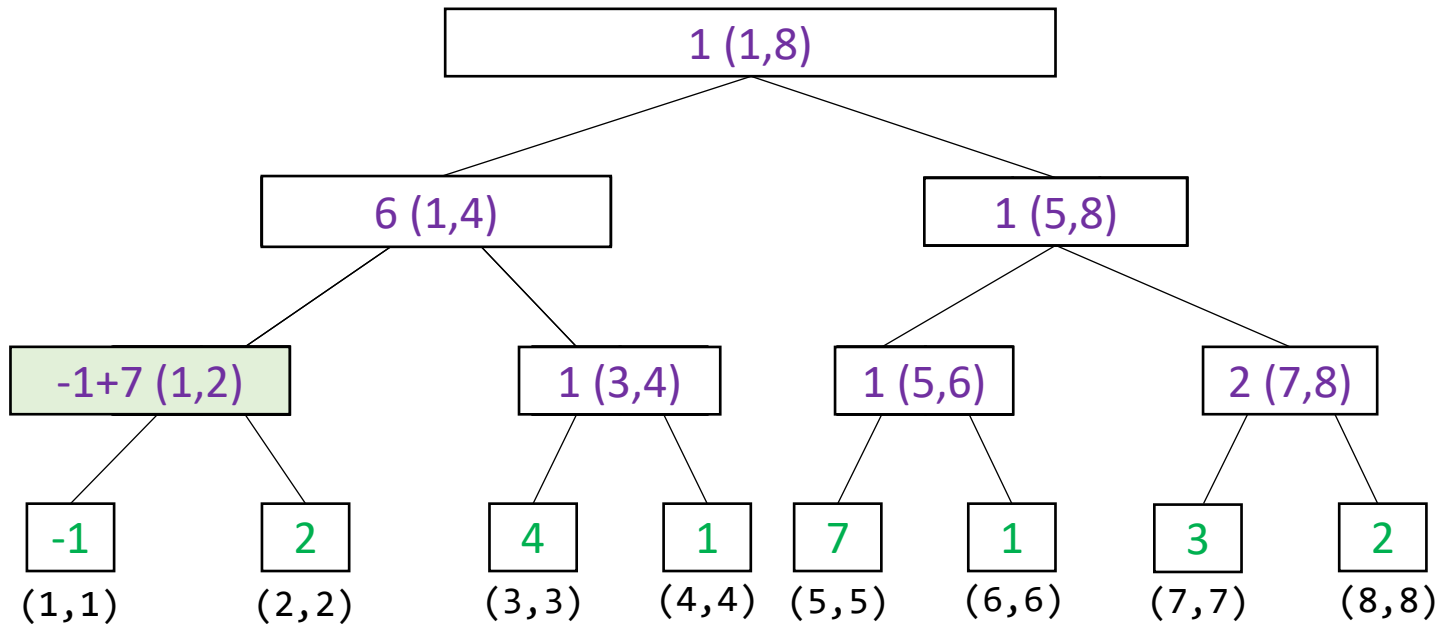
Complete it before checking overlap!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	7	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

[1,8] – node is updated, partial overlap

[1,4] – node is updated, partial overlap

[1,2] – pending task in corresponding location (idx-4) of lazy tree.

Complete it before checking overlap!

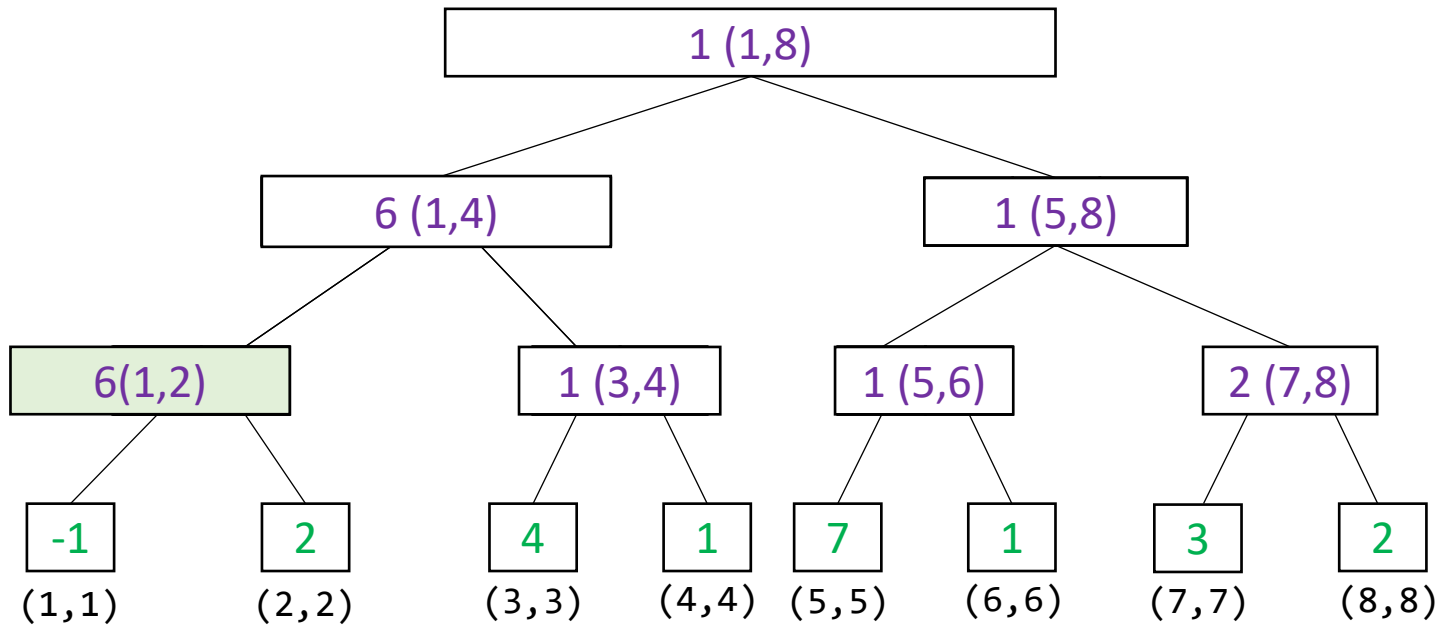
After adding 7, we pass this info to the corresponding children of the lazy tree.

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	7	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

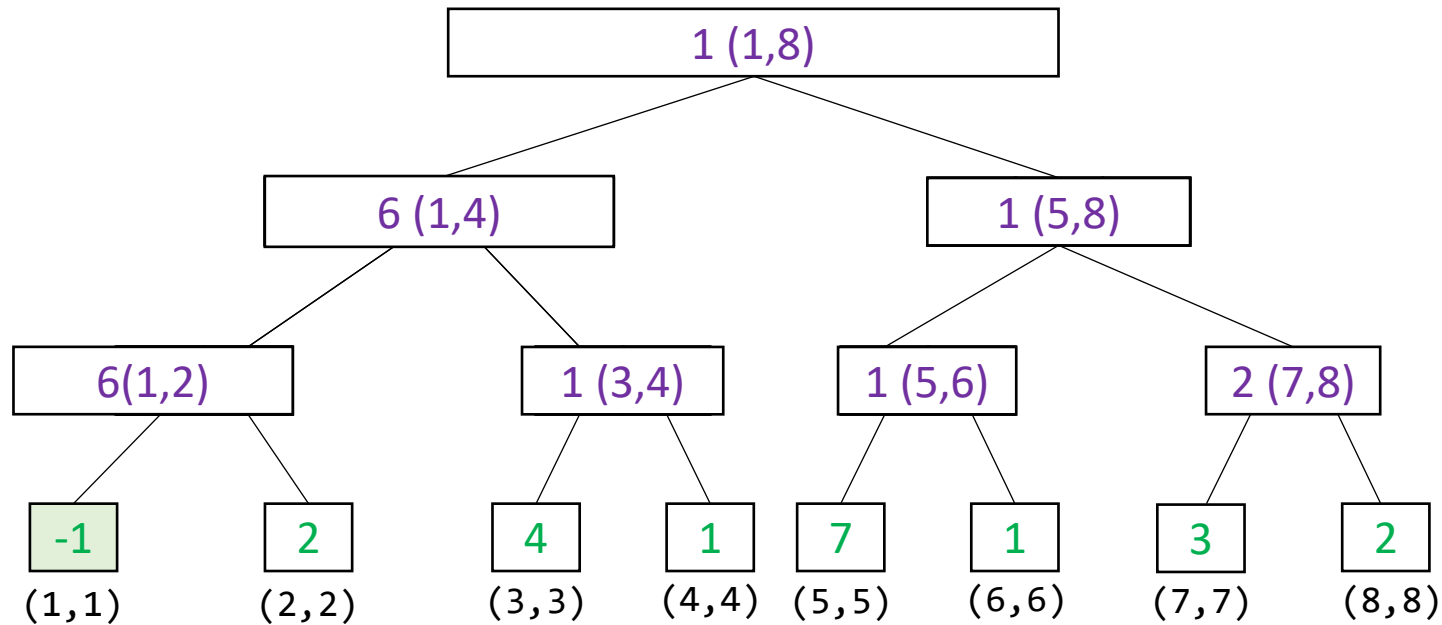
[1,2] – node updated, partial overlap

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	7	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

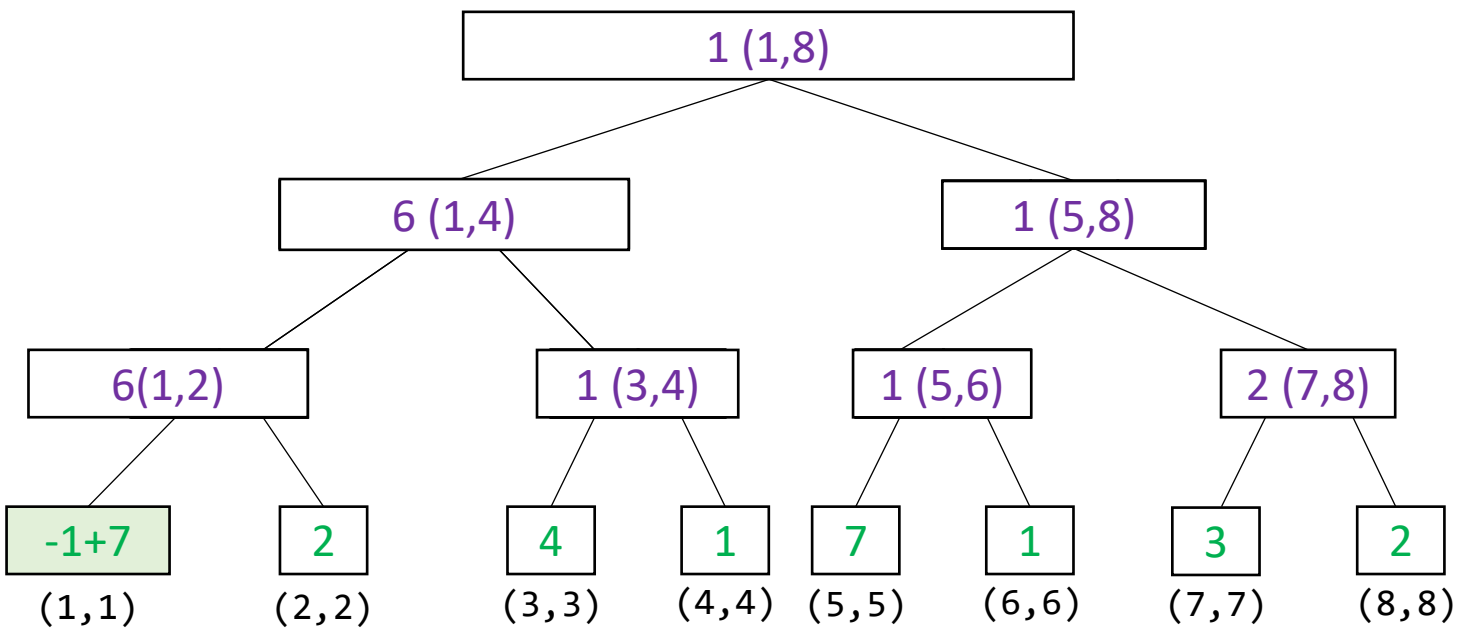
[1,1] – need to update the node

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	7	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

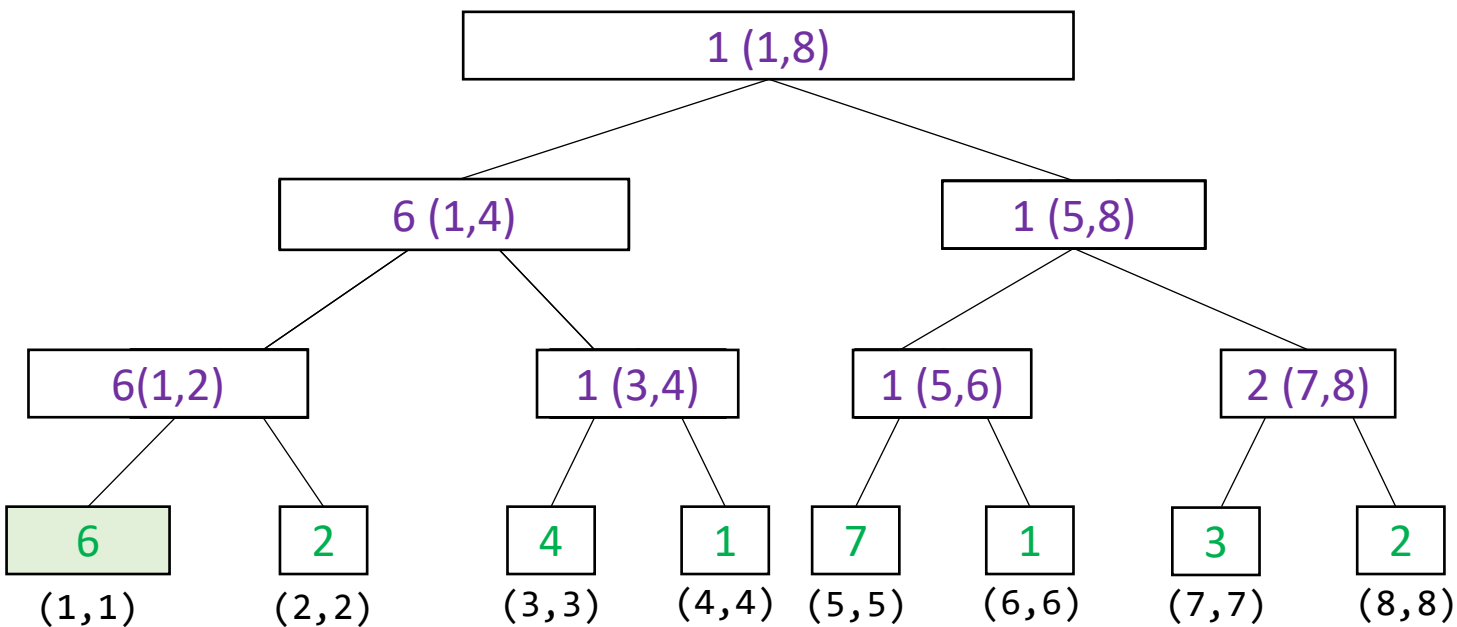
[1,1] – node updated. No room for passing this info forward (index out of bound)

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

[1,1] – node updated. No room for passing this info forward (index out of bound)

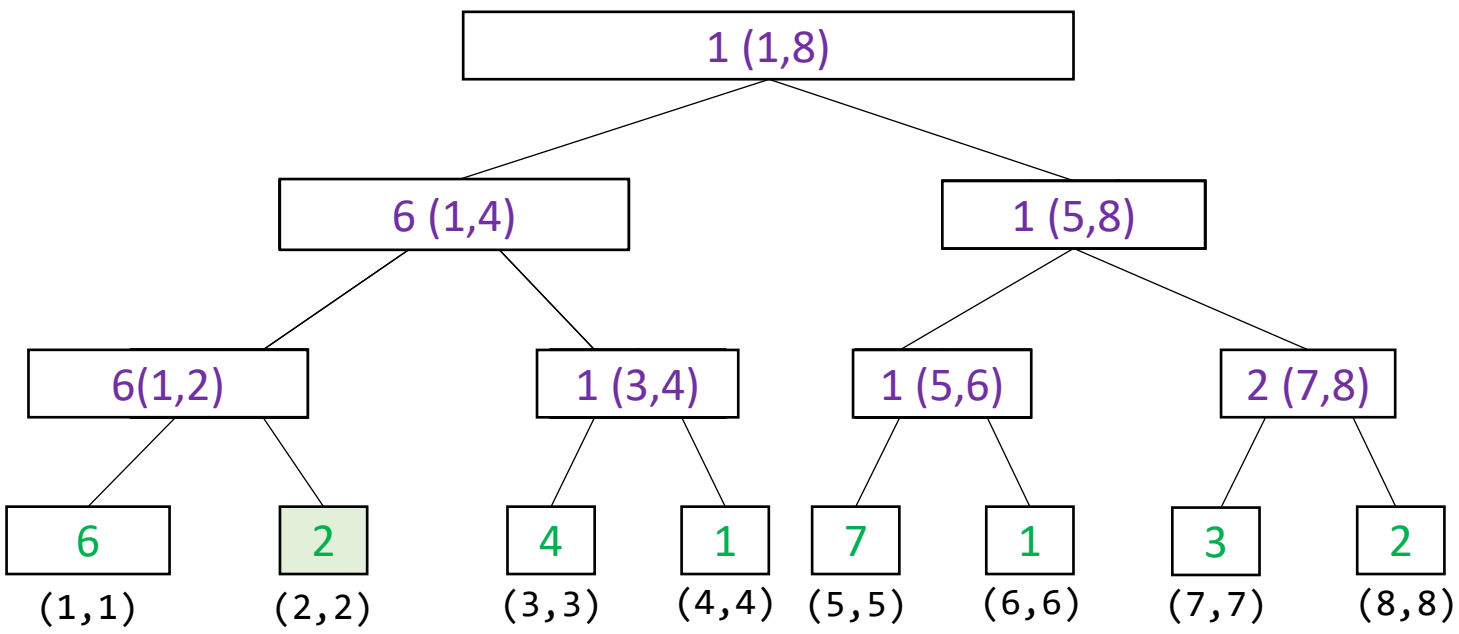
[1,1] – no overlap. No increment.

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

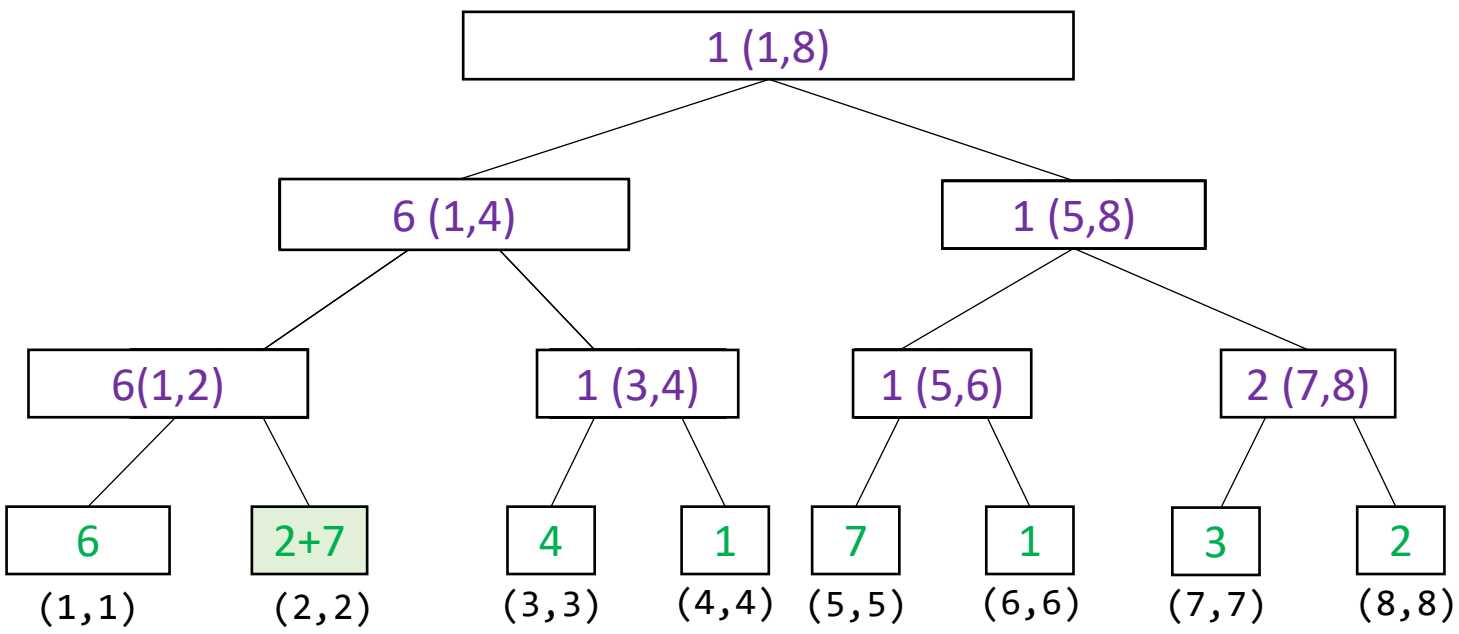
[2,2] – update this node before any further step!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	7	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

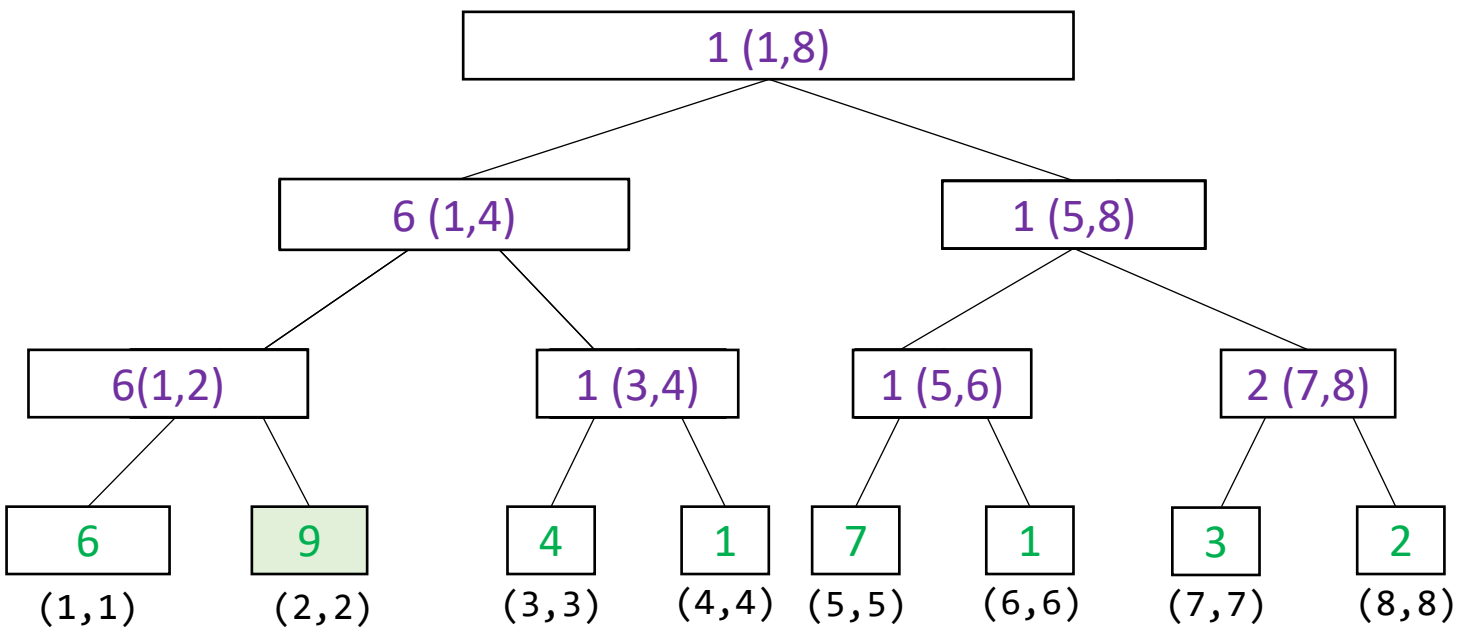
[2,2] – update this node before any further step!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

[2,2] – Node updated.

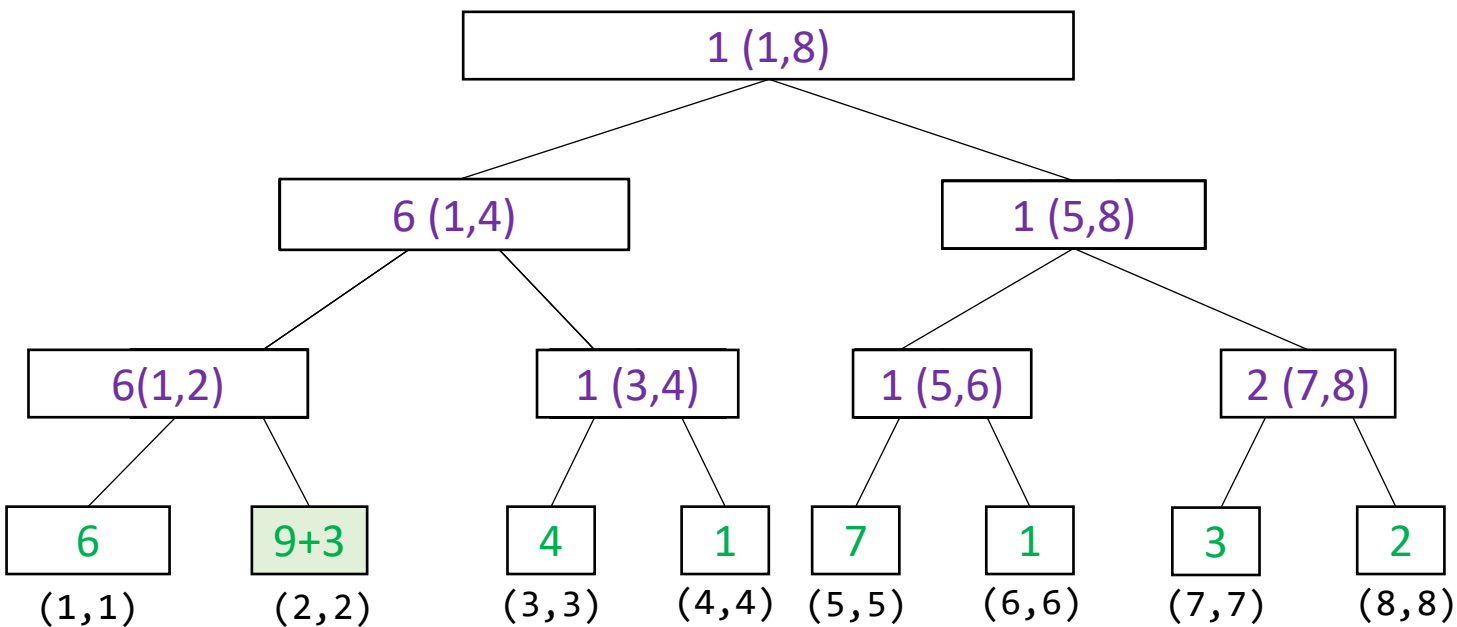
Full overlap. Now, we can increment the value as per our task!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

[2,2] – Node updated.

Full overlap. Now, we can increment the value as per our task!

Add [2,2] by 3.

Nothing to propagate in Segment tree, as this is a leaf node.

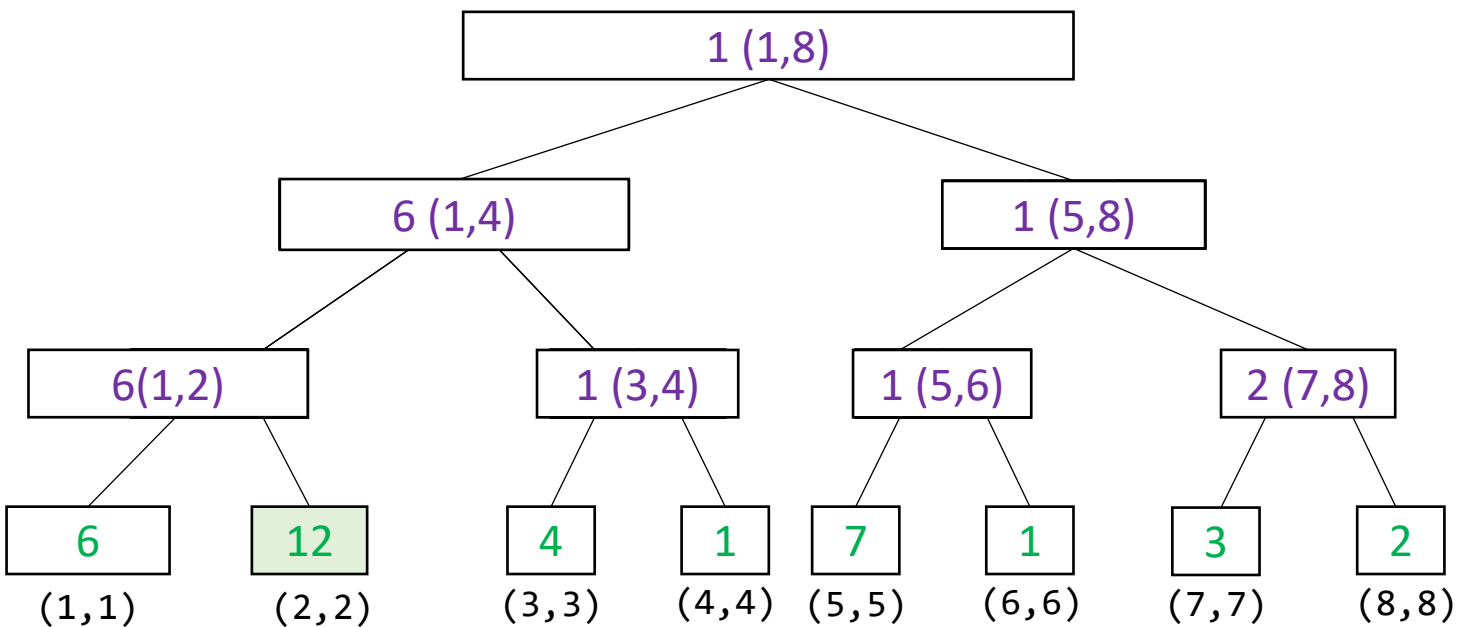
Else, we would note this change in the corresponding child location of lazy tree.

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, Increment [2,2] by 3

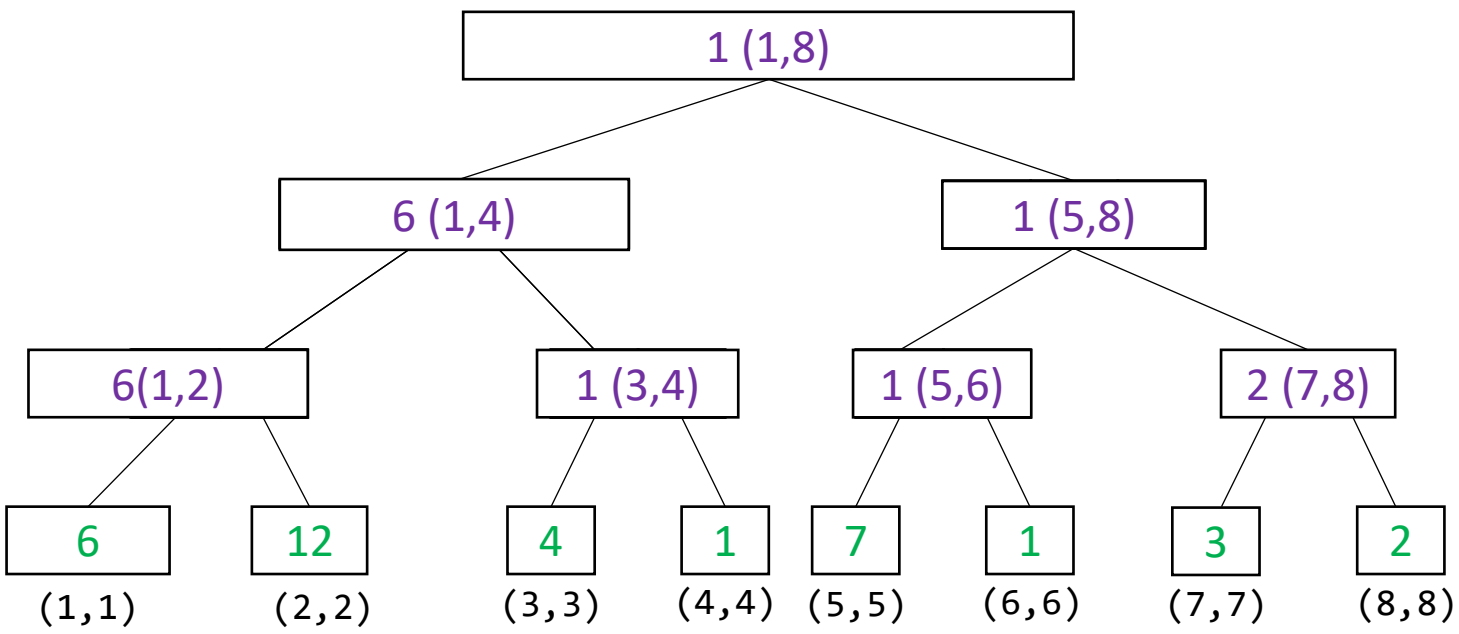
Finally, the update function recursively tracks back to the root. No change in min value!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



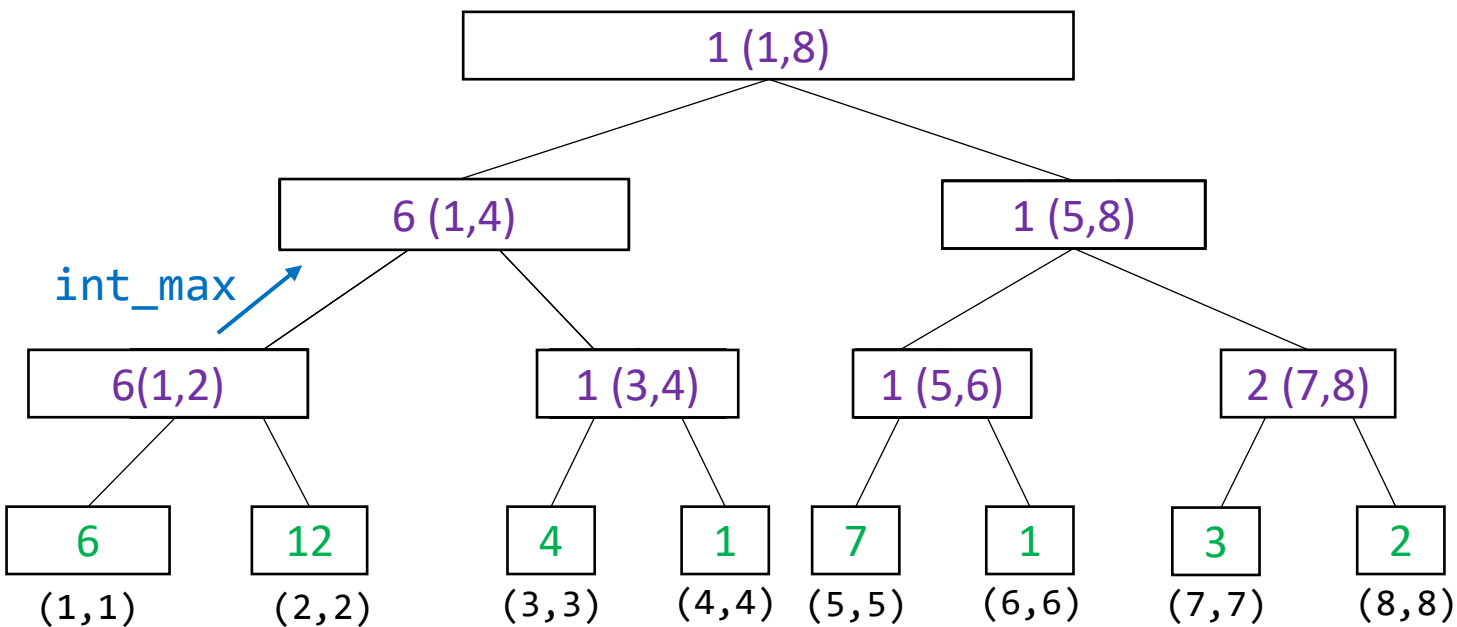
Now, return min of [4,6]

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

Root [1,8] – node updated, partial overlap
[1,4] – updated, partial overlap

[1,2] – updated, No overlap,
return INT_MAX

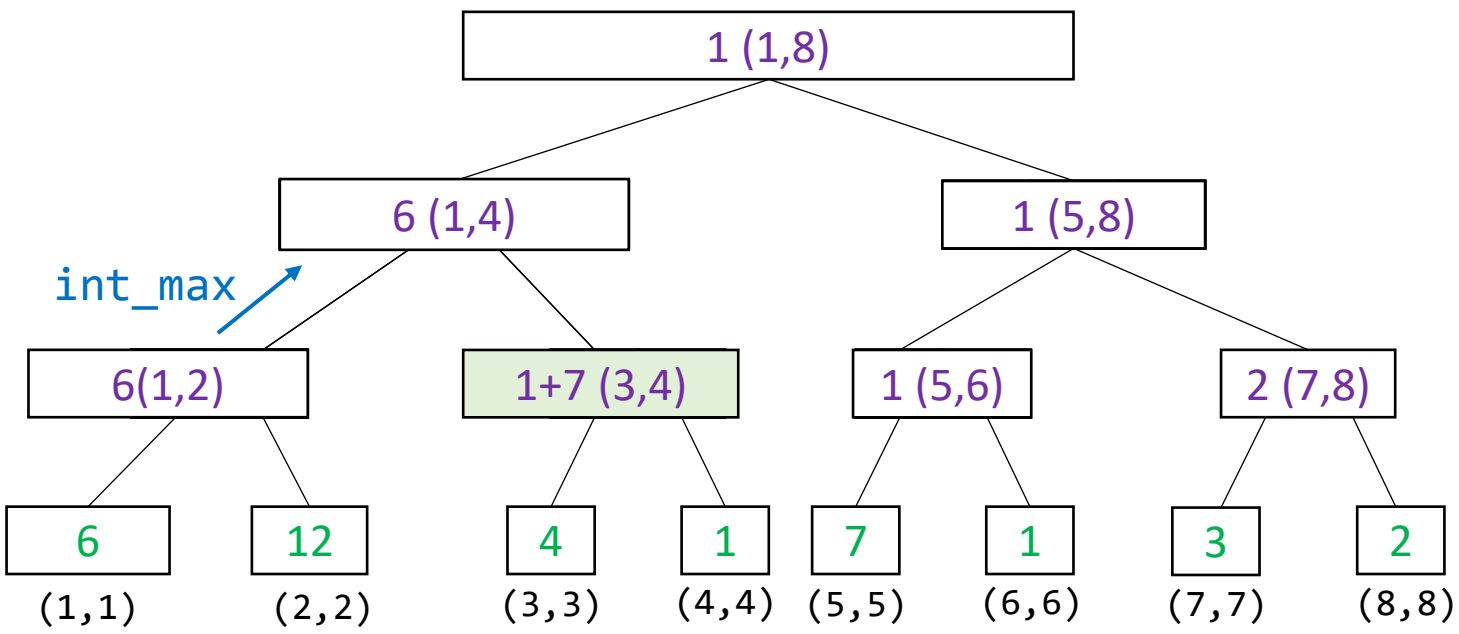
[3,4] – not updated!

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	7	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[3,4] – Node updated. Lazy tree value propagated to children.

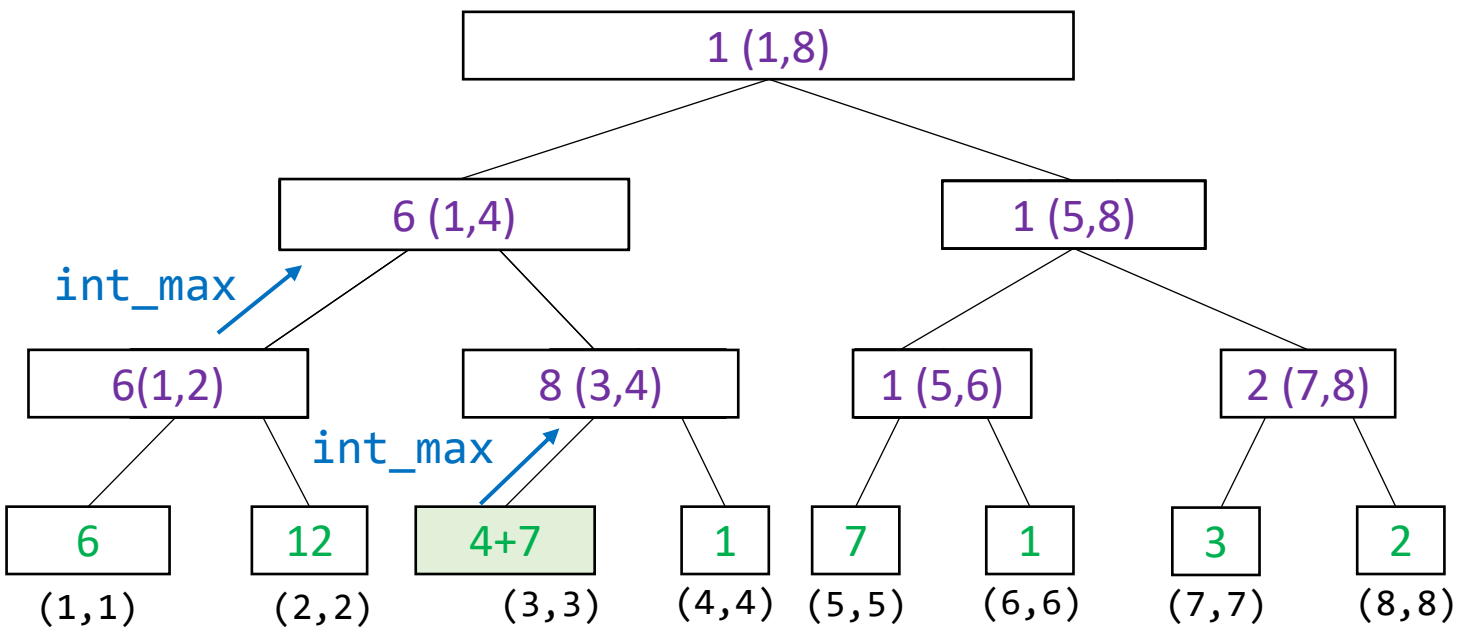
Partial overlap

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	7	7	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[3,3] – Update node

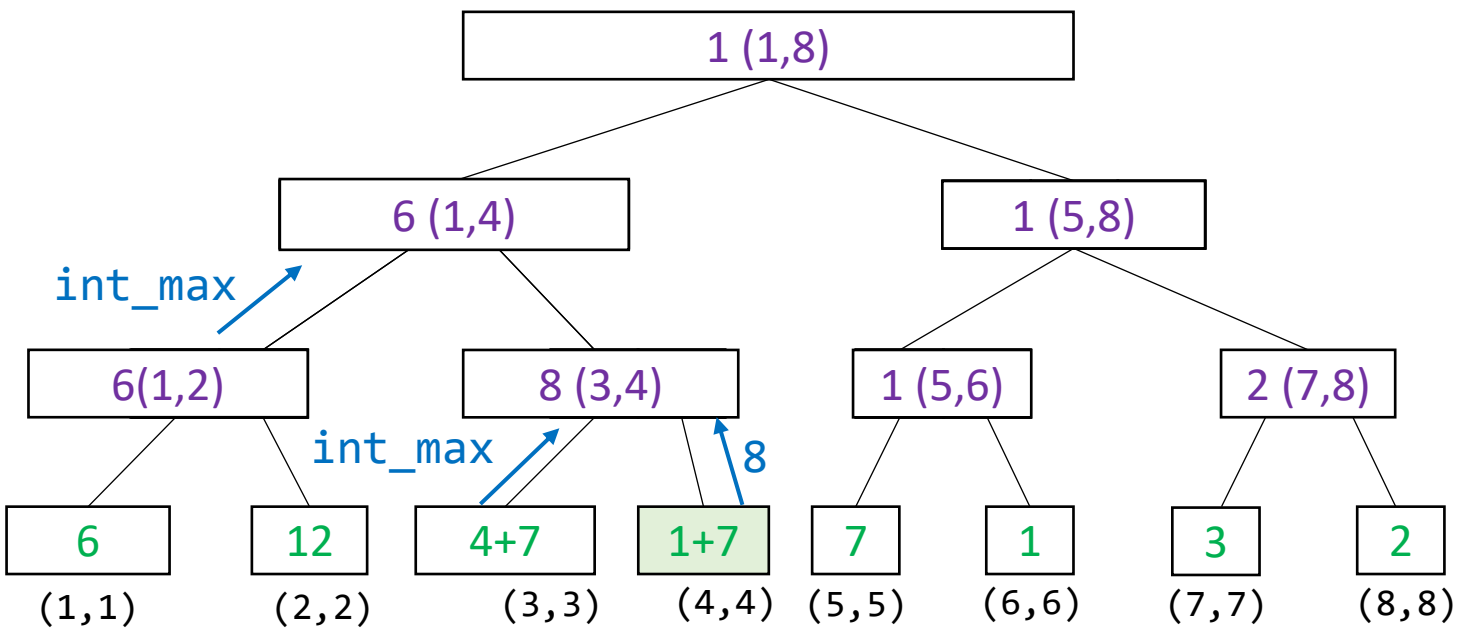
No overlap, return INT_MAX

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	7	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[4,4] – Update node

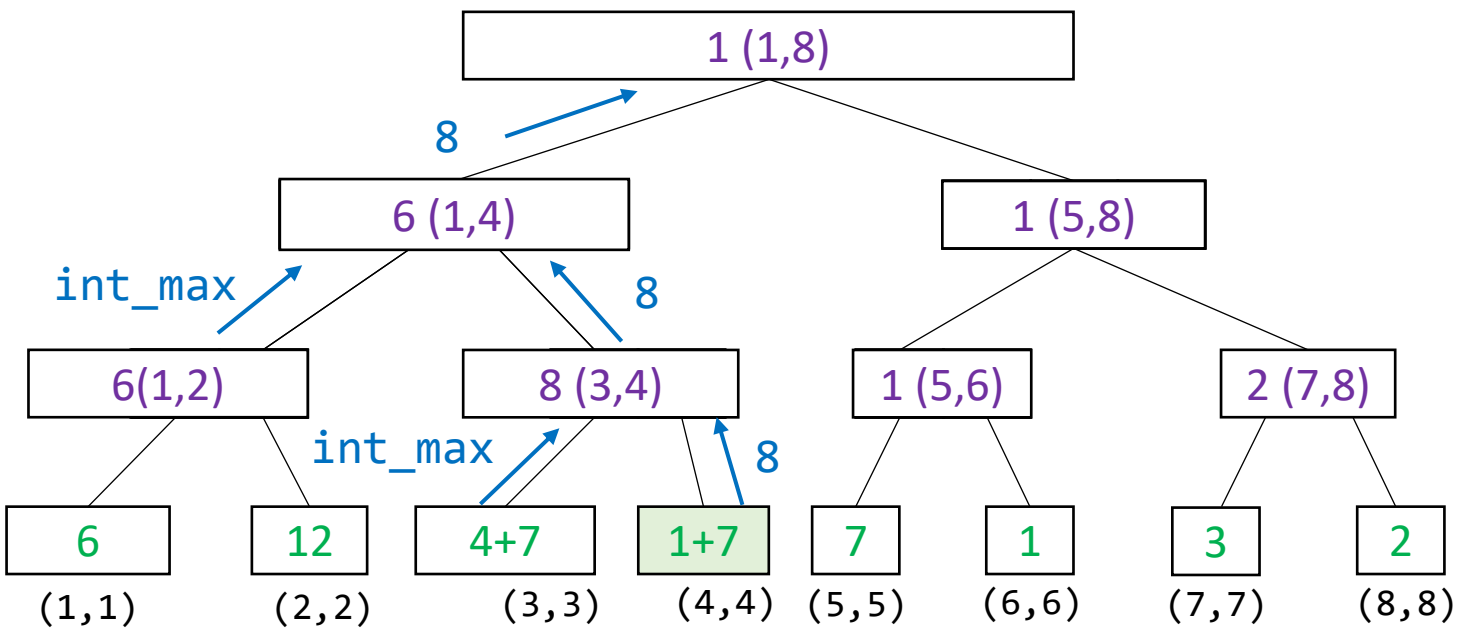
Full overlap, return node value

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[4,4] – Update node

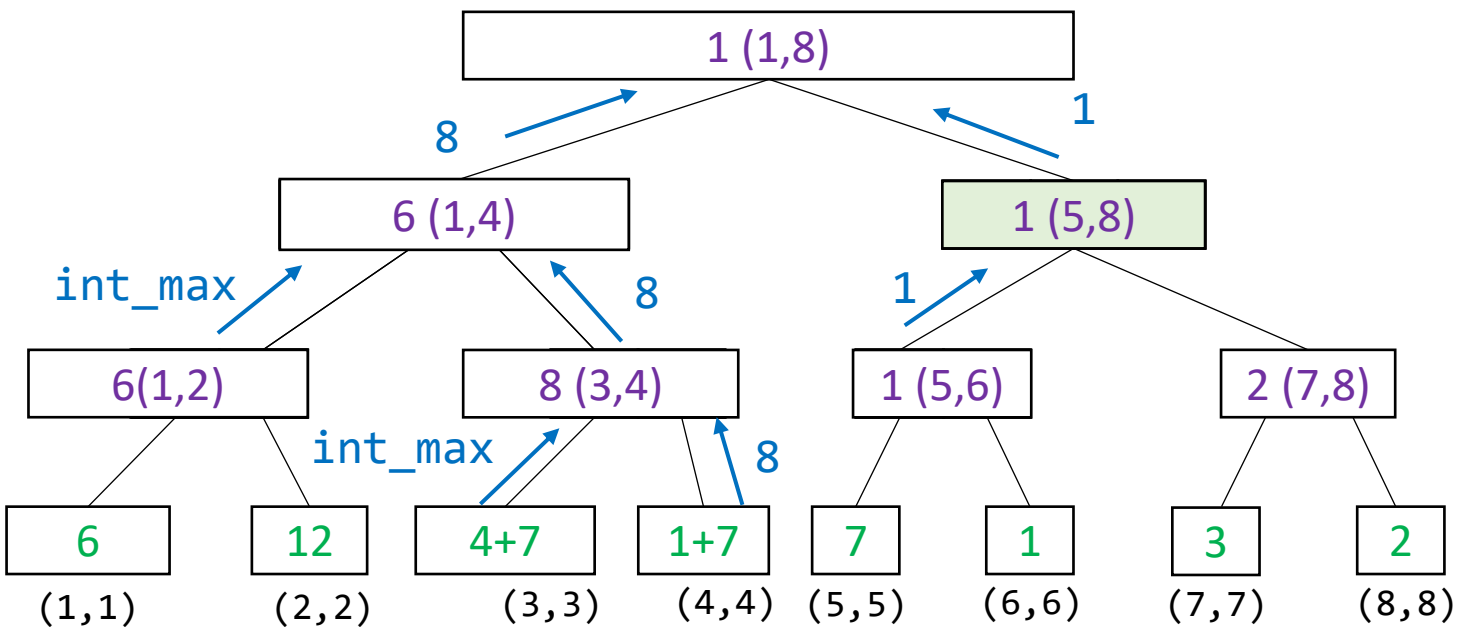
Full overlap, return node value

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[5,8] – Node updated. Partial overlap

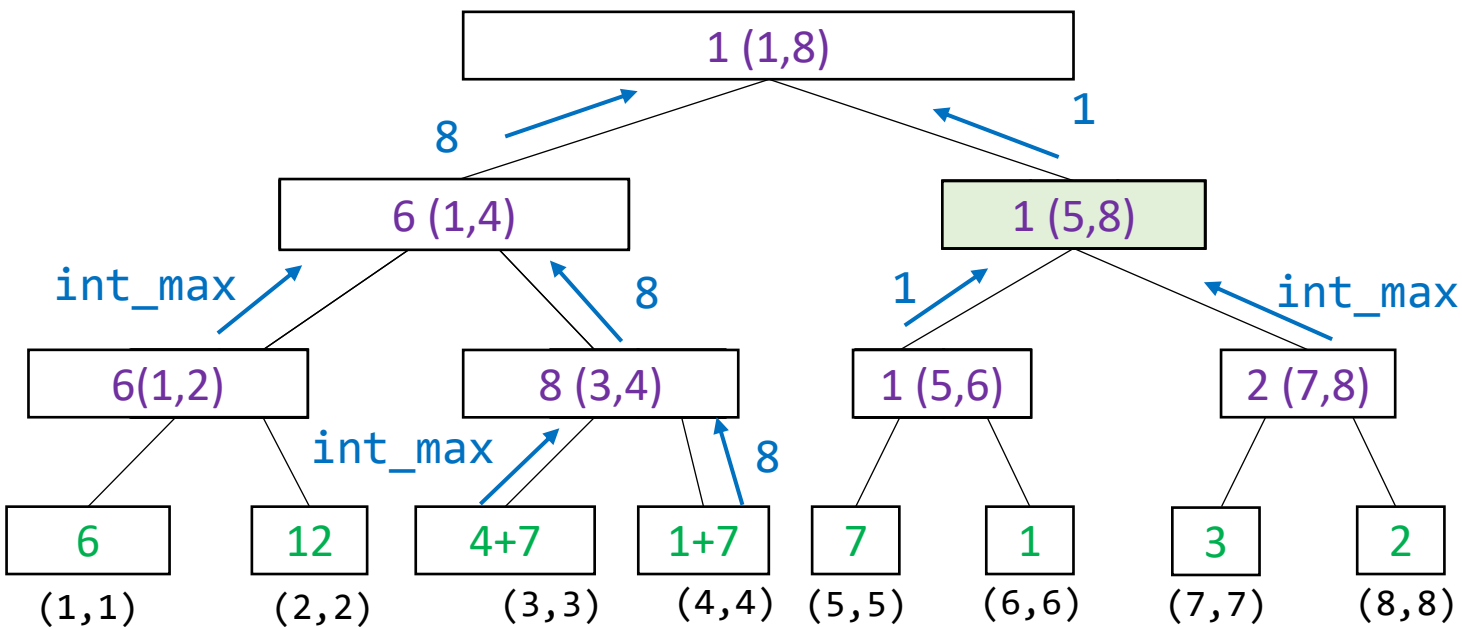
[5,6] – node updated, full overlap. Return value

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

[5,8] – Node updated. Partial overlap

[5,6] – node updated, full overlap. Return value

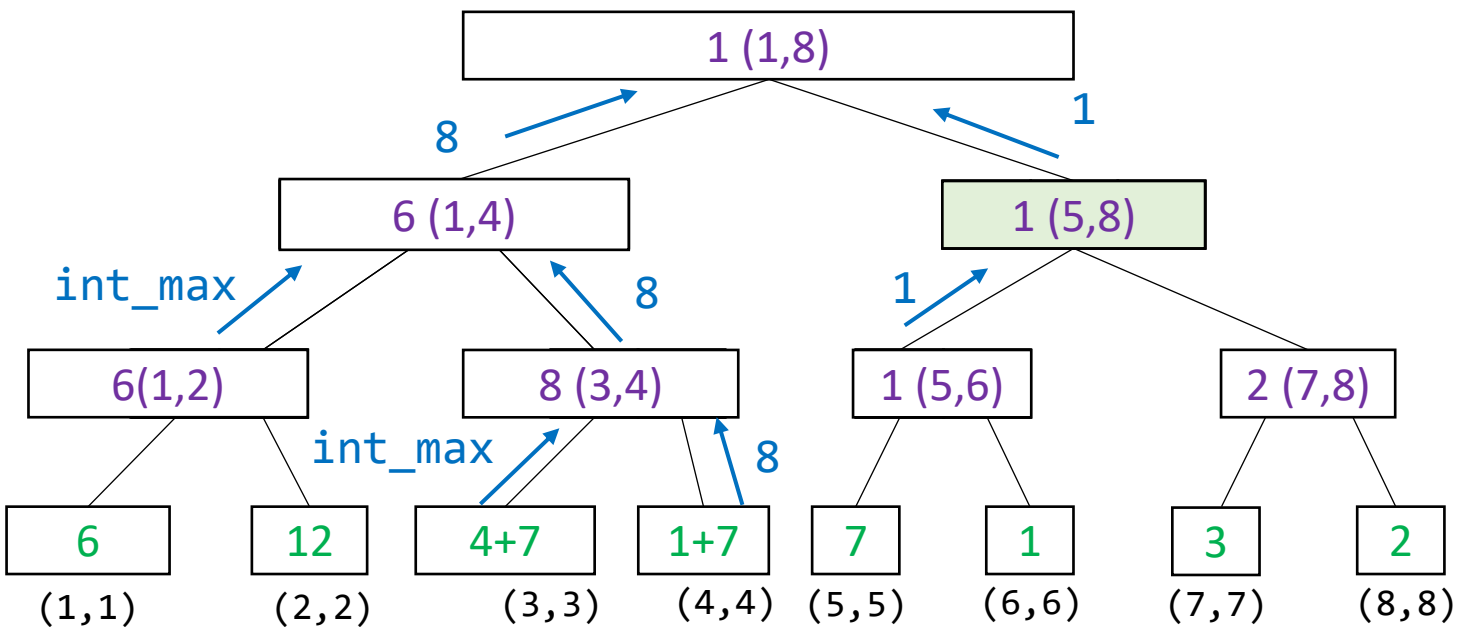
[7,8] – node updated. No overlap. Return INT_MAX

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Now, return min of [4,6]

Finally, result is 1.

Segment Tree

1	6	1	-1	1	1	2	-1	2	4	1	7	1	3	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Lazy Tree

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

```

1 int update (int node, int b, int e, int i, int newValue) {
2     if(i>e || i<b)
3         return                //out of range
4
5     if(b>=i && e<=i)
6         tree[node] += newValue //position is found
7         return
8
9     int left= node*2
10    int right= node*2 + 1
11    int mid= (b + e)/2
12
13    update(left, b, mid, i, newValue)
14    update(right, mid+1, e, i, newValue)
15
16    tree[node] = min(tree[left], tree[right])
17 }

```

Basic update algorithm

What changes are needed for lazy propagation?

```

1  int lazy_update (int node, int b, int e, int i, int newValue) {
2      if( lazy[node]!=0 )
3
4
5
6
7
8
9
10
11
12
13
14     int left= node*2
15     int right= node*2 + 1
16     int mid= (b + e)/2
17     lazy_update(left, b, mid, i, newValue)
18     lazy_update(right, mid+1, e, i, newValue)
19     tree[node] = min(tree[left], tree[right])
20 }

```

```

1  int lazy_update (int node, int b, int e, int i, int newValue) {
2      if( lazy[node]!=0 )
3          segment[node] += lazy[node]
4
5
6
7
8
9
10
11
12
13
14  int left= node*2
15  int right= node*2 + 1
16  int mid= (b + e)/2
17  lazy_update(left, b, mid, i, newValue)
18  lazy_update(right, mid+1, e, i, newValue)
19  tree[node] = min(tree[left], tree[right])
20  }

```

```

1  int lazy_update (int node, int b, int e, int i, int newValue) {

2      if( lazy[node]!=0 )
3          segment[node] += lazy[node]
4          if(b!=e)
5              lazy[node*2] += lazy[node]
6              lazy[node*2+1] += lazy[node]
7
8
9
10
11
12
13

14  int left= node*2
15  int right= node*2 + 1
16  int mid= (b + e)/2

17  lazy_update(left, b, mid, i, newValue)
18  lazy_update(right, mid+1, e, i, newValue)
19  tree[node] = min(tree[left], tree[right])
20  }

```

```

1  int lazy_update (int node, int b, int e, int i, int newValue) {

2      if( lazy[node]!=0 )
3          segment[node] += lazy[node]
4          if(b!=e)
5              lazy[node*2] += lazy[node]
6              lazy[node*2+1] += lazy[node]
7          lazy[node] = 0

8
9
10
11
12
13

14  int left= node*2
15  int right= node*2 + 1
16  int mid= (b + e)/2

17  lazy_update(left, b, mid, i, newValue)
18  lazy_update(right, mid+1, e, i, newValue)
19  tree[node] = min(tree[left], tree[right])
20  }

```



```

1  int lazy_update (int node, int b, int e, int i, int newValue) {

2      if( lazy[node]!=0 )
3          segment[node] += lazy[node]
4          if(b!=e)
5              lazy[node*2] += lazy[node]
6              lazy[node*2+1] += lazy[node]
7          lazy[node] = 0

8      if(i>e || i<b)      return          //out of range
9      if(b>=i && e<=i)
10         tree[node] += newValue //position is found
11
12
13

14     int left= node*2
15     int right= node*2 + 1
16     int mid= (b + e)/2

17     lazy_update(left, b, mid, i, newValue)
18     lazy_update(right, mid+1, e, i, newValue)
19     tree[node] = min(tree[left], tree[right])
20 }

```

```

1  int lazy_update (int node, int b, int e, int i, int newValue) {

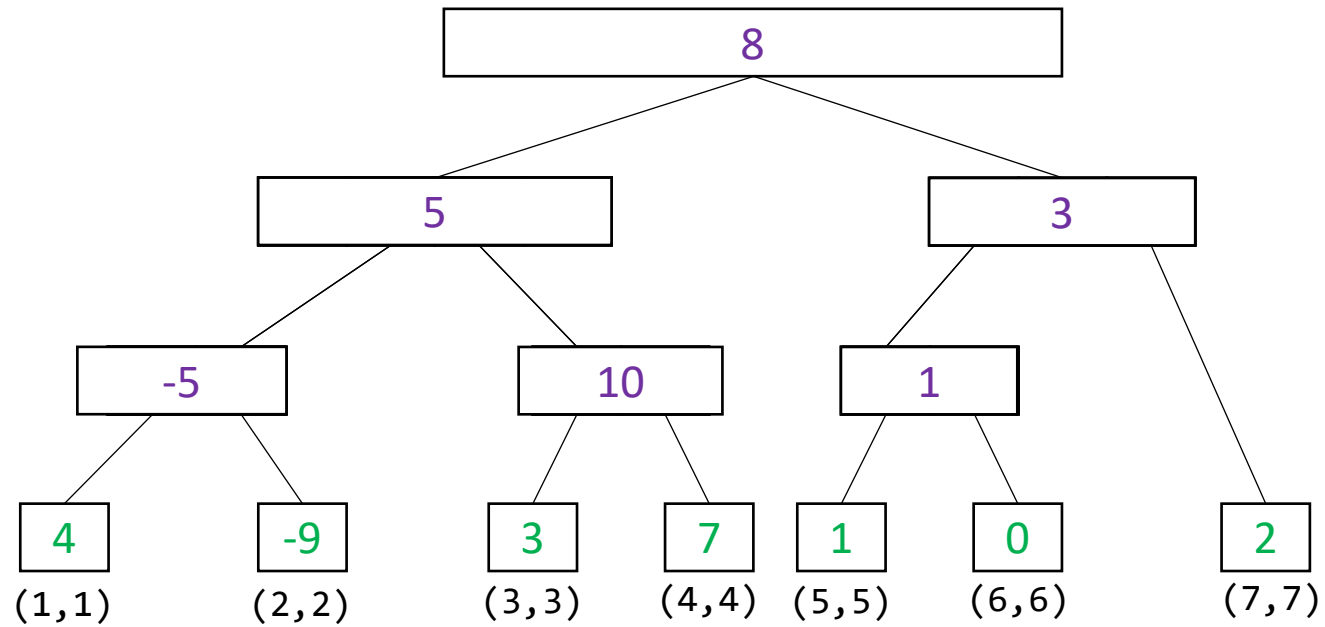
2      if( lazy[node]!=0 )
3          segment[node] += lazy[node]
4          if(b!=e)
5              lazy[node*2] += lazy[node]
6              lazy[node*2+1] += lazy[node]
7          lazy[node] = 0

8      if(i>e || i<b)      return          //out of range
9      if(b>=i && e<=i)
10         tree[node] += newValue //position is found
11         if(b!=e)
12             lazy[node*2] += lazy[node]
13             lazy[node*2+1] += lazy[node]

14     int left= node*2
15     int right= node*2 + 1
16     int mid= (b + e)/2

17     lazy_update(left, b, mid, i, newValue)
18     lazy_update(right, mid+1, e, i, newValue)
19     tree[node] = min(tree[left], tree[right])
20 }

```



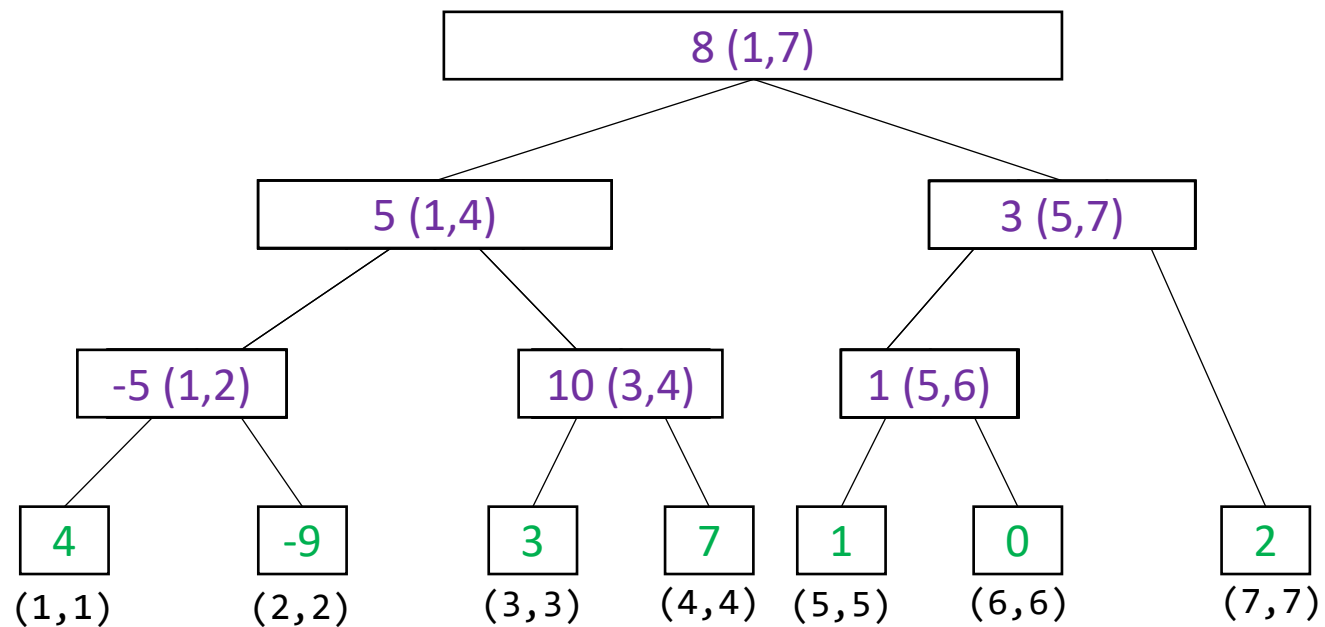
Segment tree for Summation

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13



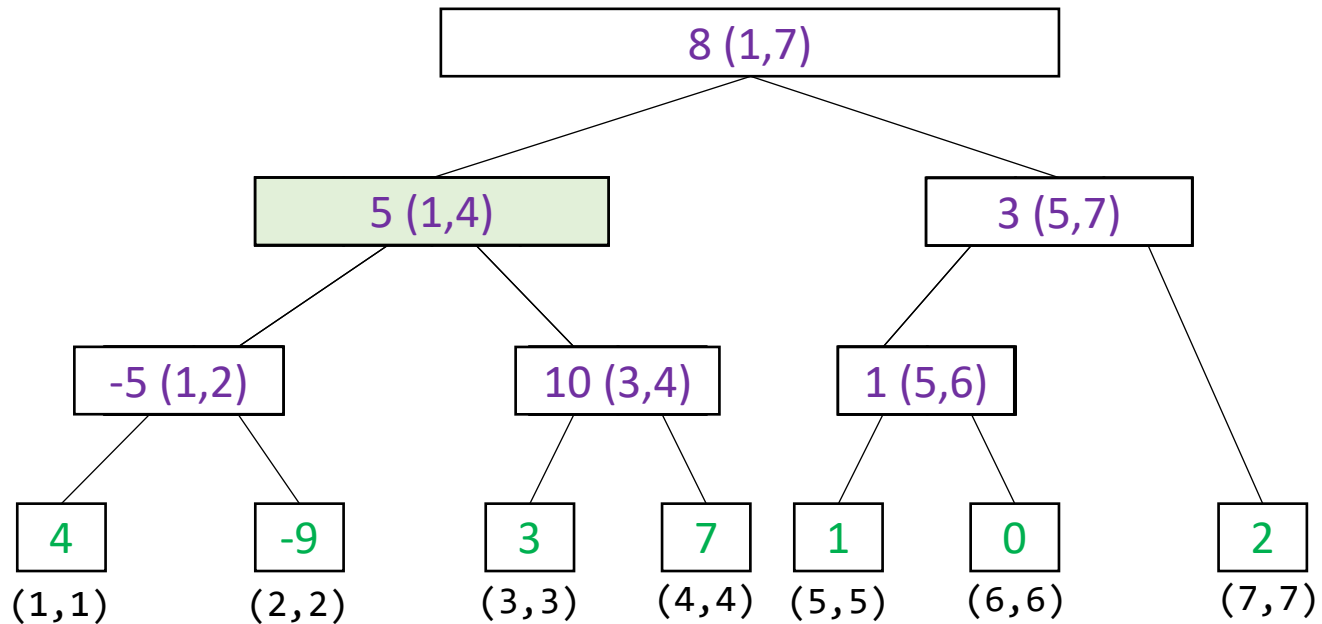
Task: Increment [1,6] by 2

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13



Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13

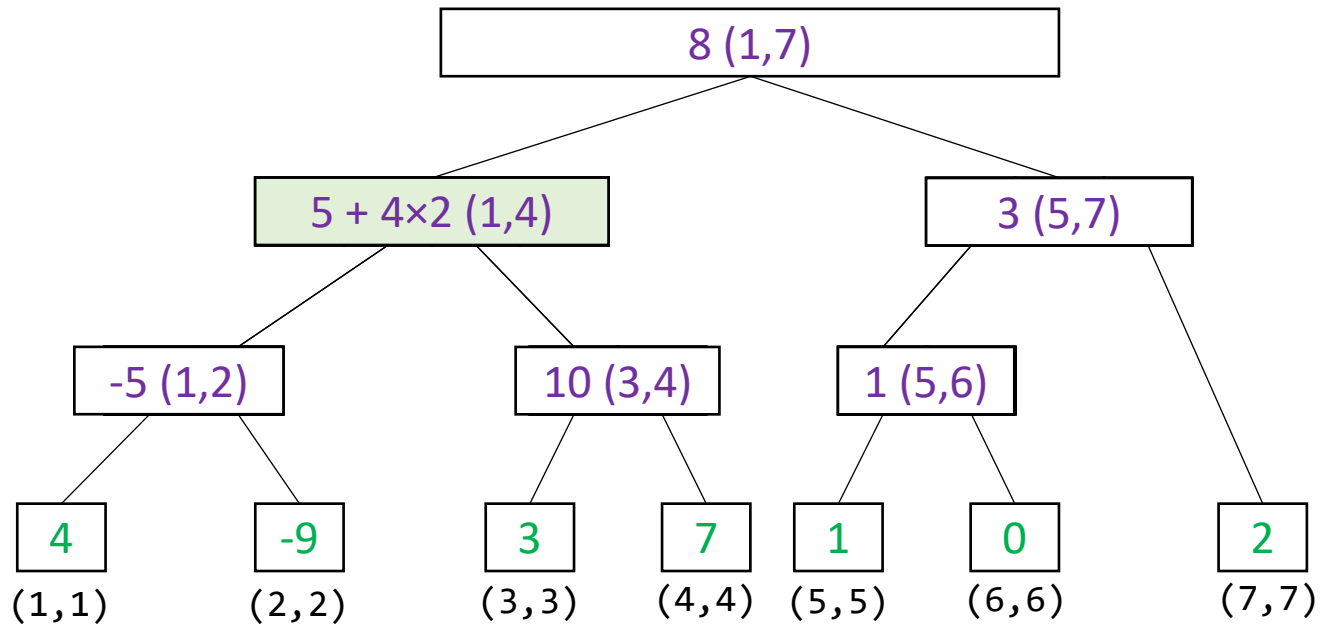
Task: Increment [1,6] by 2

[1,7] partial overlap

[1,4] full overlap.

Increment node, don't dig further.

Note pending task in the corresponding children of the lazy tree



Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13

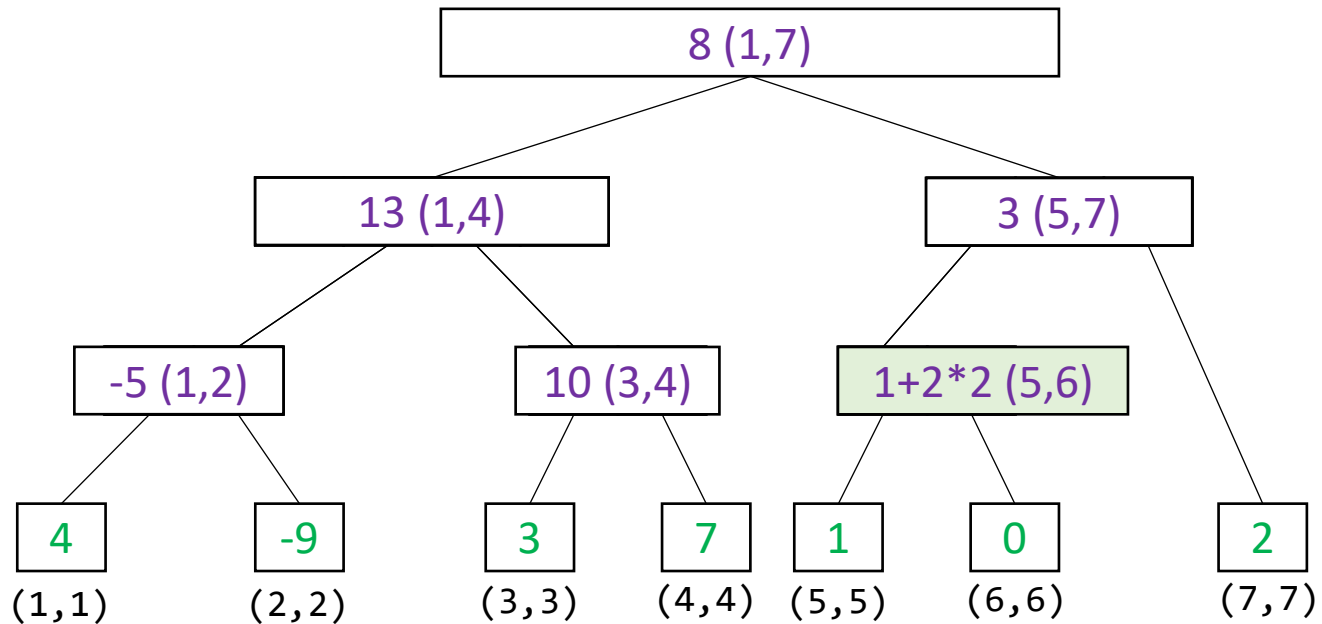
Task: Increment [1,6] by 2

[1,7] partial overlap

[1,4] full overlap.

Increment node, don't dig further.

Note pending task in the corresponding children of the lazy tree



Task: Increment [1,6] by 2

[5,7] partial overlap

[5,6] node updated. Full overlap

Update node.

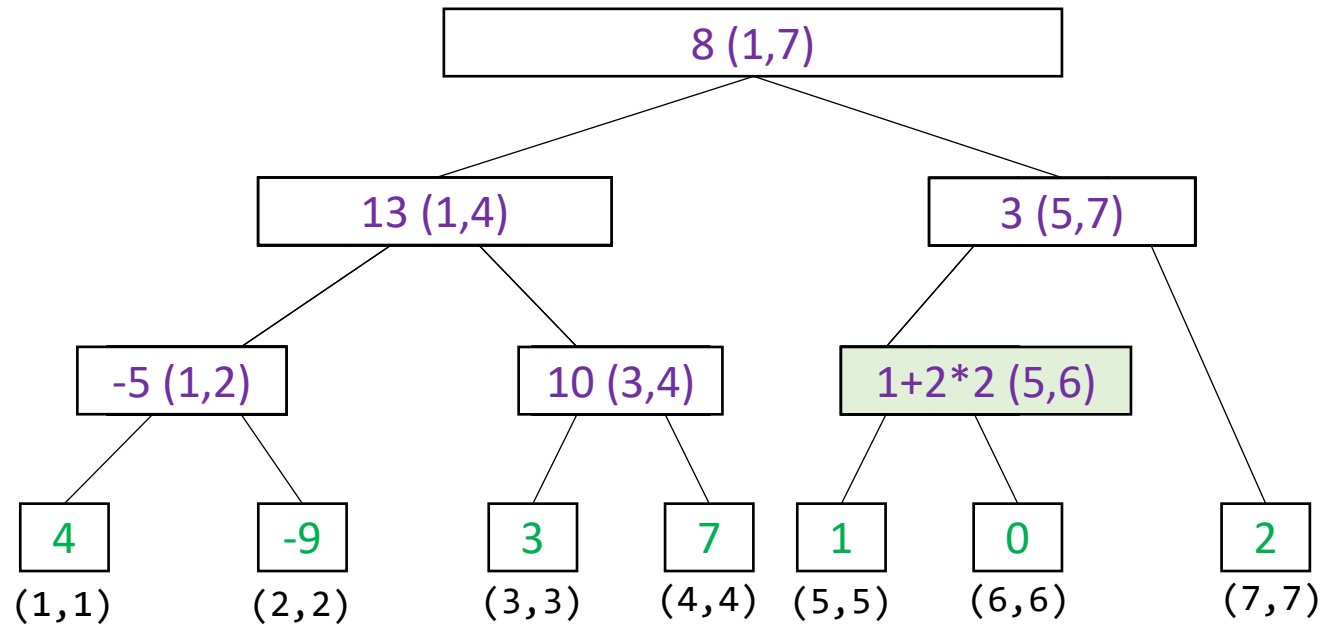
Update data in children location of Lazy tree

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: Increment [1,6] by 2

[7,7] no overlap. Return

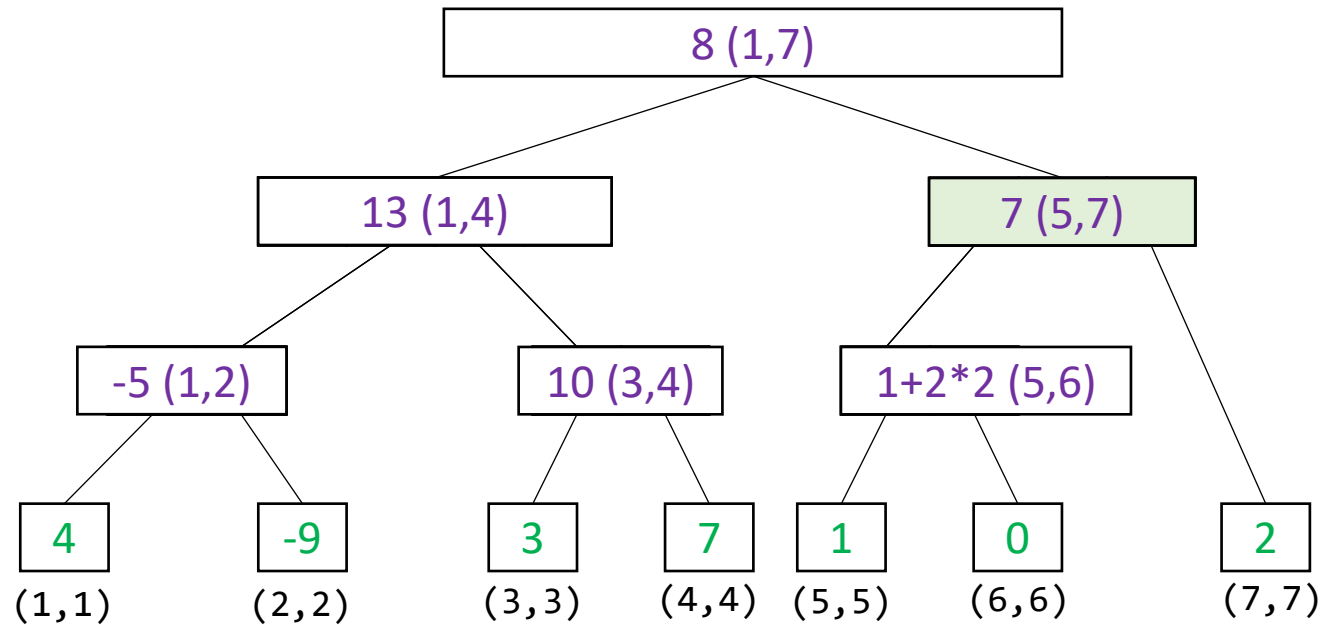
Backtrack and update nodes!

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: Increment [1,6] by 2

[7,7] no overlap. Return

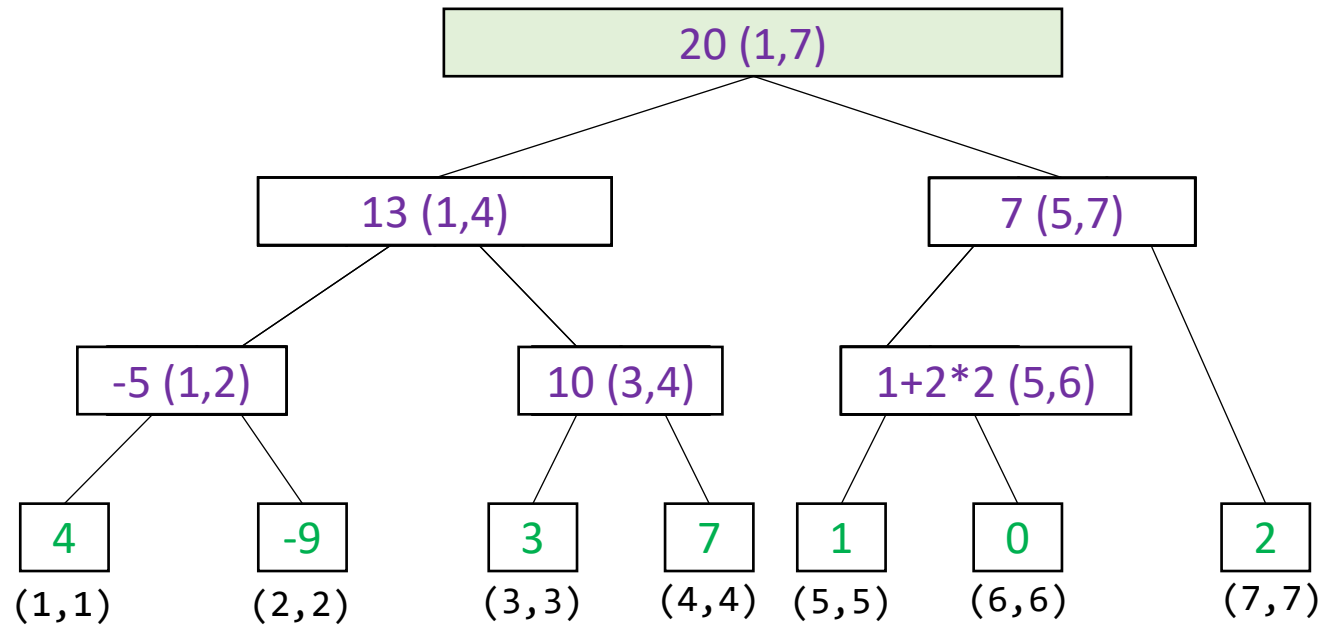
Backtrack and update nodes!

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: Increment [1,6] by 2

[7,7] no overlap. Return

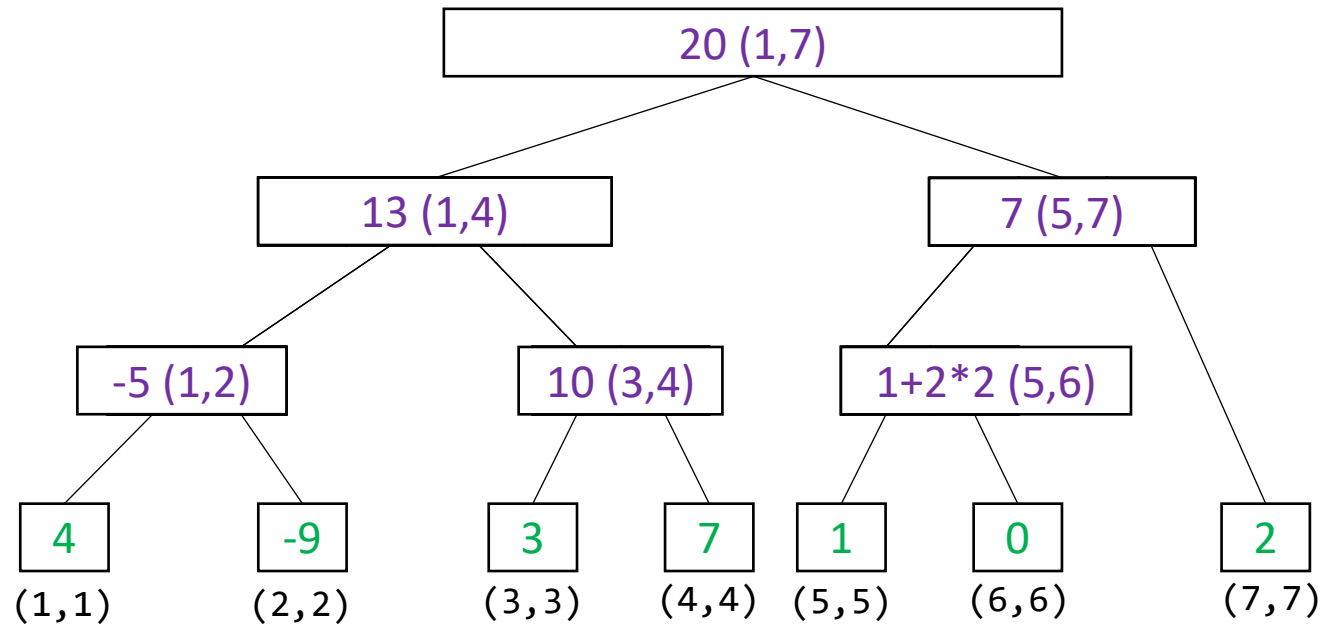
Backtrack and update nodes!

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



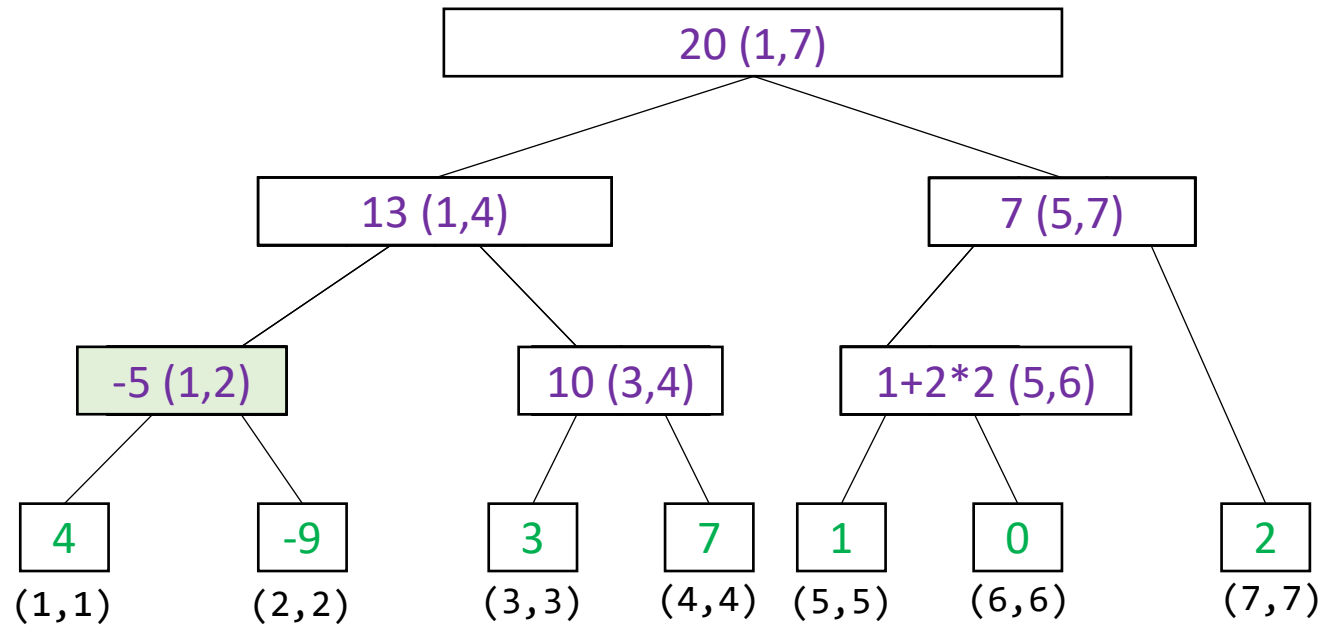
Task: return sum of range [3,4]

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[1,7]: updated, partial overlap

[1,4]: updated, partial overlap

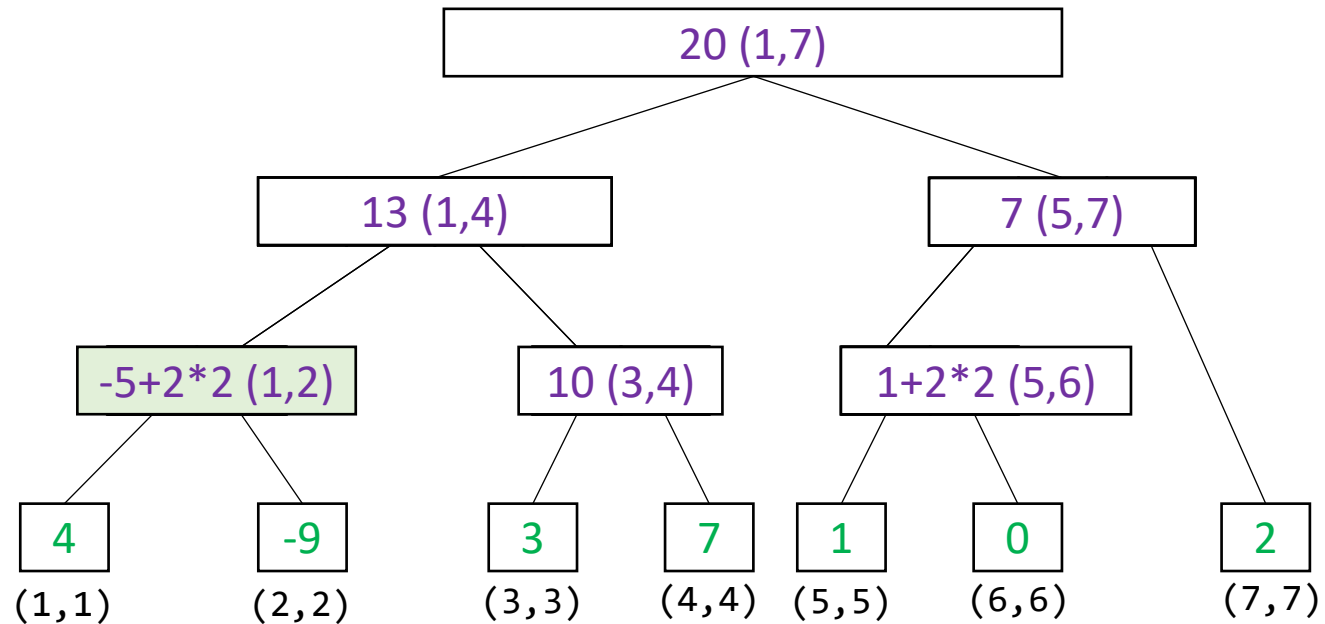
[1,2]: not updated. Fix it!

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	2	2	0	0	0	0	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[1,7]: updated, partial overlap

[1,4]: updated, partial overlap

[1,2]: not updated. Fix it!

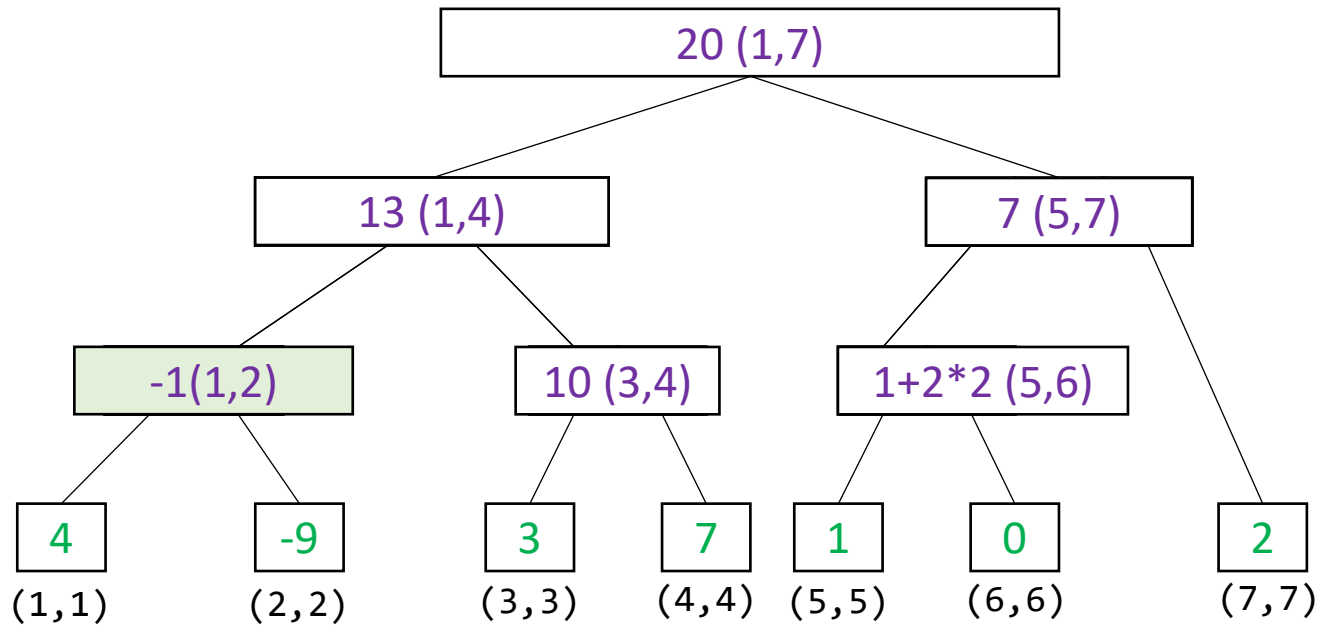
Node updated, lazy values passed to children

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	2	0	0	2	2	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[1,7]: updated, partial overlap

[1,4]: updated, partial overlap

[1,2]: not updated. Fix it!

Node updated, lazy values passed to children

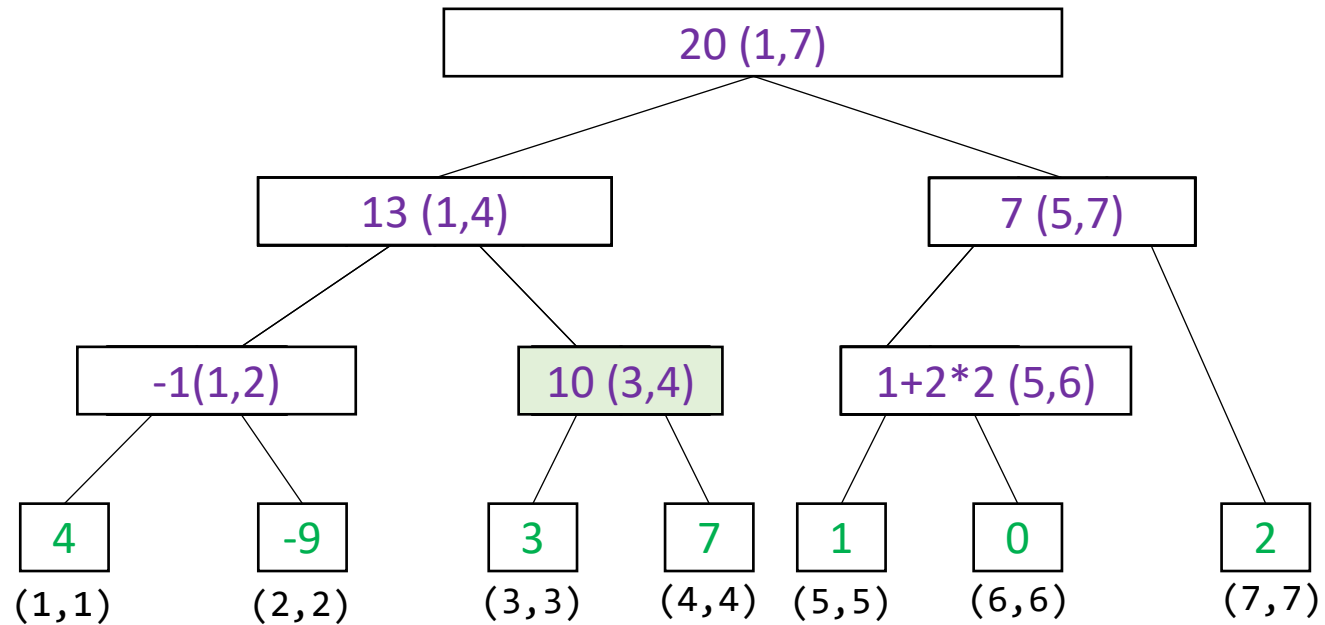
No overlap, skip

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	2	0	0	2	2	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

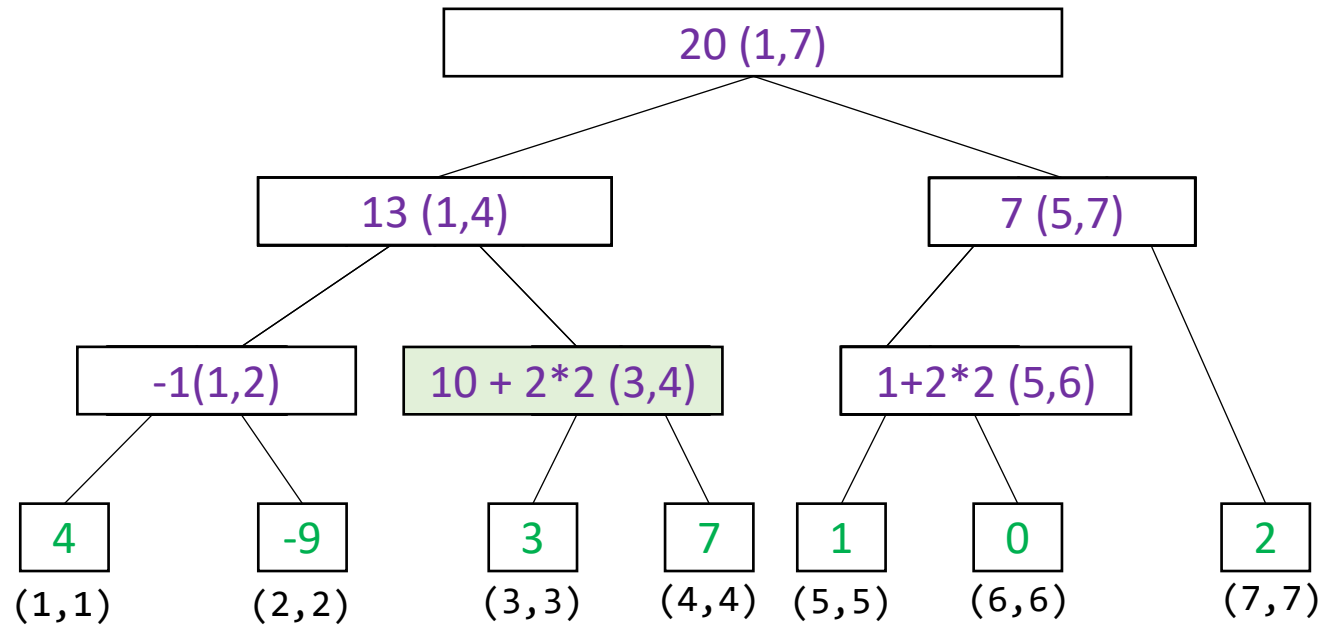
[3,4]: node needs to be updated.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	2	0	0	2	2	0	0	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[3,4]: node needs to be updated.

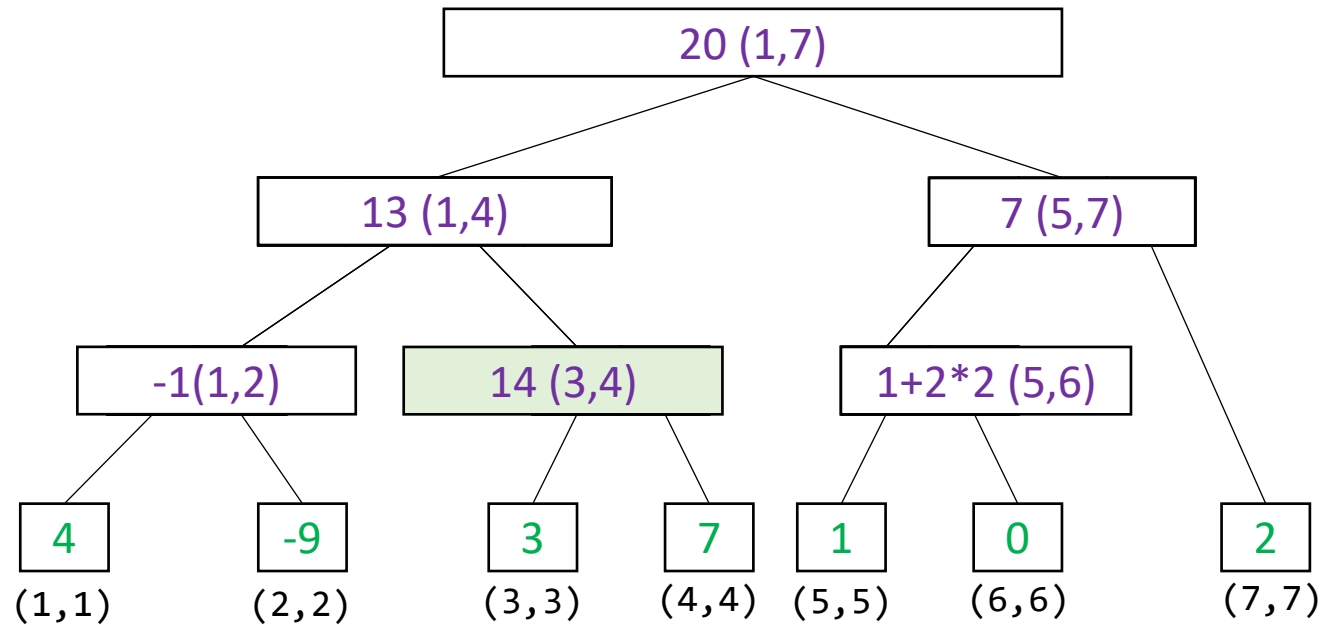
Node updated, lazy values passed to children.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[3,4]: node needs to be updated.

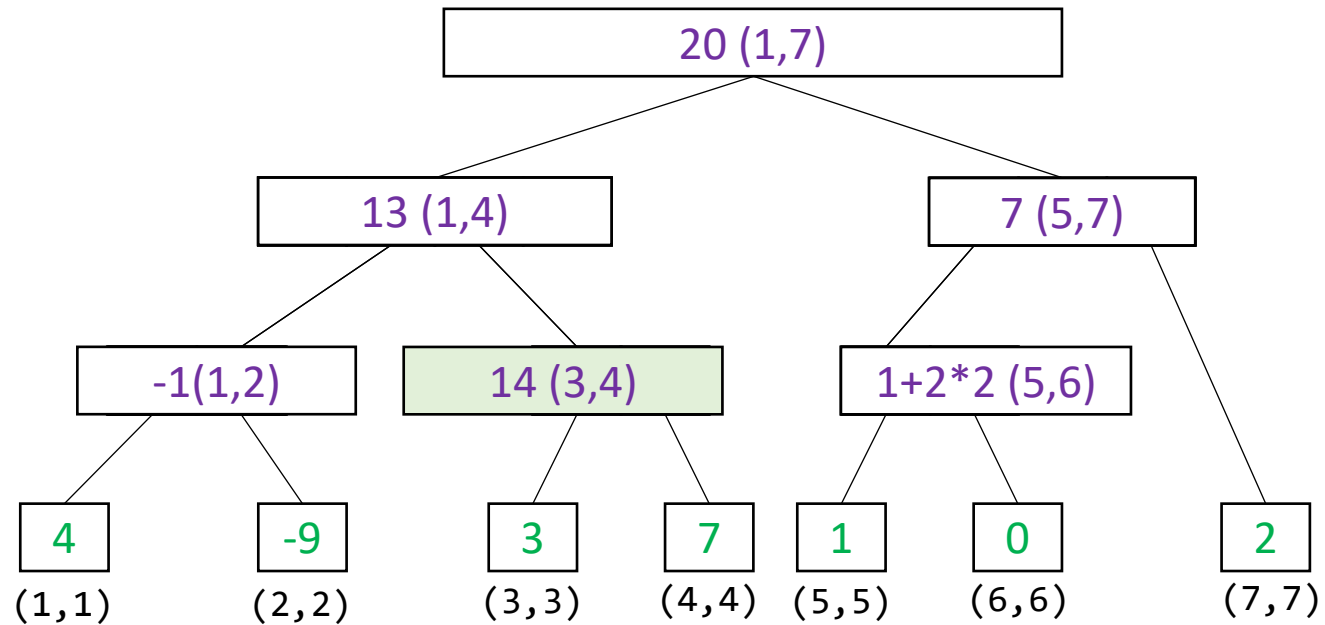
Node updated, lazy values passed to children.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[3,4]:

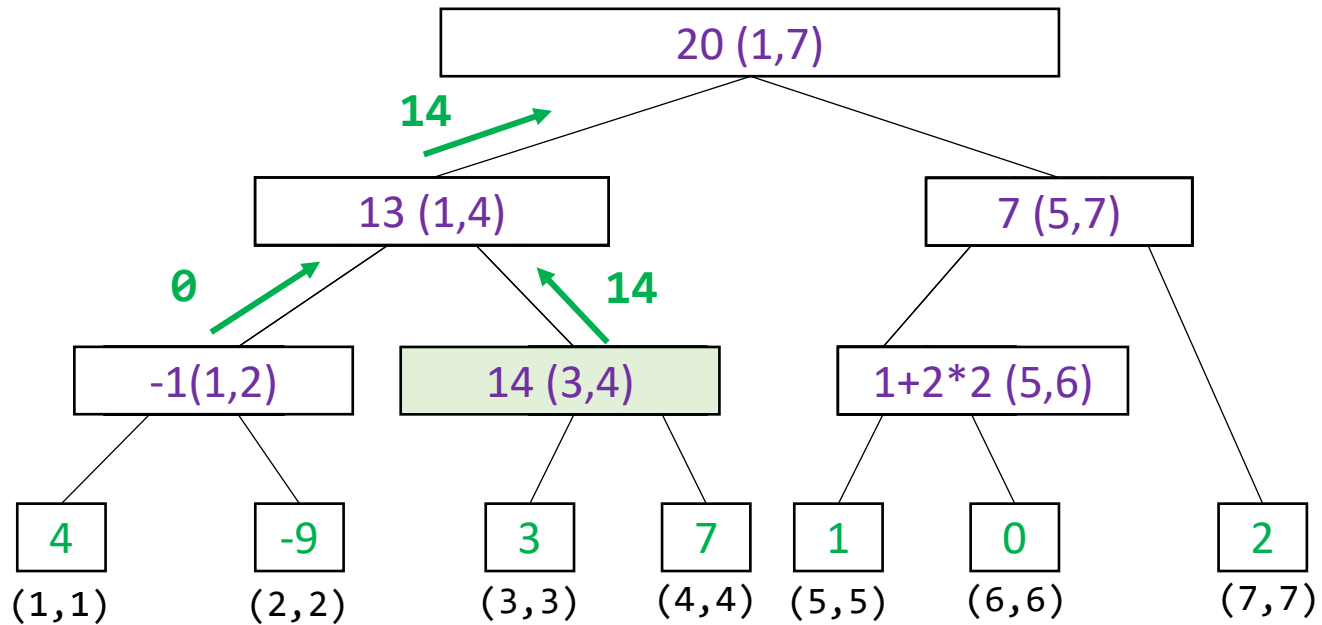
Full overlap. Return value.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

[3,4]:

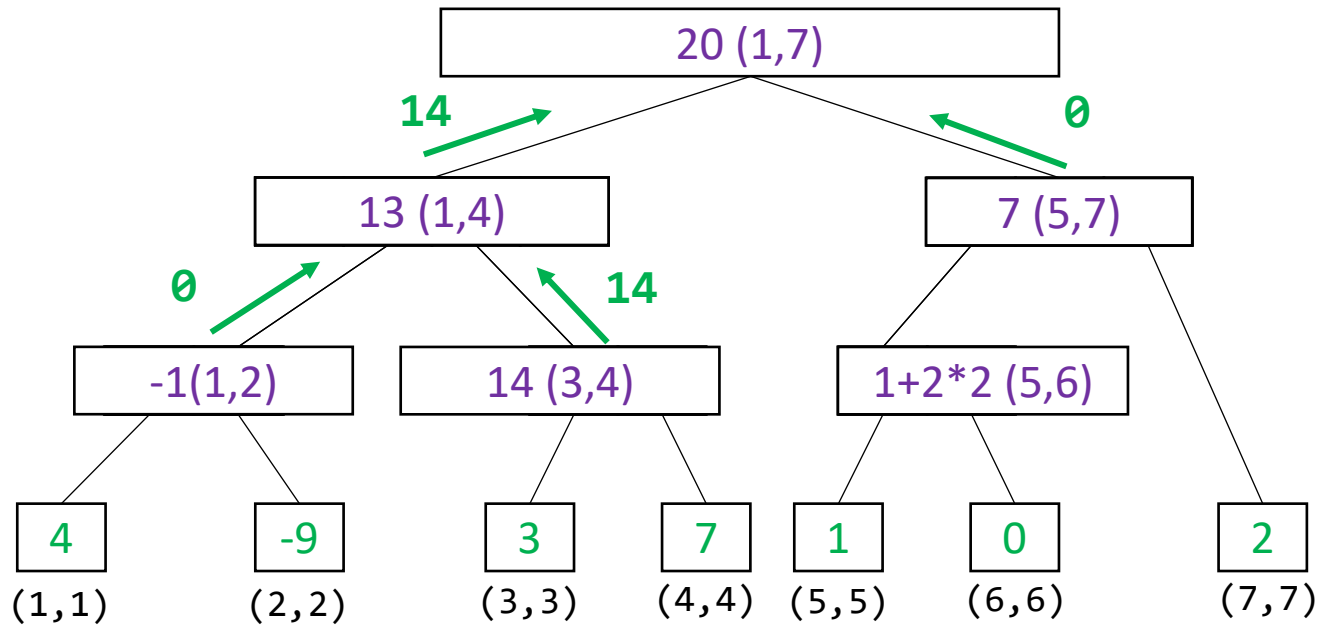
Full overlap. Return value.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

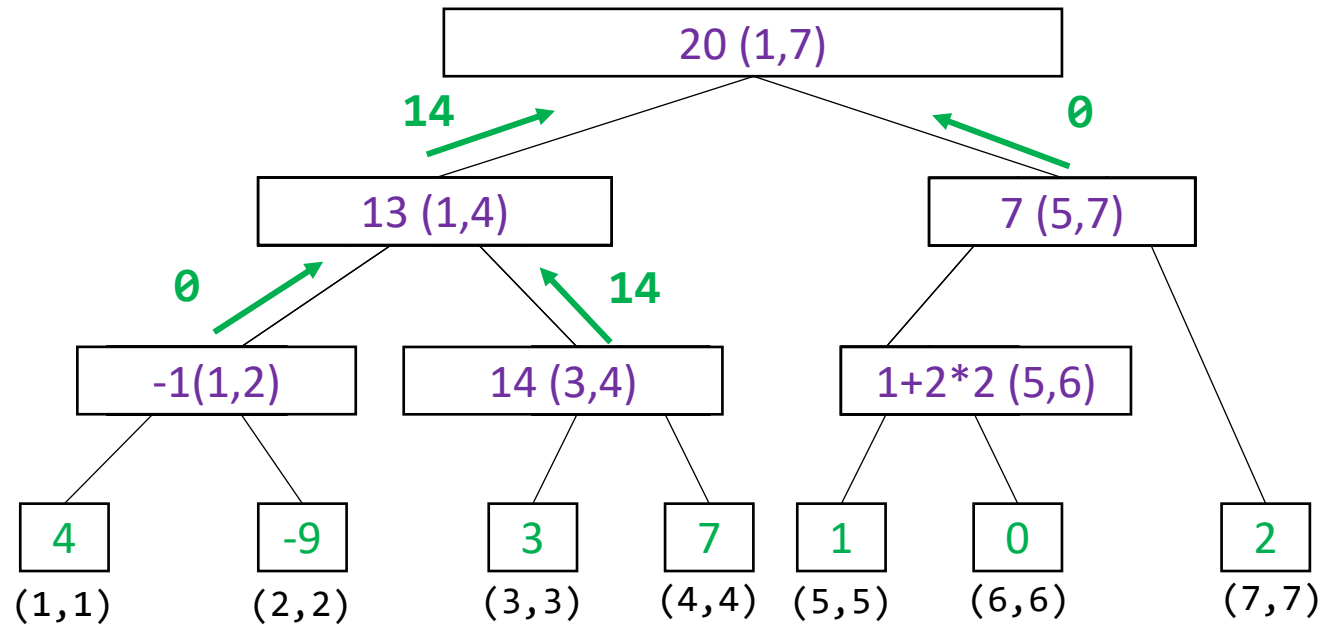
[5,7]: Node updated. No overlap.
Return 0.

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13



Task: return sum of range [3,4]

Final result: 14

Segment Tree

8	5	3	-5	10	1	2	4	-9	3	7	1	0
1	2	3	4	5	6	7	8	9	10	11	12	13

Lazy tree

0	0	0	0	0	0	0	2	2	2	2	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13

Thank you